

Table of Contents

1. [Object and link types](#) • [Types reference](#) • [Palantir](#)
2. [Object and link types](#) • [Types reference](#) • [Palantir](#)
3. [Object and link types](#) • [Object types](#) • [Overview](#) • [Palantir](#)
4. [Object and link types](#) • [Object types](#) • [Create an object type](#) • [Palantir](#)
5. [Object and link types](#) • [Object types](#) • [Edit object types](#) • [Palantir](#)
6. [Object and link types](#) • [Object types](#) • [Enable Gotham integration](#) • [Palantir](#)
7. [Object and link types](#) • [Object types](#) • [Create Ontology objects from Gaia](#) • [Palantir](#)
8. [Object and link types](#) • [Object types](#) • [Metadata reference](#) • [Palantir](#)
9. [Object and link types](#) • [Properties](#) • [Overview](#) • [Palantir](#)
10. [Object and link types](#) • [Properties](#) • [Edit object type properties](#) • [Palantir](#)
11. [Object and link types](#) • [Properties](#) • [Add value formatting](#) • [Palantir](#)
12. [Object and link types](#) • [Properties](#) • [Add conditional formatting](#) • [Palantir](#)
13. [Object and link types](#) • [Properties](#) • [Metadata reference](#) • [Palantir](#)
14. [Object and link types](#) • [Properties](#) • [Edit-only properties](#) • [Palantir](#)
15. [Object and link types](#) • [Properties](#) • [Required properties](#) • [Palantir](#)
16. [Object and link types](#) • [Properties](#) • [Mandatory control properties](#) • [Palantir](#)
17. [Object and link types](#) • [Properties](#) • [Base types](#) • [Palantir](#)
18. [Object and link types](#) • [Properties](#) • [Property reducers](#) • [Palantir](#)
19. [Object and link types](#) • [Properties](#) • [Derived properties](#) • [Palantir](#)
20. [Object and link types](#) • [Structs](#) • [Overview](#) • [Palantir](#)
21. [Object and link types](#) • [Structs](#) • [Create a struct type](#) • [Palantir](#)
22. [Object and link types](#) • [Structs](#) • [Edit struct types](#) • [Palantir](#)
23. [Object and link types](#) • [Structs](#) • [Automapping struct types](#) • [Palantir](#)
24. [Object and link types](#) • [Structs](#) • [Structs and shared properties](#) • [Palantir](#)
25. [Object and link types](#) • [Structs](#) • [Designate struct main fields](#) • [Palantir](#)
26. [Object and link types](#) • [Shared properties](#) • [Overview](#) • [Palantir](#)
27. [Object and link types](#) • [Shared properties](#) • [Create shared properties](#) • [Palantir](#)
28. [Object and link types](#) • [Shared properties](#) • [Edit shared properties](#) • [Palantir](#)
29. [Object and link types](#) • [Shared properties](#) • [Use shared properties on object types](#) • [Palantir](#)
30. [Object and link types](#) • [Shared properties](#) • [Metadata reference](#) • [Palantir](#)
31. [Object and link types](#) • [Link types](#) • [Overview](#) • [Palantir](#)
32. [Object and link types](#) • [Link types](#) • [Create a link type](#) • [Palantir](#)
33. [Object and link types](#) • [Link types](#) • [Edit link types](#) • [Palantir](#)
34. [Object and link types](#) • [Link types](#) • [Metadata reference](#) • [Palantir](#)
35. [Object and link types](#) • [Allow users to edit objects and links](#) • [Palantir](#)
36. [Object and link types](#) • [Value types](#) • [Overview](#) • [Palantir](#)
37. [Object and link types](#) • [Value types](#) • [Create a value type](#) • [Palantir](#)
38. [Object and link types](#) • [Value types](#) • [Use value types](#) • [Palantir](#)
39. [Object and link types](#) • [Value types](#) • [Value type versions](#) • [Palantir](#)

Ontology building / Object and link types / Types reference

Types reference

When you define your Ontology, you can use a wide variety of types to represent real-world definitions of the data you brought into Foundry. The types used in Foundry are categorized as *Ontology types* or *data types*:

- **Ontology types** are used to model a real-world domain into an Ontology.
- **Data** types are used to represent data values. Data types in Foundry are inspired by similar concepts in [RDF ↗](#), [OWL ↗](#) and [XSD ↗](#).

Ontology resources

The following types are available to build and define your Ontology.

Object type

An **object type** is a schema definition of a real-world entity or event, comprised of individual objects. For example, both `JFK` and `LHR` can be objects of an `Airport` object type.

[Learn more about object types.](#)

Property

A **property** of an object type is a characteristic that informs a real-world entity or event. For example, if `LHR` is an object of the `Airport` object type, `name` and `country` are properties of `LHR` object, the property values would be the following:



Send feedback

- **country:** United Kingdom

[Learn more about properties.](#)

Shared property

A **shared property** is a property that can be used on multiple object types in your Ontology. Shared properties allow for consistent data modeling across object types and centralized management of property metadata.

[Learn more about shared properties.](#)

Link type

A **link type** is the schema definition of a relationship between two object types. A **link** refers to a single instance of that relationship between two objects.

[Learn more about link types.](#)

Action type

An **action type** is the schema definition of a set of changes or edits to objects, property values, and links that a user can make all at once. Action types also include the side effect behaviors that happen when an Action occurs. Once an action type is configured in the Ontology, end users can make changes to objects by applying Actions.

[Learn more about action types.](#)

Object type groups

Object type groups are a classification primitive that helps users better search and explore their ontology.

[API Reference ↗](#) [Object type groups.](#)

Interfaces

Send feedback

An **interface** is an Ontology type that describes the shape of an object type and its capabilities. Interfaces provide object type polymorphism, allowing for consistent modeling of and interaction with object types that share a common shape.

Learn more about [interfaces](#).

Difference between object types and objects

To clarify the difference between object types and objects, examples are provided below. Note that the same distinction applies to link types and links.

Object type definitions

Object type definitions, sometimes just referred to as "object types", refer to type-level information about ontology entities such as object types, link types, and action types. For example, the metadata for an object type may include display name, property names, property data types, and description. Metadata does not refer to the actual data or values of an object type's properties or primary key; these are considered ontology data.

Object instances

Object instances, sometimes just referred to as "objects", are the actual primary key and property values for specific instances of an ontology entity. For example, an `Airplane` object type can have an object instance with a `Plane ID` property having the value `my_plane_id1`, and a `Maximum Occupancy` property having value `240`.

Value types

Value types are semantic wrappers around a field type comprised of metadata and constraints that can enhance type safety, improve expressiveness, and provide additional context. Value types encapsulate domain-specific data types and enforce data validation in a consistent manner across the platform. Commonly used value types include email addresses, URLs, UUIDs, and enumerations.

[API Reference ↗](#)

[Send feedback](#)

While field types and base types are defined statically, value types are customized within the context of a given [space](#). As a result, users cannot create new field types or base types but are able to create **value types** dynamically.

[Learn more about value types.](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

Ontology building / Object and link types / Types reference

Types reference

When you define your Ontology, you can use a wide variety of types to represent real-world definitions of the data you brought into Foundry. The types used in Foundry are categorized as *Ontology* types or *data* types:

- **Ontology types** are used to model a real-world domain into an Ontology.
- **Data** types are used to represent data values. Data types in Foundry are inspired by similar concepts in [RDF ↗](#), [OWL ↗](#) and [XSD ↗](#).

Ontology resources

The following types are available to build and define your Ontology.

Object type

An **object type** is a schema definition of a real-world entity or event, comprised of individual objects. For example, both `JFK` and `LHR` can be objects of an `Airport` object type.

[Learn more about object types.](#)

Property

A **property** of an object type is a characteristic that informs a real-world entity or event. For example, if `LHR` is an object of the `Airport` object type, `name` and `country` are properties of `LHR` object, the property values would be the following:



Send feedback

- **country:** United Kingdom

[Learn more about properties.](#)

Shared property

A **shared property** is a property that can be used on multiple object types in your Ontology. Shared properties allow for consistent data modeling across object types and centralized management of property metadata.

[Learn more about shared properties.](#)

Link type

A **link type** is the schema definition of a relationship between two object types. A **link** refers to a single instance of that relationship between two objects.

[Learn more about link types.](#)

Action type

An **action type** is the schema definition of a set of changes or edits to objects, property values, and links that a user can make all at once. Action types also include the side effect behaviors that happen when an Action occurs. Once an action type is configured in the Ontology, end users can make changes to objects by applying Actions.

[Learn more about action types.](#)

Object type groups

Object type groups are a classification primitive that helps users better search and explore their ontology.

[API Reference ↗](#) [Object type groups.](#)

Interfaces

Send feedback

An **interface** is an Ontology type that describes the shape of an object type and its capabilities. Interfaces provide object type polymorphism, allowing for consistent modeling of and interaction with object types that share a common shape.

Learn more about [interfaces](#).

Difference between object types and objects

To clarify the difference between object types and objects, examples are provided below. Note that the same distinction applies to link types and links.

Object type definitions

Object type definitions, sometimes just referred to as "object types", refer to type-level information about ontology entities such as object types, link types, and action types. For example, the metadata for an object type may include display name, property names, property data types, and description. Metadata does not refer to the actual data or values of an object type's properties or primary key; these are considered ontology data.

Object instances

Object instances, sometimes just referred to as "objects", are the actual primary key and property values for specific instances of an ontology entity. For example, an `Airplane` object type can have an object instance with a `Plane ID` property having the value `my_plane_id1`, and a `Maximum Occupancy` property having value `240`.

Value types

Value types are semantic wrappers around a field type comprised of metadata and constraints that can enhance type safety, improve expressiveness, and provide additional context. Value types encapsulate domain-specific data types and enforce data validation in a consistent manner across the platform. Commonly used value types include email addresses, URLs, UUIDs, and enumerations.

[API Reference ↗](#)

[Send feedback](#)

While field types and base types are defined statically, value types are customized within the context of a given [space](#). As a result, users cannot create new field types or base types but are able to create **value types** dynamically.

[Learn more about value types.](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

Object types

An **object type** is the schema definition of a real-world entity or event. An **object or object instance** refers to a single instance of an object type; an object corresponds to a single real-world entity or event. An **object set** refers to a collection of multiple object instances; that is, an object set represents a group of real-world entities or events.

For example, in the Ontology Manager, you may create an `Employee` object type that defines the characteristics for “All employees” or all objects of that type. An object refers to a single instance of the `Employee` object type, like the notional employees “Melissa Chang”, “Akriti Patel”, or “Diego Rodriguez.” A group of objects like “All tenured employees” represents an object set.

Similarly, in the Ontology Manager, you may create a `Flight` object type that defines characteristics for “All flights” or all objects of that type. An object refers to a single instance of the `Flight` object type, like “JFK → SFO 2021-02-24” or “TLV → LHR 2020-04-16.” A group of objects like “All arrived flights” represents an object set.

The concepts underpinning the Ontology have analogous concepts in the structure of a dataset. The definition of an object type in the Ontology is analogous to that of a dataset, while the definition of an object is analogous to that of a row in the dataset. The definition of an object set is analogous to a filtered set of rows in a dataset. For example, an `Employee` dataset may define the schema for “All employee rows.” In this case, a single row refers to a single employee, like “Melissa Chang”, “Akriti Patel,” or “Diego Rodriguez.” If you filter the dataset based on tenure, you will have a set of rows that represent “All tenured employees.”

Rather than being an abstract data model, the Foundry Ontology maps each ontological

concept to the organization's actual data, enabling this data asset to power real-world applications. Objects are created and displayed in user applications by adding backing data to an object type in the Ontology Manager. To create objects, you can use the

API Reference ↗

Send feedback

an organization will add backing datasources to the `Employee` object type and connect their employee directory and other enterprise data into the Ontology.

Get started by learning how to [create an object type](#).

PREVIOUS
← [Types reference](#)

NEXT
[Create an object type](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Object types / Create an object type

Create an object type

The primary way to create and configure a new object type is with a **guided step-by-step helper**. The guided helper is the recommended method, but if you exit the helper before completing the object creation process, you can also **manually** complete the process by specifying the metadata, backing datasource, property mappings, and keys (primary and title) for the new object type.

After creating a new object type, you can change the API name from the assigned default.

This page also contains troubleshooting information on the new object type creation process.

i If you want a programmatic way of creating an object type, use a SuperRepo. SuperRepos define object types in code with Ontology-as-code, and object types created in the Ontology Manager can be imported into a SuperRepo.

Create a new object type with the helper

- Create a new object type
- Choose a backing datasource
- Object type metadata
- Create properties for the object type
- Configure the primary key and title key

API Reference ↗

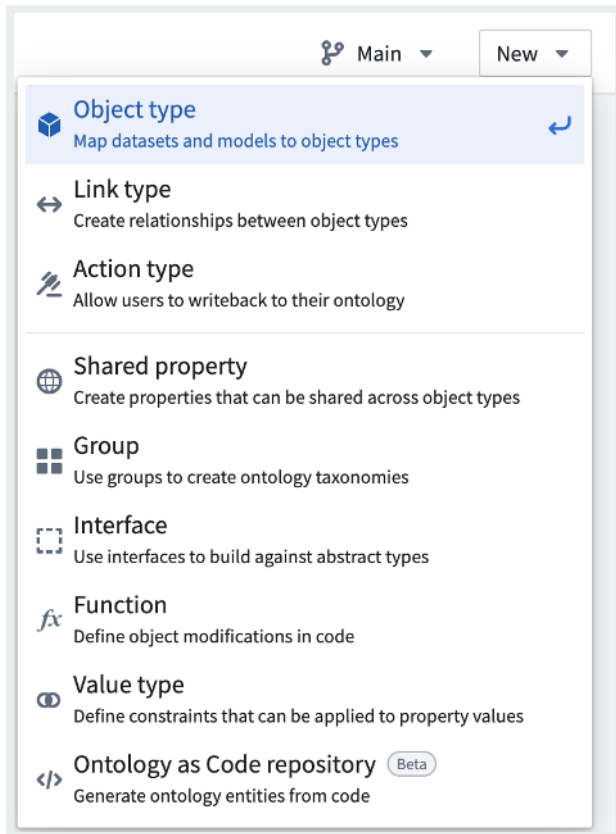
Save location

☰ 🔍 Change to ontology

Send feedback

Create a new object type

To create a new object type, select the **Create your first object type** option from the Ontology Manager homepage or choose **New > Create object type** located on top right on the same page.



The **Create new object type** helper will appear.

[API Reference ↗](#)

[Send feedback](#)

Create a new object type

1 Datasource

2 Metadata

3 Properties

4 Actions

5 Save location

STEP 1

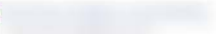
Object type backing

Datasource

 Continue without datasource
Generate a dataset for permissions purposes

 Use existing datasource
Select a preexisting Foundry datasource ✓

Select a datasource to back this object type

Replace

 Instance permission

Next

Choosing a backing datasource

If you have an existing datasource in Foundry containing data to back the object type then you can select it. This will automatically populate the object type's metadata. It will also map every column of the backing datasource to a property, but you can discard added properties in the **Properties** step.

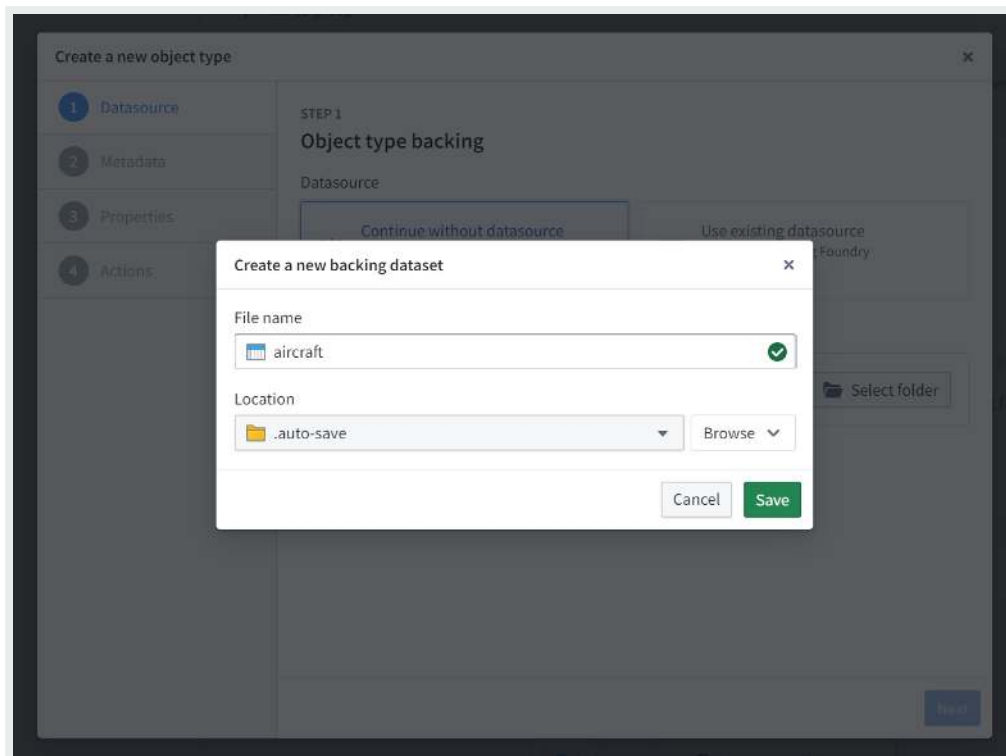
Warning

A backing datasource for an object type may not contain `MapType` or `StructType` columns.

If you do not have an existing datasource containing data for the object type, you can choose to continue without an existing datasource and select a location to generate a dataset for permissions. This option is not available if you are using Object Storage v1. As permissions for all objects of a type are determined by the location of their backing dataset, you will be prompted to choose a location to which you want to save an empty dataset.

[API Reference ↗](#)

Send feedback



Object type metadata

In this step, provide the following information regarding your new object type:

- **Icon:** Select the default icon to customize the icon and color of the object type; this icon and color will be displayed in user applications when a user views an object of this type.
- **Name:** The name shown to anyone accessing an object of this type in user applications.
- **Plural name:** The name shown to anyone accessing multiple objects of this type in user applications.
- **Description:** Explanatory text for anyone accessing the objects of this type in user applications. For example, users searching in Object Explorer will view the description of the object type in their search results.
- **Groups:** Choose whether this object type will be part of any groups. This is a mechanism for organizing your ontology, making it easier to filter for the object types you want to work with later.

[API Reference ↗](#)

[Send feedback](#)

Create a new object type [X]

STEP 2
Configure object type metadata

Icon: [Icon] Name: [Aircraft] [X] Plural name: [Aircraft] [X]

Description: [A database of aircraft] [X]

Groups: [Add to group]

[Back] [Next]

Create properties for the object type

In the third step of the dialog you can customize which properties the object type will have. If you have chosen an existing Foundry datasource, any columns will be mapped automatically, but can be discarded during this step.

Every object type requires at least one property. This is because object types need a primary key to uniquely identify them. The wizard allows you to add any other desired properties.

Note that property types that require advanced configuration, such as media, cannot be generated as part of the bootstrapping wizard and must be added after you have exited it.

[API Reference ↗](#)

[Send feedback](#)

Configure the primary key and title key

As part of the **Properties** step you need to choose a primary key and title key:

- **Title key:** The property that acts as a display name for objects of this type.
 - For example, selecting the `full name` property as the title key of the `Employee` object type would use the values of that property, such as “Melissa Chang”, “Akriti Patel”, or “Diego Rodriguez” as the display names for each respective notional `Employee` object.
- **Primary key:** The property that acts as a unique identifier for each instance of an object type. Each row in the backing datasource must have a different value for this property.
 - For example, the value of the `employee ID` property will be used to identify “Melissa Chang” as a unique employee within the organization.

A list of supported property types can be found in the [object type properties documentation](#).

API Reference ↗

Send feedback

Create a new object type

1 Datasource

2 Metadata

3 Properties

4 Actions

5 Save location

STEP 3

Create properties

Primary key ?

Title ?

” Aircraft id

” Display name

Source	Property
> primary-key	” Aircraft id Primary key
User input / actions	” Owner id
User input / actions	Date of manufacture
User input / actions	” Display name Title

+ Add property

Back

Next

⚠ Be sure to check your backing datasources for duplicates before assigning a primary key. The primary key you select must be unique for every record in the datasource. If your ontology is using [Object Storage v2](#), a duplicate primary key will cause [Funnel batch pipeline](#) errors leading to a build failure. If you are using Object Storage v1 (Phonograph), an update will appear as successful; however, the duplicate primary keys can cause unexpected changes to your ontology.

⚠ Primary keys should be deterministic. If the primary key is non-deterministic and changes on build, edits can be lost and links may disappear. Edits can be lost because ontology edits are associated with the primary key of the object. Links between objects can disappear if builds are not coordinated to update link IDs. To ensure deterministic primary keys, you should define pipeline logic such that the primary key is the function of either a single column or multiple columns. Avoid using numbered row or random key generation, since these can cause primary keys to change between build runs.

API Reference ↗

Generate actions

Send feedback

You can optionally generate a standard set of actions to edit objects of this type and assign a specific user or group that can run them.

Note that you can still make edits to these actions or create new additional actions even if you have finalized the object type and exited the helper.

The screenshot shows a web interface for creating a new object type. On the left is a sidebar with five steps: 1 Datasource, 2 Metadata, 3 Properties, 4 Actions (highlighted in blue), and 5 Save location. The main area is titled 'STEP 4 Generate action types'. It contains a section 'Select action types to generate' with three items: 'Create Aircraft' (checked), 'Modify Aircraft' (checked), and 'Delete Aircraft' (checked). Below this is a section 'Choose who can submit this action type' with three radio buttons: 'Organization' (selected), 'Group', and 'User'. At the bottom right are 'Back' and 'Create' buttons.

Create a new object type

1 Datasource

2 Metadata

3 Properties

4 Actions

5 Save location

STEP 4

Generate action types

Select action types to generate ?

- ☒ **Create Aircraft**
Set Aircraft id, Owner id, Date of manufacture, and 1 more property
- ☒ **Modify Aircraft**
Modify Owner id, Date of manufacture, and Display name
- ☒ **Delete Aircraft**
Allows deleting object instances and all of their properties

Choose who can submit this action type

☒ Organization

☐ Group

☐ User

Back Create

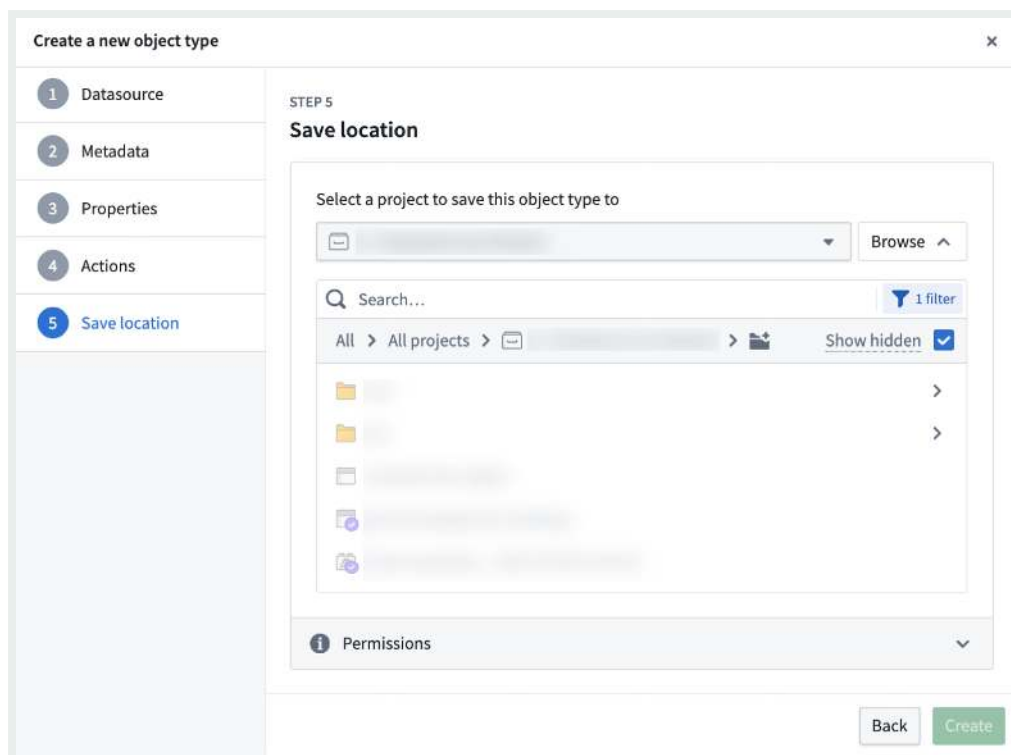
This step is not available if you are using Object Storage v1.

Save location

In the final step, choose a project to save this object type to. Then, select **Create**. Selecting **Create** will only stage your changes and **will not save** them.

[API Reference ↗](#)

[Send feedback](#)



Save change to ontology

Back in Ontology Manager, select **Save** in the upper right corner to [make the change to your ontology](#).

Create a new object type manually

When creating a new object type with the helper, it is possible to select **Create** before completing all the steps in the [Create a new object type helper instructions](#) above. Selecting **Create** before the process is complete will exit the helper and bring you to the **Overview** page.

At this point, the object type is unsaved and cannot be saved until all the steps below have been completed. The steps for completing the creation process manually (outside the **Create a new object type** helper) are described below:

API Reference ↗

[Add metadata to a new object type](#)

- [Add a backing datasource to a new object type](#)
- [Add a new property](#)

Send feedback

- [Map a single property to data](#)
- [Map all unmapped columns as new properties](#)
- [Configure the primary key and title key](#)

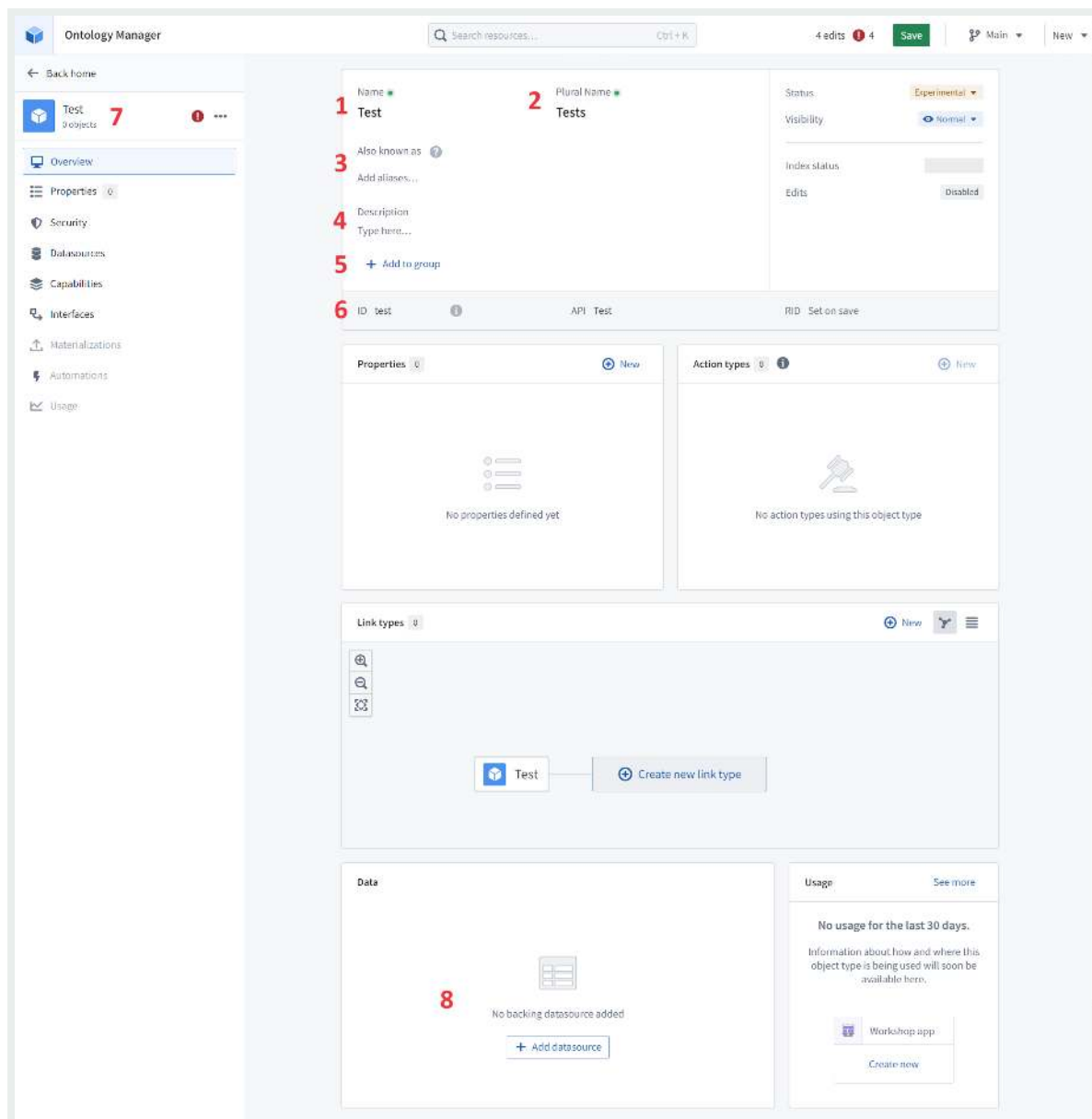
Add metadata for a new object type

On the **Overview** page's metadata section, you can edit the object type's display name, plural display name, description, and ID:

1. **Display name:** The name shown to anyone accessing an object of this type in user applications.
2. **Plural display name:** The name shown to anyone accessing multiple objects of this type in user applications.
3. **Aliases:** Additional terms that will find this object type when users search for them.
4. **Description:** Explanatory text for anyone accessing the objects of this type in user applications. For example, users searching in Object Explorer will view the description of the object type in their search results.
5. **Groups:** One or more labels that help categorize the object type.
6. **ID:** A unique identifier of the object type, primarily used to reference objects of this type when configuring a user application.
 - An ID can contain lowercase letters, numbers, and dashes.
 - The first character of an ID **must** be a lowercase letter.
 - Once a property's ID is saved and the property is referenced in user applications, **any** change to the property ID will break the application.
7. **Icon:** Select the default icon from the object type view's sidebar to customize the icon and color of the object type; this icon and color will be displayed in user applications when a user views an object of this type.
8. **Backing datasource:** The source of the data used as property values for objects of this type.

[API Reference ↗](#)

[Send feedback](#)



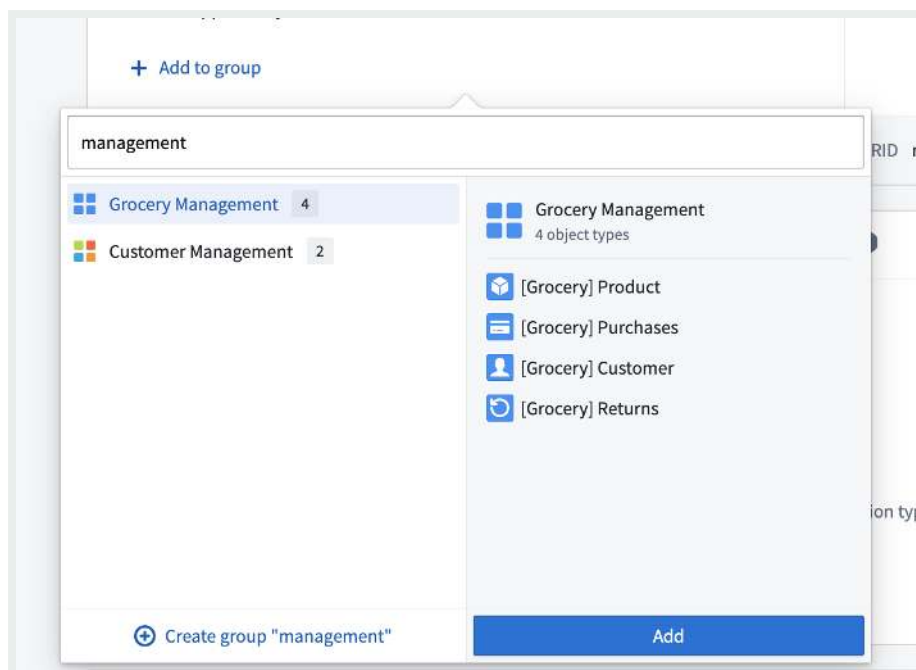
Add a group to an object type

Groups are labels that help categorize object types. From the object type metadata widget, you can:

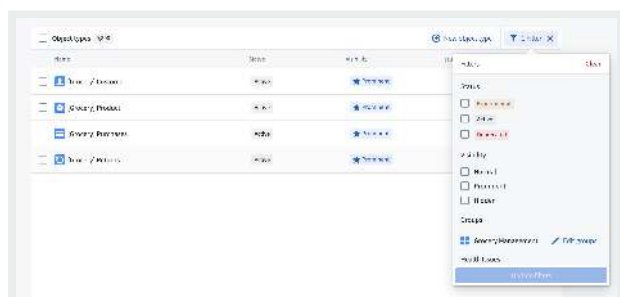
- Add a group from a list of existing groups.
- Create a new group by typing the name of that group.
- Remove a group from your object type.

API Reference ↗

Send feedback



Groups are searchable in the [Ontology Manager's Search bar](#) and [Search bar dialog](#). The table of object types in the Ontology Manager supports displaying and filtering by group. Groups are also displayed on the [Object Explorer home page](#).



⚠ Warning

Groups as labels in object type metadata replace the previous method of adding `oe_home_page_object_type_group` type class to the primary key property; this previous method is no longer available.

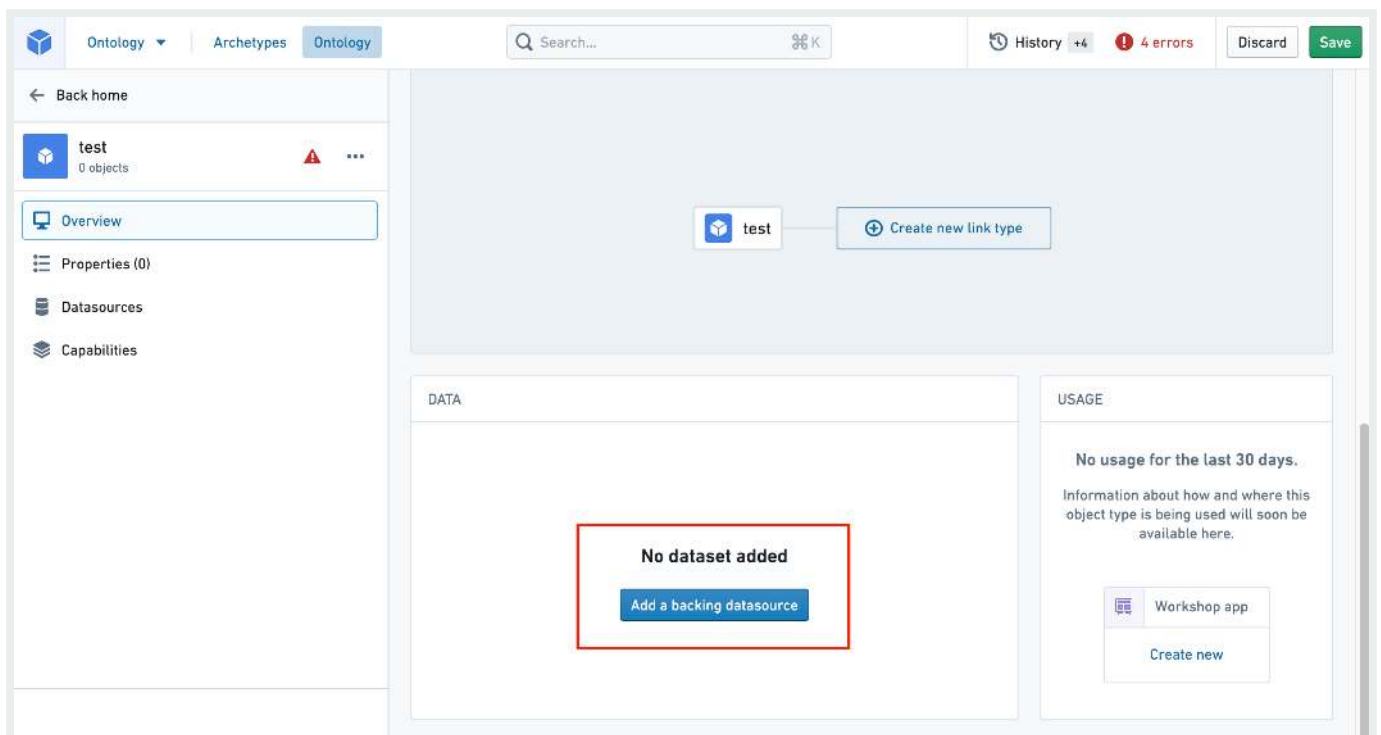
[API Reference ↗](#)

Add a backing datasource to a new object type

[Send feedback](#)

In order to populate property values for objects of this type with data, you must add a backing datasource. You can do this by:

- Navigating into the Property editor by selecting **Create new** from the **Properties** section of the **Overview** page, or by selecting the **Edit property mapping** button from the **Properties** page of the object type view's sidebar.
- Then, select the **Add a backing datasource** button as shown in the image below. This will allow you to select any of the available datasources in Foundry as a backing datasource.
 - Note that a single datasource can only be used to back one object type.

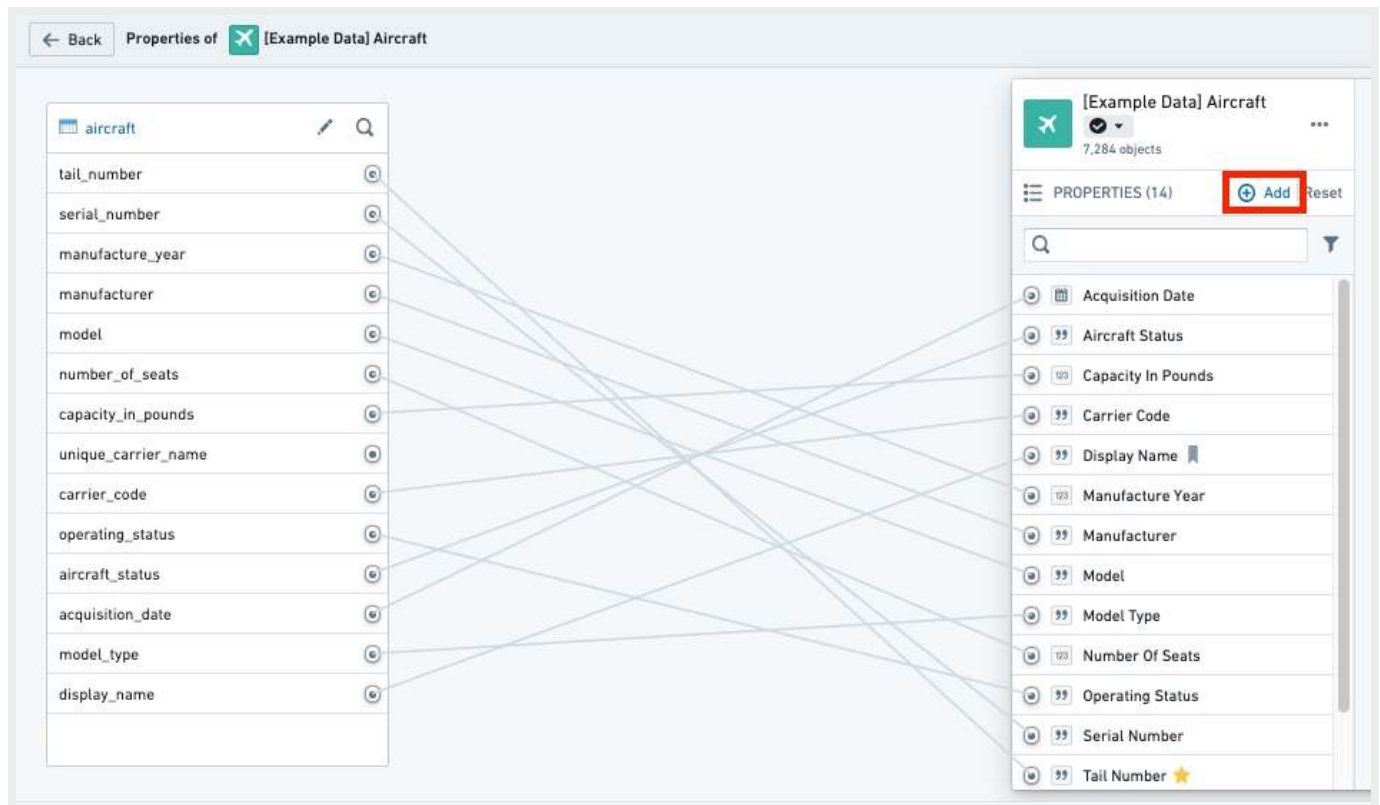


Add a new property

From within the property editor, select **Add** in the **Properties** pane on the right side of the screen. This will add a new property to the object type.

[API Reference ↗](#)

[Send feedback](#)



Map a single property to data

It is possible to map properties to columns in a backing datasource in any of the following ways:

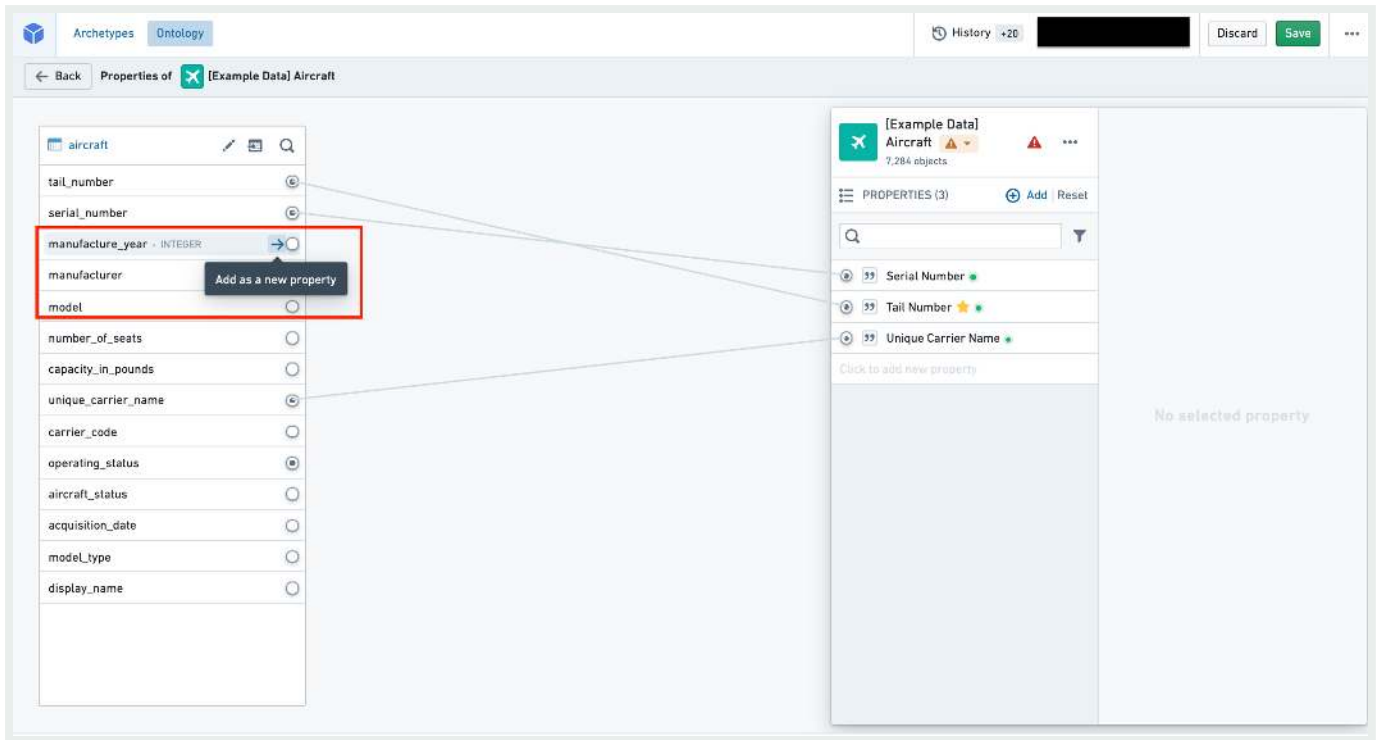
- [Map a column to a new property](#)
- [Map a column to an existing property](#)
- [Map a property to a column](#)

Map a column to a new property

In the datasource pane on the left side of the screen (see image below), you can see all of the columns of the datasource. Hover over the column you want to map and select the **Add as a new property** button to create a new property mapped to this column. The property ID, display name, and base type will be inferred from the name of the column.

[API Reference ↗](#)

[Send feedback](#)



Map a column to an existing property

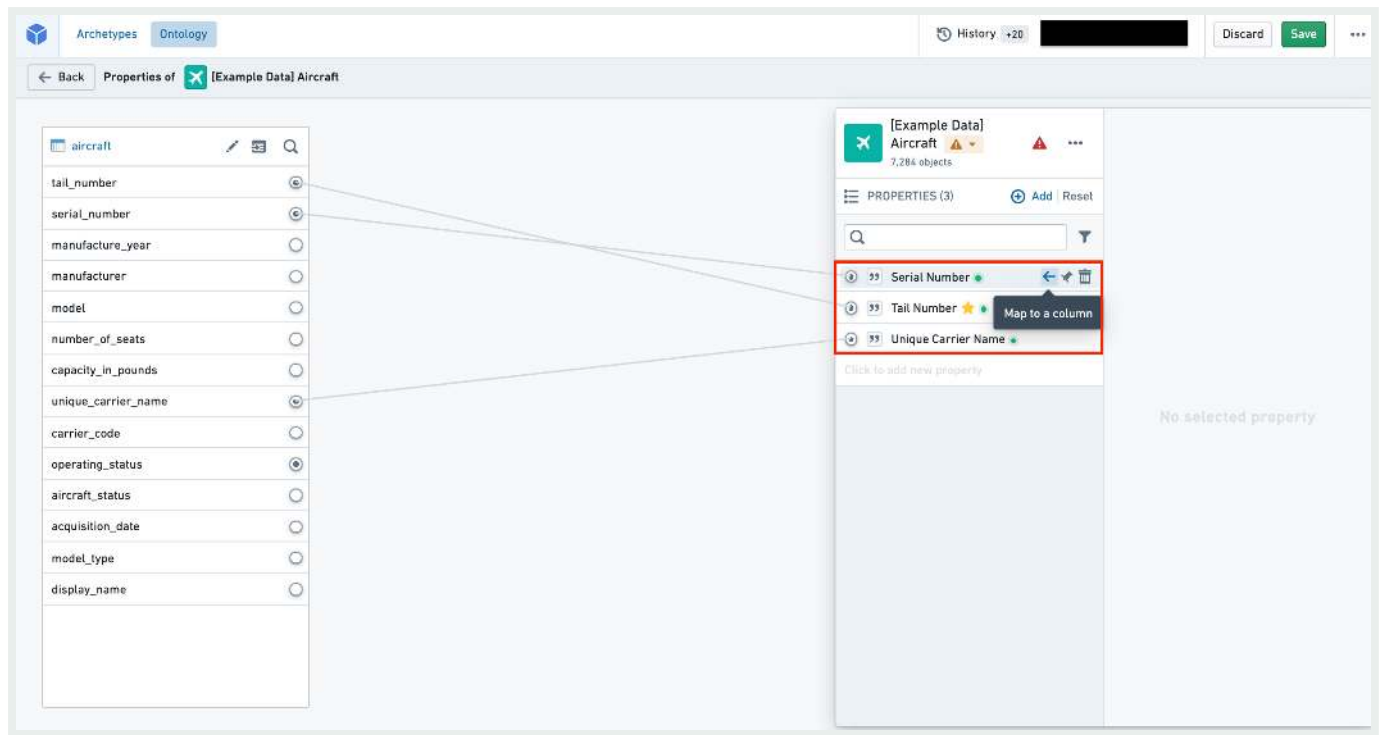
In the datasource pane on the left side of the screen, you can see all of the columns of the datasource. Hover over an unmapped column and select the **Add as a new property** button. If a property already exists with a property ID that matches the column name, the column will be mapped to the existing property.

Map a property to a column

In the properties pane on the right side of the screen, hover over the property you want to map to a column and select **Map to a column**. This will open a dropdown from which you can select the column you want to map to your property.

[API Reference ↗](#)

[Send feedback](#)



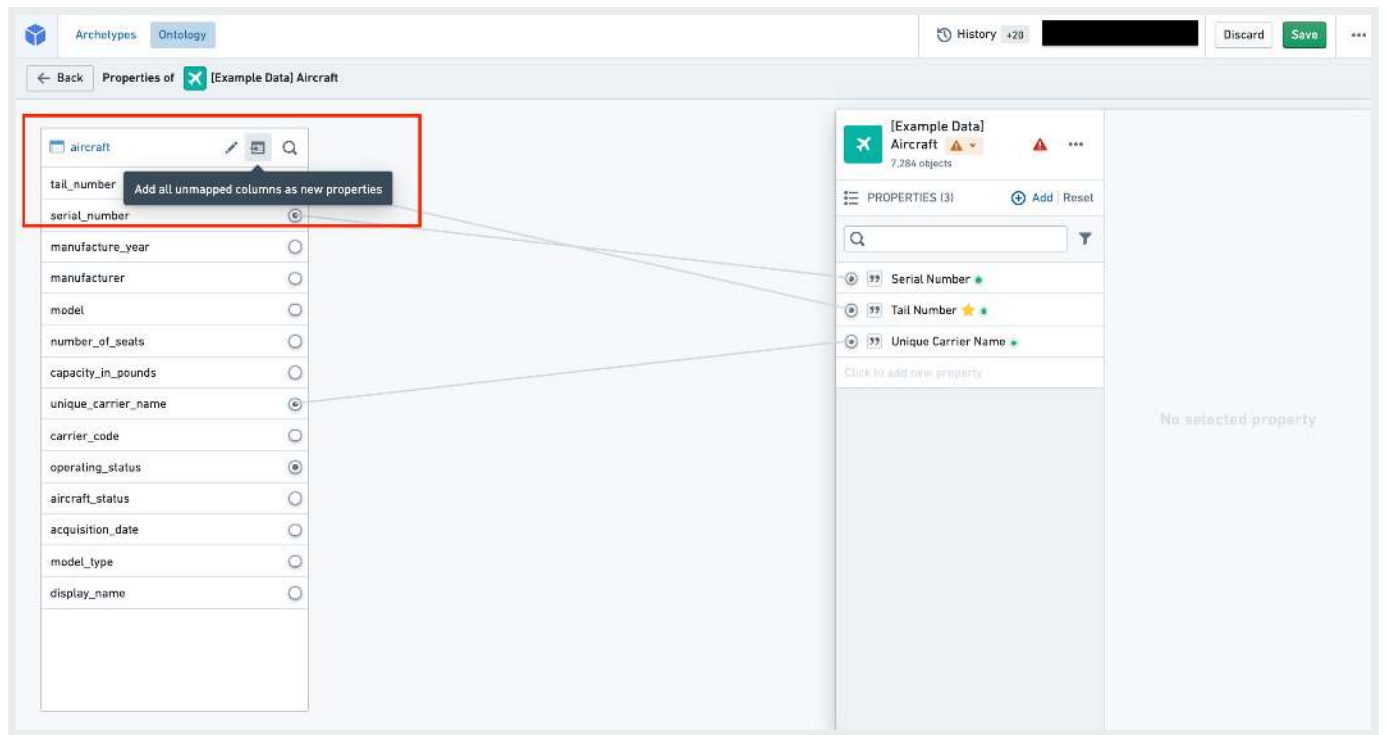
Map all unmapped columns as new properties

Next to the datasource name in the datasource pane, you will find an **Add all unmapped columns as new properties** button. Selecting the button will create properties for all the unmapped columns in the datasource. The IDs, display names, and base types of the properties will be inferred from the corresponding columns in the datasource.

- Once a property's ID is saved and the property is referenced in user applications, **any** change to the property ID will break the application.

[API Reference ↗](#)

[Send feedback](#)



Configure the primary key and title key

Now that you've created your new object type, added a backing datasource, and mapped it to new properties, you still need to configure the primary key and title key before being able to save your object type. You can navigate to the property metadata pane in the property editor (see image below) to set a property as the primary key and title key:

[API Reference ↗](#)

[Send feedback](#)

[Example Data]

Aircraft

7,284 objects

⚠️

✓

⋮

☰

PROPERTIES (14)

⊕ Add | Reset

⏏

📅 Acquisition Date

🔍 Aircraft Status

🔍 Capacity In Pounds

🔍 Carrier Code

🔍 Display Name

🔍 Manufacture Year

🔍 Manufacturer

🔍 Model

🔍 Model Type

🔍 Number Of Seats

🔍 Operating Status

🔍 Serial Number

🔍 Tail Number

🔍 Unique Carrier Name

STATUS

Experimental

API NAME

acquisitionDate

KEYS

☐ Primary key (current: Tail Number)
 ☐ Title key (current: Display Name)

DATE AND TIME FORMATTING

CONDITIONAL FORMATTING

+ Add a rule

PROPERTY BASE TYPE

date

TYPE CLASSES

+ Add new type class

RENDER HINTS

☒ Selectable

Requires resync

- Primary key:** The property that acts as a unique identifier for each instance of an object type. Each row in the backing datasource must have a different value for this property.
 - For example, the value of the `employee ID` property will be used to identify “Melissa Chang” as a unique (notional) employee within the organization.
 - To configure the primary key, select the property you want to assign to the primary key in the properties pane of the property editor and check the **Primary key** option.
 - Edits are permanently attached to the primary key value you made them for. Any time you change the primary key of an object type, you will be prompted to delete all existing edits.
- Title key:** The property that acts as a display name for objects of this type.

API Reference ↗

selecting the `full name` property as the title key of the `Employee` object type will use the values of that property, such as “Melissa Chang” “Akriti Patel” or “Diego Rodriguez” as the display names for each respective notional object.

Send feedback

- To configure the title key, select the property you want to assign to the title key in the properties pane of the property editor and check the **Title key** option. Note that until this point, your changes have been staged but **have not yet been saved**. To save your new object type to the Ontology, follow the instructions on [how to save changes to the Ontology](#).



Be sure to check your backing datasources for duplicates before assigning a primary key. The primary key you select must be unique for every record in the datasource. If your Ontology is using [Object Storage v2](#), a duplicate primary key will cause [Funnel batch pipeline](#) errors leading to a build failure. If you are using Object Storage v1 (also known as Phonograph), an update will appear as successful; however, the duplicate primary keys can cause unexpected changes to your Ontology.

Configure API names

An API name is the name used when referring to an object type or a property programmatically in code. All new object types and properties are automatically assigned API names that are inferred from their display names. [Learn more about API names](#).

You can change the automatically assigned API names as follows:

- An object type's API name can be edited in the object type's **Overview** page.
- A property's API name can be edited in the properties pane of the property editor.

Naming guidelines

Object type API names follow functional coding standards. An object type's API name must:

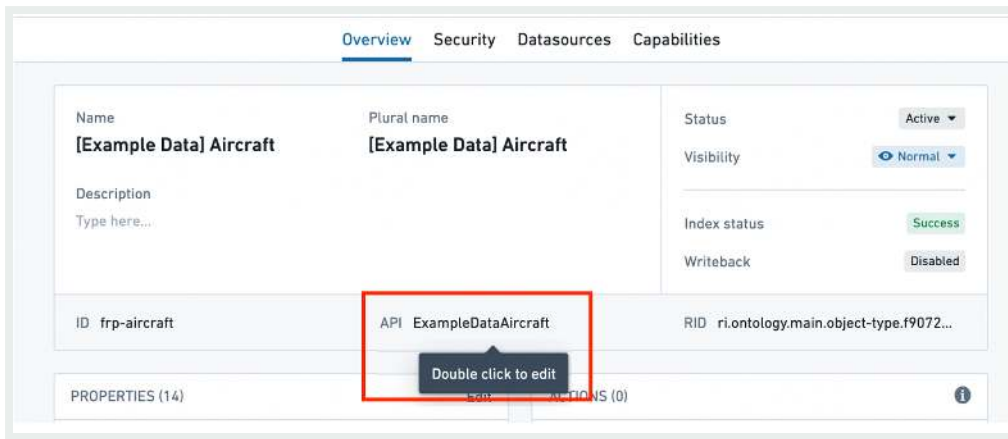
- Begin with an uppercase character and consist of only alphanumeric characters.
- Be written in PascalCase (also known as UpperCamelCase, in which the first letter of

API Reference ↗

compound word is capitalized; for instance, "ThisExampleName").

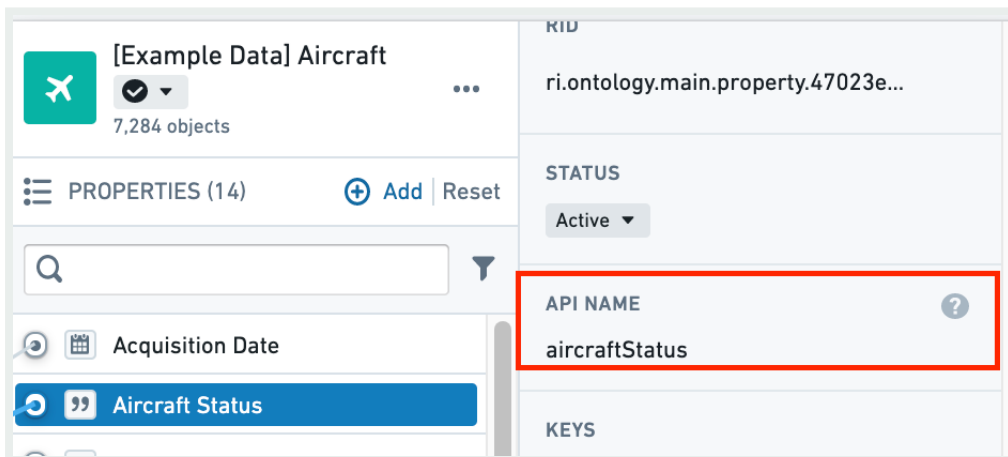
- Be unique across all object types.
- Be between 1 and 100 characters long.

Send feedback



A property's API name must:

- Begin with a lowercase character and consist of only alphanumeric characters.
- Be written in camelCase (in which the first letter of each word after the first word in a compound word is capitalized; for instance, "thisExampleName").
- Be unique across all properties belonging to the same object type.
- Be between 1 and 100 characters long.



Troubleshooting

API Reference ↗ object type fields

To save a new object type, the following object type fields must not be

Send feedback

- ID
- Display name
- Plural display name
- Backing datasource
- API name

Additionally, the following property fields must not be empty:

- Property ID
- Property display name
- Backing column
- Property API name
- Title key
- Primary key

Valid ID checklist

Object type ID

Object type IDs:

- Can be made up of lowercase letters, numbers, and dashes.
- Should start with a letter.
- Must be unique across all object types.

Property type ID

Property type IDs:

- Can be made up of lowercase or uppercase letters, numbers, dashes, and underscores.
- Should start with a letter.
- Must be unique across all properties that belong to the same object type.

[API Reference ↗](#)

In line with functional coding standards, an object type's API name must

[Send feedback](#)

- Consist of only alphanumeric characters and underscores.
- Be unique across all object types.
- Be between 1 and 100 characters long.

A property's API name must:

- Be valid Unicode.
- Be unique across all properties belonging to the same object type.
- Be between 1 and 100 characters long.

Note that there are a number of reserved keywords that cannot be used for API names. They are: `ontology`, `object`, `property`, `link`, `relation`, `rid`, `primaryKey`, `typeId`, and `ontologyObject`.

Errors

Error: `Phonograph2:DatasetAndBranchAlreadyRegistered`

If you receive the error `Phonograph2:DatasetAndBranchAlreadyRegistered`, the datasource backing the object type you are trying to save is already backing a different object type in the Ontology and cannot be used again.

PREVIOUS
← Overview

NEXT
Edit object types →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

API Reference ↗

Send feedback

Edit object types

⚠ Warning

Editing an object type and its properties can have **application-breaking consequences that can disrupt user workflows**. Read the section below on [potential breaking changes](#) **before** proceeding with any object type or property edits.

Potential breaking changes

Writeback is the process of persisting user edits on Ontology objects to a durable Foundry dataset. This dataset is known as a writeback dataset. In [Object Storage v1 \(Phonograph\)](#), you must configure a writeback dataset for an object type, or for a many-to-many link type with a join table. This is required before the object type or link type can receive user edits. Learn more about how writeback datasets compare to [materialized datasets](#) in Object Storage v2.

Object type without writeback

Changes that require Object Storage v1 (Phonograph) to unregister and reregister the backing datasources of an object type will make the objects of that type **unavailable** in user applications during that reindex time; these changes are described below.

The following changes will unregister and reregister (or delete) the backing datasources of an object type when saved:

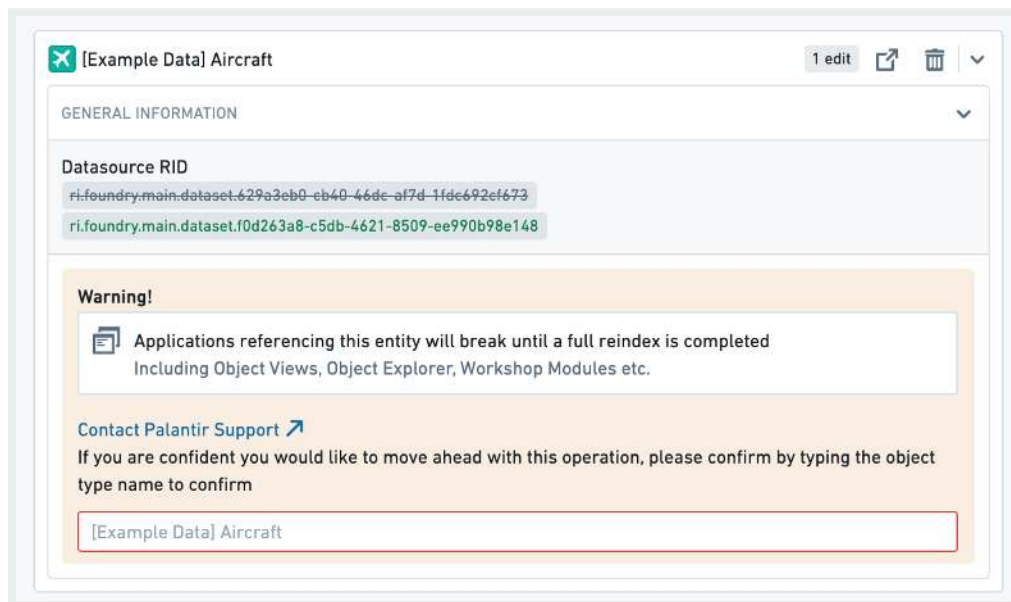


- Changing an object type's backing datasource.

Send feedback

- Changing the primary key of an object type.
- Deleting an object type.

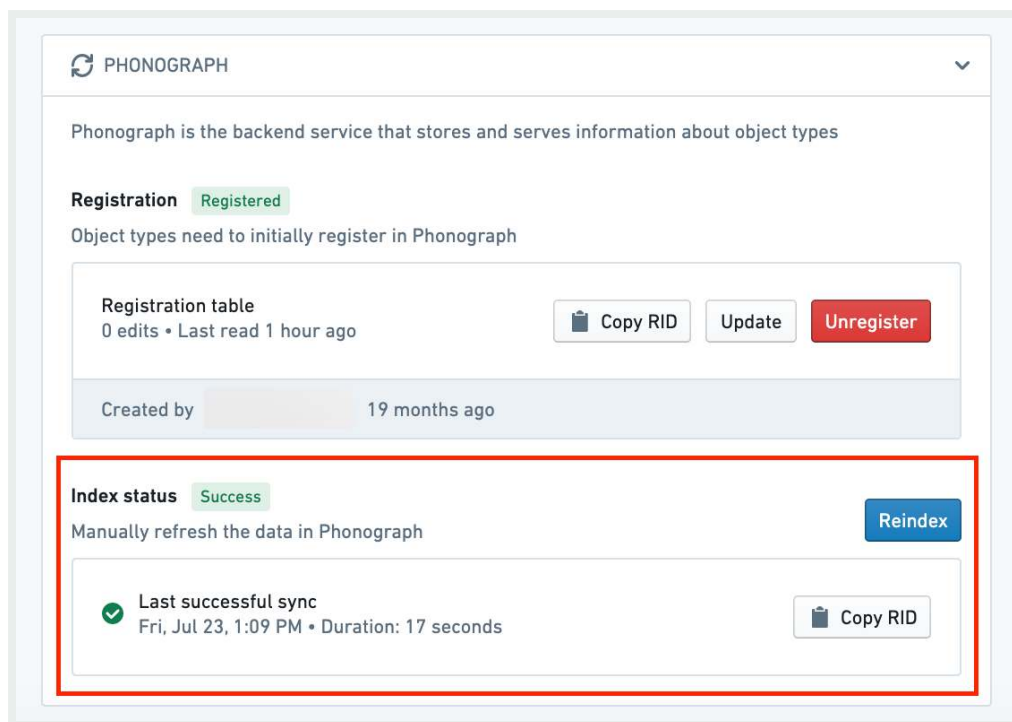
When you try to save any of these changes, you will be warned about the potential impact on user applications.



For example, if an object type is used in a Workshop application, that Workshop application will be broken until the reindex completes. You can track the progress of the reindex for an object type in the **Phonograph** pane of its **Datasources** page.

[API Reference ↗](#)

[Send feedback](#)



[Learn more about Object Storage v1 \(Phonograph\).](#)

Object type with writeback

If an object type has writeback enabled, extra precaution should be taken when making edits to that object type. The history of edits made to an object type are stored in Object Storage v1 (Phonograph). Every time a writeback dataset is built, the history of edits is reapplied to get the final state of edited objects in the writeback dataset. When the backing datasources of an object type are unregistered from Object Storage v1 (Phonograph), the history of edits in Object Storage v1 (Phonograph) is deleted and future builds of the writeback dataset will fail.

In addition to the changes that require unregistering that were listed in the [previous section](#), unregistering is required for object types with writeback when schema changes are made to **any** property of an object type that has **ever** received edits, even if it does not currently receive edits. Schema changes include changes to the ID and base type of a property.

Types that **do not** require unregistering and therefore do not risk losing the edit history.

[API Reference ↗](#)

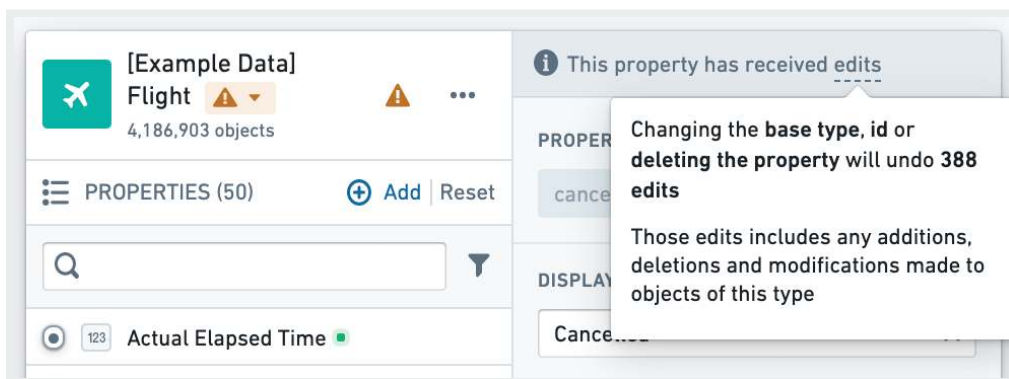
[Send feedback](#)

- Changing the display name, title key, render hints, type classes, and visibility of a property that has received edits will **not** require the object type to unregister.
- Deleting fields or making schema changes to fields that have never received edits will **not** require the object type to unregister, and therefore will not erase or undo edits on other fields that are receiving edits.

⚠ Warning

Object Storage v1 (Phonograph) will **not** automatically unregister the backing datasources of an object type in response to one of these schema changes. Instead, the reindex will fail and will only succeed if the saved schema changes are undone, or if you manually unregister and reregister the backing datasources of the object type in the **Phonograph** pane of the object type's **Datasource** page.

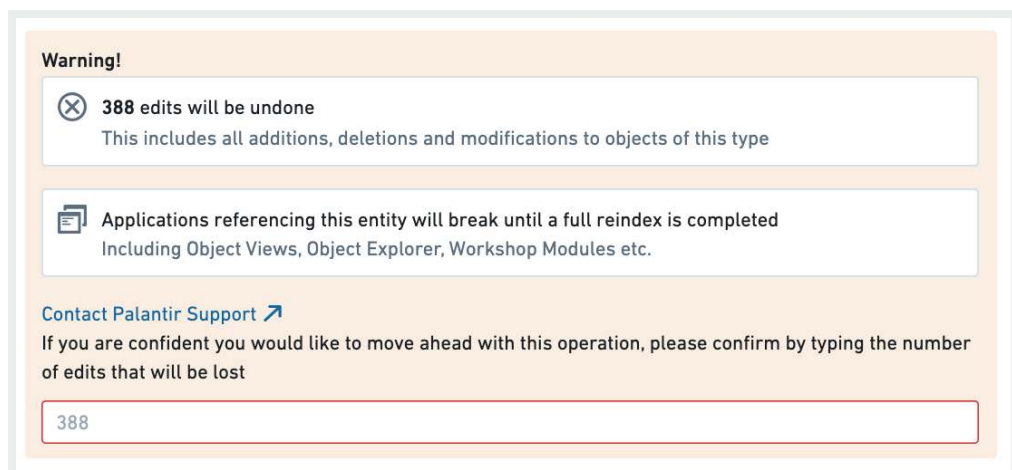
The properties pane in the property editor highlights whether a field has ever received edits.



Furthermore, when you try to save any changes that risk erasing the edit history, you will be warned about the potential impact on edits.

[API Reference ↗](#)

[Send feedback](#)



Now that you understand the considerations in editing existing object types and properties, you can safely make your changes.

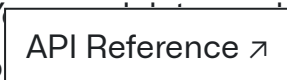
Edit an existing object type

- [Navigate to an existing object type](#)
- [Delete an object type](#)
- [Change a backing datasource](#)
- [Edit an object type's metadata](#)

Navigate to an existing object type

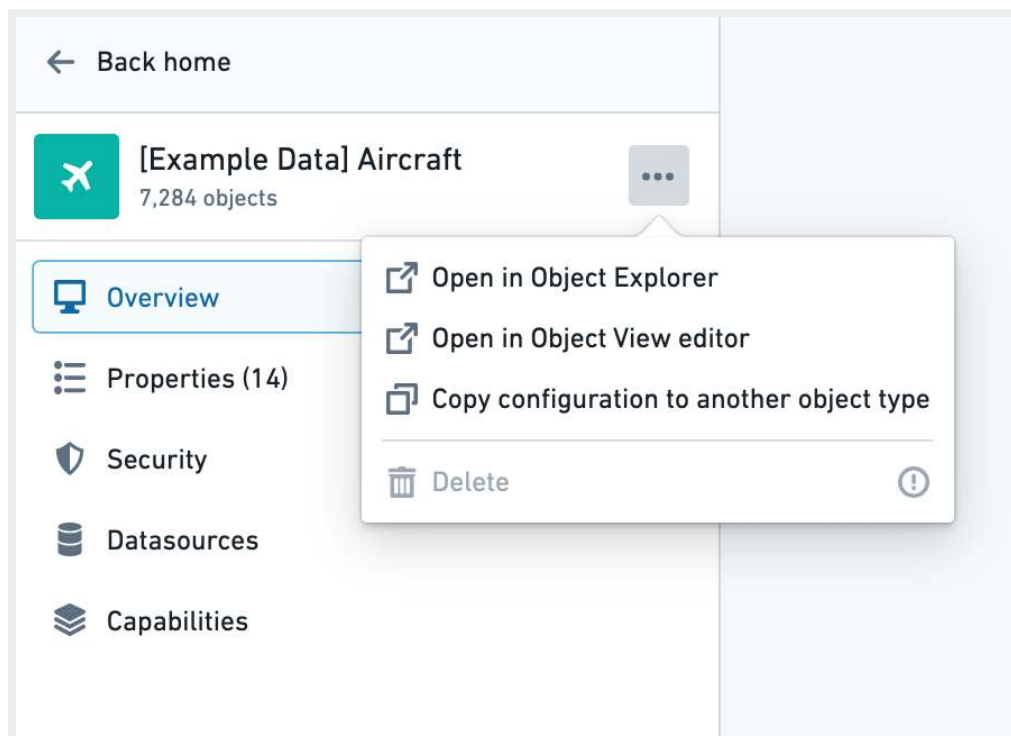
You can always change the object type you are working on by selecting the object type page from the home page sidebar and selecting a different object type from the list. You can also always search for a new object type in the search bar in the application header. [Read more about navigation.](#)

Delete an object type

You can delete an object type by selecting the  (three dots) icon at the top right of the  bar (see image below) and then selecting the **Delete** option from the dropdown. A dialog will pop up to confirm you want to stage the object associated link types for deletion.

[Send feedback](#)

- Note that the deletion of the object type only takes effect after you save your changes, and will break any views or applications referencing the object type.
- Object types with an **active** status cannot be deleted. [Read more about statuses.](#)



Change a backing datasource

You can change a backing datasource with the following steps:

1. Navigate to the property editor by selecting **Edit property mapping** at the top of the **Properties** page of an object type.
2. Select the  **Replace** button at the top of the **Datasources** pane. This will allow you to browse and select available datasources in Foundry.

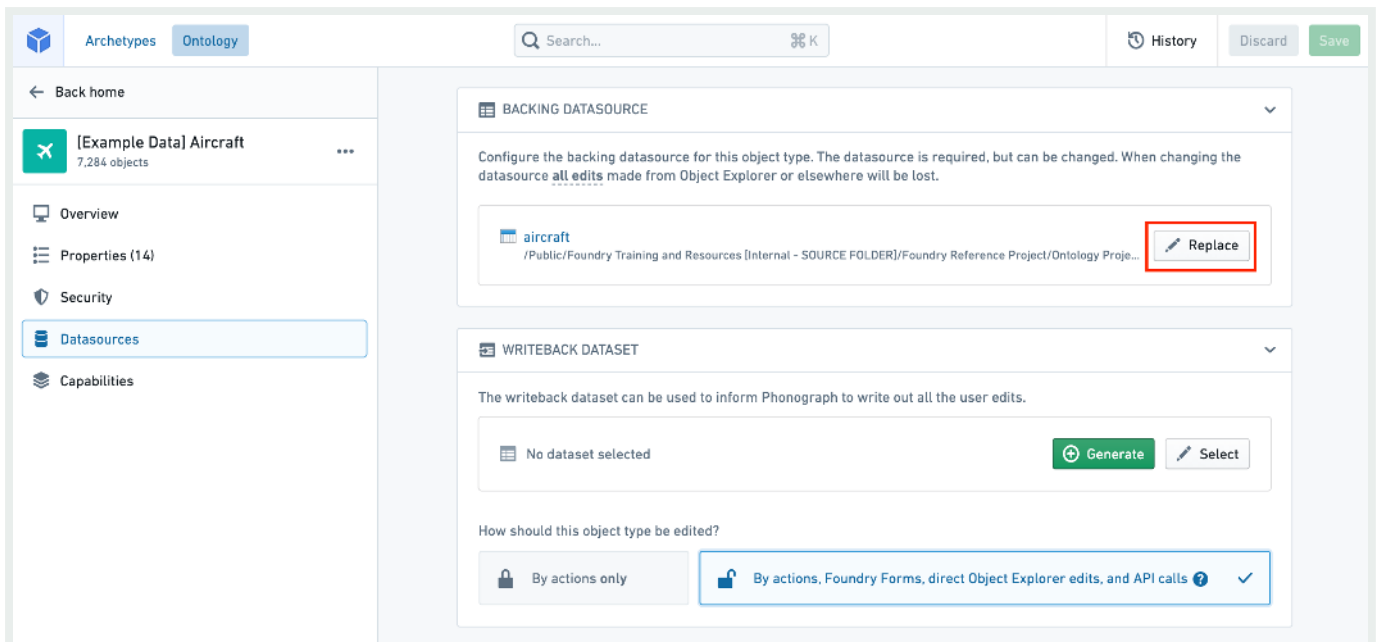
Warning

Changing the backing datasource of an object type will remove any connection between columns in the old datasource and the object type's properties. Properties

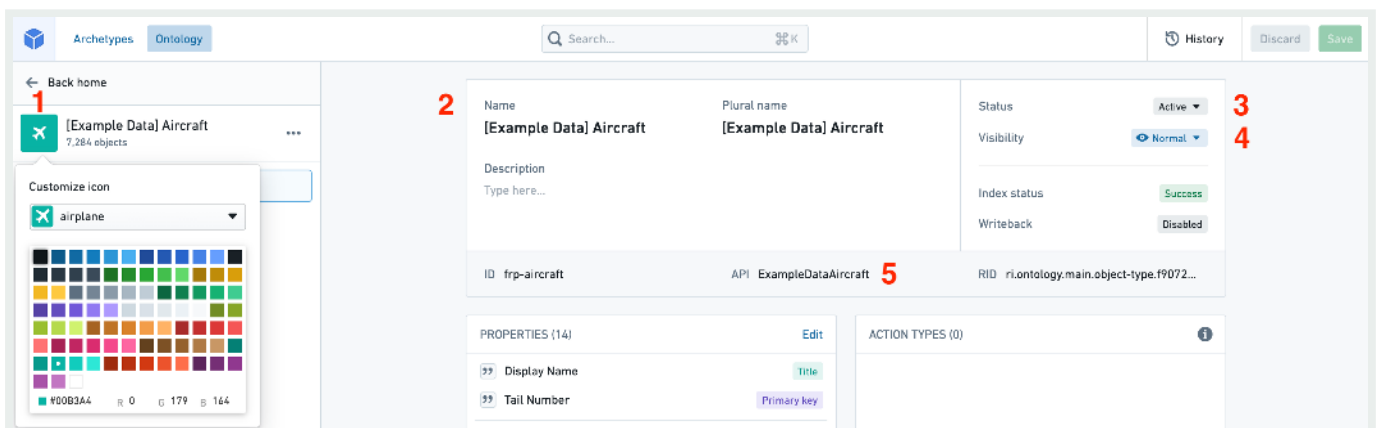
API Reference ↗

ically remapped for you **only if** you change to a new datasource with the **same schema** as the old datasource. Otherwise, you will need to manually re-map the object type's properties to the new datasource.

Send feedback



Edit an object type's metadata



1. **Icon:** Select the default icon to customize the icon and color of the object type that will appear in user applications when a user views an object of this type.
2. **Display names and description:** Select into the existing display names or description to edit the text.
3. **Status:** Select the existing status to open a dropdown of available statuses. Choose from the `deprecated`, `experimental`, and `active` statuses.
 - Read more about [statuses](#).
4. **Visibility:** Select the existing visibility to open a dropdown of available visibilities. A `prominent` object type will lead applications to show this object type first to users. A `API Reference` type will not appear in user applications.
5. **API name:** Select into the existing API name to change its value.
 - Note that you cannot change the API name for object types with
 - Read more about [statuses](#).

Send feedback

- Read more about [valid API names](#).

i The object ID of an object type cannot be edited after the initial object type creation process.

Troubleshooting

Error: `Phonograph2:FoundryColumnNameNotFound`

If you receive the error `Phonograph2:FoundryColumnNameNotFound`, a column has been removed from the datasource backing the object type you are trying to save and a property is left unmapped. The property needs to either be mapped or deleted.

Error: `Phonograph2:InvalidColumnRemoval`

If you receive the error `Phonograph2:InvalidColumnRemoval`, a column has been removed that was backing a property that has received edits. Either the column needs to be added back to the datasource, or the object type needs to be unregistered and reregistered.

See the section above on [potential breaking changes](#) to learn more.

Error: `Phonograph2:InvalidColumnFieldSchemaChange`

If you receive the error `Phonograph2:InvalidColumnFieldSchemaChange`, a property that has received edits has had its ID or key changed. Either the change needs to be reverted, or the object type needs to be unregistered and reregistered.

See the section above on [potential breaking changes](#) to learn more.

Error:

`OntologyMetadata:IncompatibleFoundryFieldSchemaForPropertyType`

API Reference ↗

If you receive the error

`OntologyMetadata:IncompatibleFoundryFieldSchemaForPropertyType`

save a property with a base type that is incompatible with the column type that is backing it.

Send feedback

For example, the type of column X may been changed to “string”, but is mapped to property X of base type “integer”.

Error: Phonograph2:SchemaMismatch

If you receive the error `Phonograph2:SchemaMismatch`, you likely made an intentional change to the schema that backs the object but have not yet updated the object's property types in Ontology Manager. Modify the Ontology by editing the property's data type to accept the new type. Publish the changes and rebuild the dataset, then initiate a re-index of the object.

Error: FieldTypeIncompatibleWithOntologyPropertyType

If you receive the error `FieldTypeIncompatibleWithOntologyPropertyType` or receive the message "Failed to Update Object Type in Phonograph", there is a mismatch between the data types in the dataset that backs your object and the data types that the ontology expects. You must ensure that any schema updates are reflected in both the dataset and the ontology.

If you did make any intentional changes to the ontology or the dataset, communicate with the owner of the object and its backing data source to understand recent changes.

PREVIOUS
← [Create an object type](#)

NEXT
[Enable Gotham integration](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Object types / Enable Gotham integration

Enable Gotham integration through type mapping

i Note

If your enrollment contains Map Rendering Service (MRS), then you do *not* need to complete the type mapping process to enable Gotham integration. You can [create new ontology objects from](#) or [add existing objects to](#) Gotham without additional configuration. Follow the instructions below to check if your enrollment contains MRS.

Contact Palantir Support with questions about MRS availability, installation, or its additional documentation present in Gotham.

How to check if your enrollment contains MRS

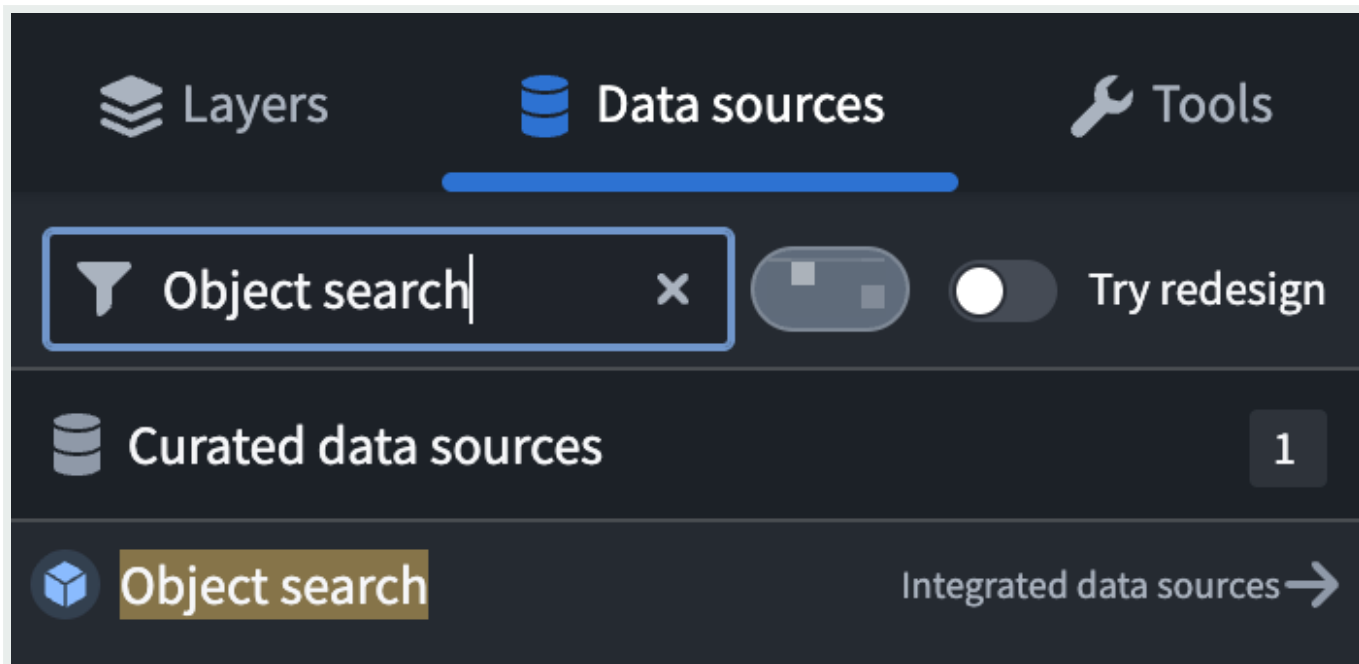
1. Launch [Quicksearch](#) to search for and select **Gaia Home**.
2. Open an existing or create a new Gaia map.
3. Select **Data sources** from the left panel.
4. Search for `Object search` in the **Find...** search bar.

If you are able to locate and select **Object search**, then your enrollment contains MRS.

[API Reference ↗](#)



[Send feedback](#)



Use type mapping to create a unified representation of your ontology

Type mapping enables a unified representation of your ontology across Foundry and Gotham that you manage within Foundry's [Ontology Manager](#) application. You can create new Gotham types based on existing Foundry object types, property types, and shared property types which remain synchronized as your ontology evolves over time. After completing the type mapping process, Gotham will be able to query Foundry object types and their metadata through the [Object Set Service](#) - a Foundry backend service which supports object data searching, filtering, aggregating, and loading.

i Note

Type mapping is only available for enrollments using both Foundry and Gotham and must be enabled by a platform administrator before use. Once enabled for a Foundry Ontology, type mapping cannot be disabled. Only one Foundry Ontology per Gotham install can have type mapping enabled. Contact Palantir Support for assistance with type mapping enablement.

[API Reference ↗](#)

[Send feedback](#)

When to type map object types in your ontology

Follow the [instructions below](#) and type map object types in Ontology Manager if your enrollment does not [contain MRS](#) and any of the conditions below are true:

- You plan to use custom symbology that is not present as a [tactical graphic](#) or [MIL-STD 2525 symbol](#). [Blueprint](#) [symbols](#) are supported with or without type mapping.
- You plan to configure [search templates](#), a [function-backed Enterprise Map Layer \(EML\)](#), or a [versioned object set-backed EML](#) to add data from your ontology to Gaia.

Toggle on type mapping in Foundry's Ontology Manager

To integrate data in your Foundry Ontology with Gotham, you will first need to toggle on type mapping for your object type of interest within Foundry's Ontology Manager by following the steps below:

1. Launch Ontology Manager from your home screen.
2. Locate and select your object type to type map.
3. Select **Capabilities** within the object type's left-hand panel.
4. Scroll down to the **Gotham Integration** panel and toggle on [Allow objects of this type to be accessed from Gotham applications](#).

➡ Gotham Integration

Allow objects of this type to be accessed from Gotham applications ☒

Installation status: [Ready to integrate](#) 

Gotham URI: [No Gotham URI available](#) 

Note

Type mapped objects must contain a [geopoint](#) property to display on a Gaia map. The property can be native to the object type's backing dataset(s) or derived from a latitude/longitude pair or [geopoint](#) using Pipeline Builder's [create Ontology](#)

[API Reference](#) [form feature](#).

Send feedback

Configure an object type's parent category and Gotham property types

Once you toggle on **Gotham Integration**, you can follow the steps below to create a new Gotham object type derived from an existing Foundry object type, specify the object type's **Parent category**, and configure its **Property types**:

1. Select **Create a new object type** within the **Object type** section of **Gotham Integration** to create a new object type in Gotham from an existing Foundry object type.
2. Identify the **Parent category** of the new object type in Gotham by selecting between **Entity** (such as a person, organization, or vehicle), **Event** (such as a flight, meeting, or concert), or **Document** (such as a PDF file, text document, or report) based on your use case.
3. Use the **Property types** section to map Foundry object type properties to Gotham - a property can be shared or cloned into the Gotham ontology. You will see a blue **Mapped** tag next to the property's name once you complete that configuration on a given property.
 - **Do not map property to Gotham** is the default option - Foundry properties you do not map will not propagate in the Gotham ontology.
 - **Assign to shared property** enables you to select an *existing* shared property to map against.
 - **Promote to shared property** creates a *new* shared property for use by other objects.
 - **Create a local clone of the property in Gotham** creates a duplicate of the selected property in Gotham that is compatible with its applications.

i Foundry automatically type maps all shared properties to make them available in Gotham.

4. Select the green **Save** button on the right side of the top ribbon of your screen and review.
5. ~~Review the changes~~ made to your object type and select **Save to ontology**.

[API Reference ↗](#)

[Send feedback](#)

Criteria
Advanced Settings

Object type
Create a new Gotham object type using predefined types or take over an existing object type

Create new object type

Take over existing Gotham object type

Parent category
Specify the new Gotham object type's parent category

Entity
Entity
Event
Document

Property types
Choose which properties to map into Gotham

After you save your changes to the Ontology, scroll back up to the **Gotham Integration** section of the **Capabilities** page of your object type. You will now see a **Gotham URI** assigned to the object type and be able to view the **Installation status** reported by Gotham.

Foundry's Ontology Manager will display one of the following installation statuses:

- **Ready to integrate**: An object type is ready for type mapping to enable Gotham integration.
- **Installation in progress**: The object type's installation process is underway.
- **Installation complete**: The installation process is complete, so the object type is available for use in Gotham.
- **Installation failed: {failureMessage}**: The object type's installation failed at least once and will be retried on the next installation run. The **{failureMessage}** outlines the reason for failure. Common installation failures include:
 - *Live reindex*: If there is a live reindex running on Gotham, the ontology cannot be updated. Instead of staging changes during this time, the installation will fail and rerun automatically once the live reindex finishes.
 - *Required types are not installed*: For an object type's successful installation, all related property installations - or mappings - must be completed, including shared property types.

API Reference ↗

updates: If there are ontology updates running concurrently, then the type mapper will not be able to update the ontology and will re

Send feedback

Once your object type's installation status reads **Installation complete**, you will be able to search for and use your object type in Gotham's applications.

To deprecate a type mapped object type in Gotham, you can delete the object type in Foundry's Ontology Manager. Once deleted in Foundry, the corresponding object type and its property types will not be accessible in Gotham or available to its applications.

i Gotham models Foundry dataset security and markings, meaning that Foundry data made available in Gotham through type mapping carries over the same access controls and classification.

PREVIOUS
← [Edit object types](#)

NEXT
[Create Ontology objects from Gaia](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Object types / Create Ontology objects from Gaia

Create Ontology objects from Gaia

Similar to how you can [add data from the Ontology to a Gaia map](#), you can also create Ontology objects from a shape you draw, a point you drop, or a tactical graphic you configure on a map by tagging the object within an object type which implements the Gaia Geoshape Creatable, Gaia Geopoint Creatable, or the MILSTD 2525C Symbol [interface](#).

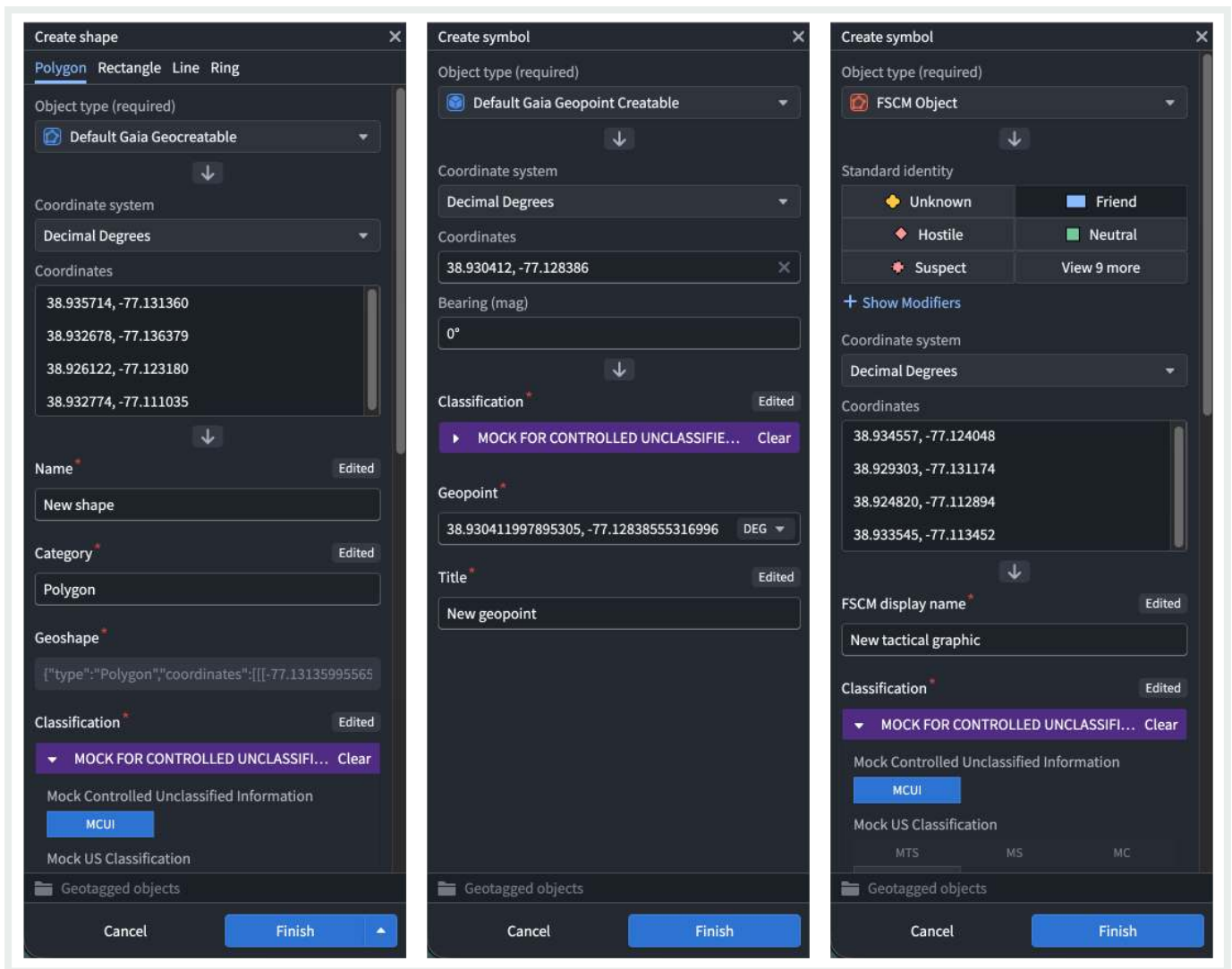
⚠ Made available through Palantir's [Defense OSDK](#), the MILSTD 2525C Symbol interface supersedes the Gaia Milsym Creatable interface, which is in the [planned deprecation](#) phase of development and will be deprecated by July 2026. Contact Palantir Support with questions about the availability of Defense OSDK interfaces or the deprecation of the Gaia Milsym Creatable interface on your enrollment.

If *both* interfaces are available on your enrollment, implement the MILSTD 2525C Symbol interface instead of the Gaia Milsym Creatable interface.

API Reference ↗



Send feedback



⚠ To create Ontology objects from a Gaia map, your enrollment must use both Foundry and Gotham.

The sections below outline the end-to-end process to integrate Gaia shapes, points, and tactical graphics with the Ontology:

- [Install all necessary Marketplace products.](#)
- [Implement the Gaia Geoshape Creatable interface on a supported object type backed by a restricted view.](#)
- [Implement the Gaia Geopoint Creatable interface on a supported object type backed by a restricted view.](#)

API Reference ↗

[LSTD 2525C Symbol interface on a supported object type backed by a](#)

- [Register your ontology and search for your object types in Gaia.](#)
- [Understand the entry points for creating objects.](#)

Send feedback

- [Draw a new shape on your map and tag it within your object type.](#)
- [Drop a new point on your map and tag it within your object type.](#)
- [Configure a tactical graphic from a shape drawn on your map and tag it within your object type.](#)

Install prerequisite Marketplace products

[Marketplace](#) is Foundry's storefront for published data products or collections of platform resources made available for user installation. You can access Marketplace through the **Search...** bar or the **Applications** portal on your home screen.

Once you launch Marketplace, install the following products:

- [Core Property Types from the Core Ontology Store](#), which contains core shared property types, such as an object's classification and geoshape, that can be used on multiple object types in your ontology.
- [Gaia Geoshape Creatable interface from the Gaia App Store](#), which describes the shape of a geospatial object type to enable consistent modeling of and interaction with other object types of the same shape.
- [Gaia Geopoint Creatable interface from the Gaia App Store](#), which describes the position of a single point object type.

Core Property Types from the Core Ontology Store

i Contact Palantir Support to install the Core Ontology on your Foundry enrollment as a [Foundry product](#) if the Core Ontology Store, which contains the Core Property Types product, is not available in Marketplace.

Gaia Geoshape Creatable interface from the Gaia App Store

In [API Reference](#) ↗ exist within your ontology to describe an object type's shape as well as its capabilities, enabling consistent modeling and interaction between object types of a common shape. You can implement an interface on multiple object types. The Gaia interface may extend any number of other interfaces. Once implemented on an object type, the Gaia

Send feedback

Geoshape Creatable interface enables you to create objects in your ontology from shapes drawn on a Gaia map.

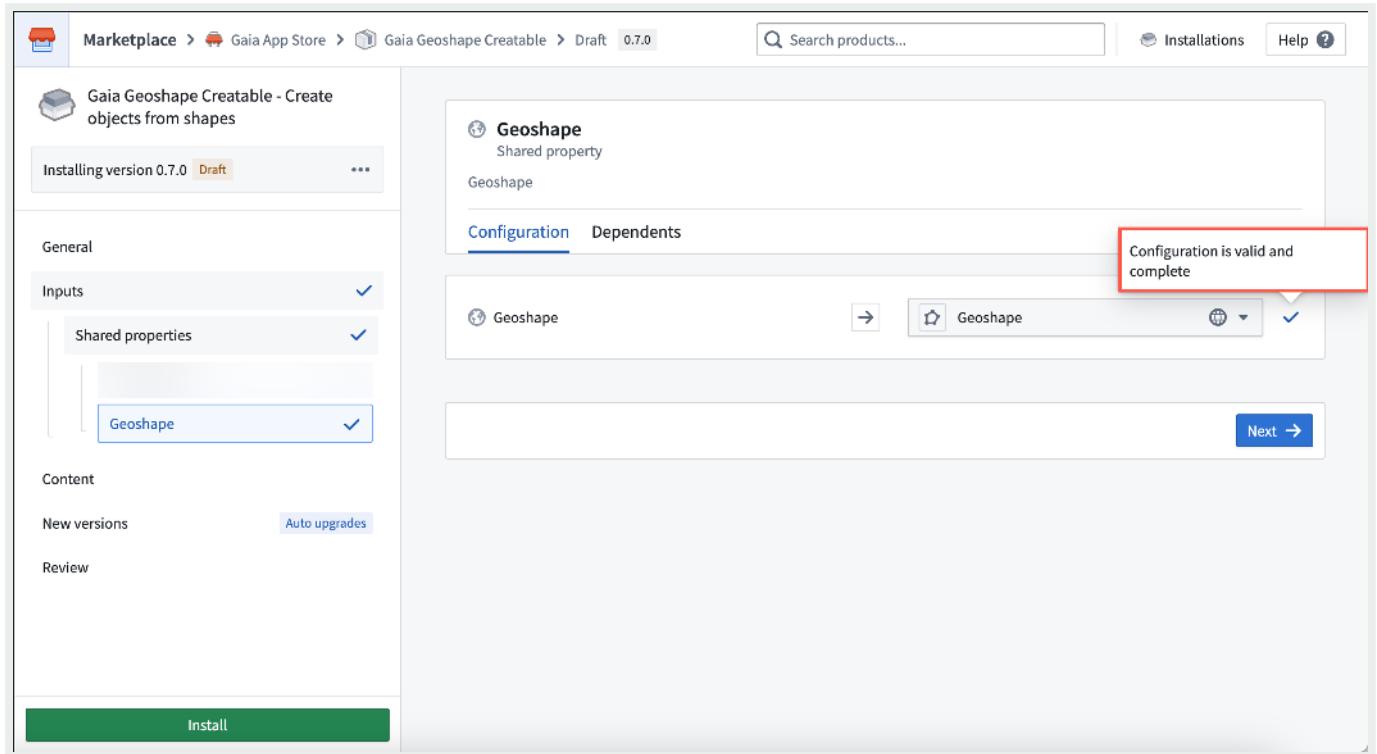
 The Gaia Geoshape Creatable interface supersedes the deprecated Gaia Geocreatable interface. Contact Palantir Support if you are unable to access the Gaia Geoshape Creatable interface in Marketplace.

To install the Gaia Geoshape Creatable interface from Marketplace:

1. Navigate back to Marketplace and search for the **Gaia App Store** in **Search stores....**
2. Choose the **Gaia Geoshape Creatable** product.
3. Select the blue **Install** button on the right side of your screen to launch a draft installation. You can optionally add a descriptive suffix to your installation.
4. Identify a **Namespace** where your installation will save. Marketplace will automatically create a new **Project** within the chosen **Namespace**.
5. Configure the product's classification and access controls under **Permissions**. The classification you select under **Classification based access control (CBAC)** defines the *maximum* classification for the installation.
6. Select **Next** to launch the installation window's **Inputs** page.
7. Select the **Shared properties** tab under **Inputs** in the left panel to map the **Geoshape** shared property from your ontology to your new interface.
8. Select **Next** to launch the installation window's **Content** page.

[API Reference ↗](#)

[Send feedback](#)



9. Optionally toggle on **Prefix Ontology entities** and insert a valid prefix. Note that your prefix may not contain certain special characters, such as parentheses or brackets.
10. Choose which **Ontology schema migrations** to enable on the right side of your screen. You can reference additional schema management information within the existing [object edits and materializations documentation](#).
11. Update the automatic configurations in **New versions** as necessary for your use case before you select **Next**. Marketplace pre-configures certain products to upgrade automatically.
12. Review your interface's configurations and select **Install**.

Gaia Geopoint Creatable interface from the Gaia App Store

To install the Gaia Geopoint Creatable interface from Marketplace's Gaia App Store, select the **Gaia Geopoint Creatable** product from the store menu. Marketplace's interface installation workflows are common across different interfaces, so you can follow the same steps outlined in the [Geoshape Creatable interface installation instructions](#) above with the following distinctions:

API Reference ↗

Properties section of the **Inputs** window, map the **Geopoint** shared property to your ontology to the Gaia Geopoint Creatable interface.

Send feedback

Create an object type that implements the Gaia Geoshape Creatable interface

Gaia can discover object types backed by a [dataset](#) or [restricted view](#) in your ontology after they implement the Gaia Geoshape Creatable interface. In the following sections, you will:

- [Create an object type-backing dataset or restricted view.](#)
- [Create an object type and ensure it integrates with Gotham.](#)
- [Configure an action type to enable object creation in Gaia.](#)
- [Implement the Gaia Geoshape Creatable interface.](#)

Create an object type-backing dataset or restricted view

i You should create a restricted view if you plan to secure objects based on a user's classification or by applying [markings](#) to control file access. If your use case does not require the additional object security provided by classification-based access or markings, then you should create a dataset to back to your object type.

To create an object type that can integrate with Gaia, you will first need to create a dataset or restricted view that contains, at a minimum, the following columns:

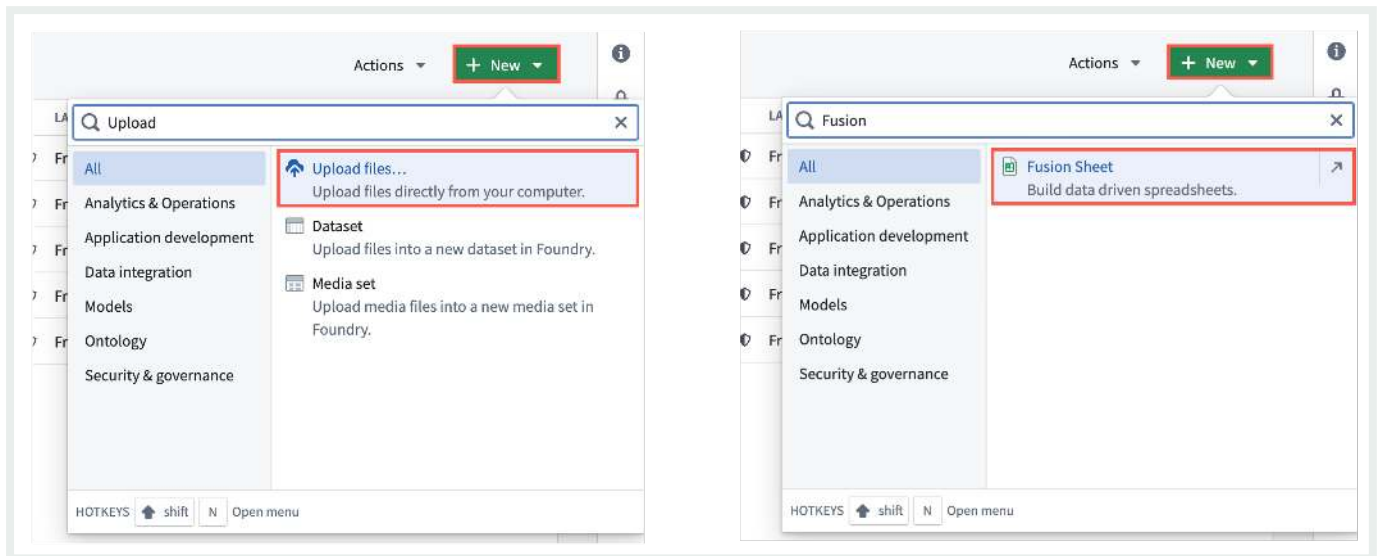
- `Geoshape`, which you will set as a `string` for Gaia to automatically populate with your drawn object's shape.
- `Object ID`, which you will set as a `string` for Foundry to automatically populate with a unique ID for each Gaia shape you create as an object. This will serve as your object type's primary key.
- `Classification`, which you will set as an `array` to capture your object's classification. A `Classification` column is *only* required if a restricted view backs your object type.

i You can configure additional columns in your dataset or restricted view based on your use case, such as `Name`, `Category`, or `Notes` columns set as `strings` to capture user-entered descriptive information about the object.



Send feedback

Select the **New** button in your Project to upload an existing file, such as a `.csv`, or use [Fusion](#) to create a standalone dataset or a dataset that backs your restricted view. If you use a dataset to back your object type, you can skip the restricted view creation instructions below and proceed to [create your object type](#).



i You can reference additional restricted view creation instructions in Foundry's [security](#) and [object permissions](#) documentation.

Your restricted view should contain a [granular policy](#) that restricts user access to the data contained in the view based on their [classification access](#). Compose the granular policy as the first step in the **Create '{restricted view}'** window.

[API Reference](#) ↗

[Send feedback](#)

Create restricted view

Save as
A new file will be created once you complete this process.

2

Compose a granular policy
A granular policy is a combination of rules that describe what rows can be seen by different people.

↑ Previous

Next ↓

Edit granular policy

Edit policy

</> View policy in JSON

🧪 Test policy

Match all of the rule below:

The user's **Classification access** satisfies column **classification**

+ Add a rule or logical operator

Compose a granular policy
A granular policy is a combination of rules that describe which rows of a dataset are visible in the restricted view.
Add a rule to get started.

⚠ If an object contains a CBAC or mandatory marking property to restrict its access, then Foundry creates objects using the CBAC or mandatory markings it inherits from the Gaia map and will *not* apply group restrictions defined in the map's security and sharing settings.

You can reference an example policy in JSON below.

```
1 {
2   "condition": {
3     "and": {
4       "conditions": [
5         {
6           "markings": {
7             "value": {
8               "field": {
9                 "fieldName": "classification"
10              },
11              "type": "field"
12            },
13            "filters": [
14              {
```

API Reference ↗

Send feedback

```

15         "markingTypes": {
16             "markingTypes": [
17                 "CBAC"
18             ]
19         },
20         "type": "markingTypes"
21     }
22 ]
23 },
24 "type": "markings"
25 }
26 ]
27 },
28 "type": "and"
29 }
30 }

```

Create an object type and ensure it integrates with Gotham

Once you configure your restricted view, launch [Ontology Manager](#) and follow the steps below to create your object type:

1. Select **New > Object type** from the top right of your screen.
2. Select **Use existing datasource** and choose **Select datasource** to locate and **Select** your restricted view before choosing **Next**.
3. Name your object type and optionally enter a **Description**.
4. Set **Object ID** as the **Primary Key** and **Name** as the **Title**.
5. Ensure **Classification**'s **Property** is an array of strings and **Geoshape**'s **Property** is geoshape. You will only need to validate the former if a restricted view backs your object type.

[API Reference ↗](#)

[Send feedback](#)

Create a new object type

1 Datasource

2 Metadata

3 Properties

4 Actions

STEP 3

Create properties

Primary key ?

Title ?

Object Id

Name

Source

Property

classification

[]

Classification

category

”

Category

name

”

Name

Title

object_id

”

Object Id

Primary key

geoshape

⌂

Geoshape

map_id

”

Map Id

+ Add property

Back

Next

6. Select **Create**, as you will generate and configure action types *after* object type creation.

With your draft object type viewable in Ontology Manager, you will next select the **Classification** and **Geoshape** properties as shared property types in your ontology. Select the **Properties** panel beneath **Overview** and follow the steps below to complete the shared property type selection process:

i If you are using a dataset to back your object type that does *not* contain a **Classification** property, then you should complete steps 1 and 4 below for your **Geoshape** property.

1. Select **Classification** from your list of properties to launch the **Property editor** window on the right side of your screen.

2. API Reference **type** dropdown menus to contain **Mandatory control** and **CBAC Marking**, respectively.

3. Configure the property's **Max Classification**.

Send feedback

  Classification  

General Source Display Interaction Details Advanced

Overview

Name

Classification 

Description

This is the core ontology classification

Base type 

 Mandatory control 

Classification 

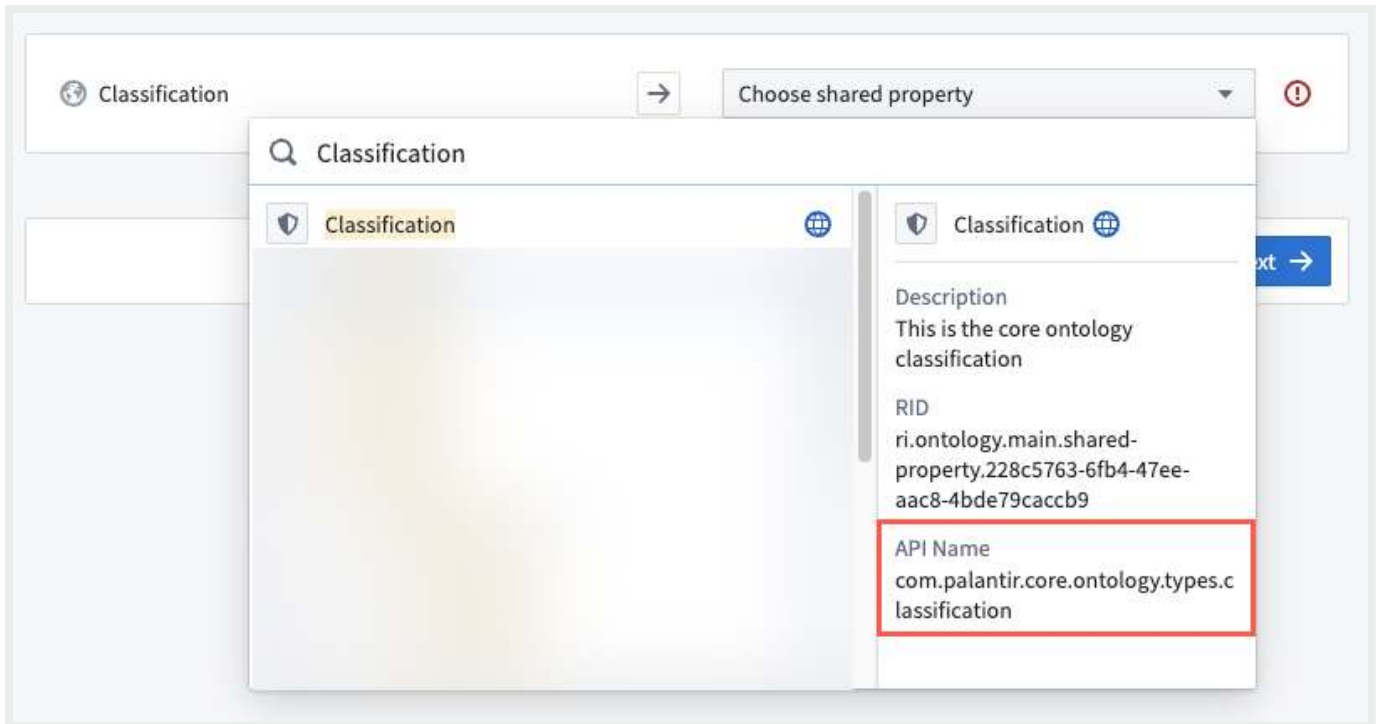
Max Classification MTS//MNF 

 Contact your Palantir Support if you are unable to select **Mandatory control** as the base type for `Classification`, as **Mandatory control** markings are not generally available across all Foundry enrollments.

4. Scroll to the bottom of the window to the **Shared property** section and use the dropdown menu to assign `Classification` as a shared property.

API Reference ↗

Send feedback



Repeat steps 1 and 4 above for your **Geoshape** property, as you will *not* need to configure its mandatory control markings or classification.

Select the green **Save** button at the top of the screen to publish incremental changes to your ontology before proceeding.

If your Foundry enrollment contains Map Rendering Service (MRS), then object types in your ontology with a geospatial property type automatically integrate with Gotham, and you can proceed to [create an action type to enable object creation in Gaia](#). Follow the steps in [the Gotham integration documentation](#) to check if your enrollment contains MRS. If your enrollment does *not* contain MRS, then [follow the instructions](#) to integrate data in your ontology with Gotham.

i Contact Palantir Support with questions about MRS installation or functionality.

Create and configure an action type to enable object creation in Gaia

[API Reference ↗](#)

Once you create your object type and ensure it integrates with Gotham, navigate back to the **Overview** window in Ontology Manager. Here, create an [action type](#) to

[Send feedback](#)

create and edit objects from shapes and configure their properties, such as **Name** and **Category**, from Gaia.

 Your Foundry enrollment must contain MRS to edit Ontology objects in Gaia, as this capability does not extend to objects added to Gaia through type mapping. Contact Palantir Support with questions about MRS installation or functionality.

Follow the steps below to configure your action type:

1. Select **New** from the **Action types** section of your object type's **Overview** window to launch the **Create a new action type** pop-up window.

 If you are unable to select the **New** button in **Action types**, then you can toggle on **Allow edits** on your object type in the **Datasources** window.

2. Select **Modify or create object** under **Object actions** before choosing **Next**.
3. Select **Auto-generated primary key** from the **Or create a new object with** dropdown menu.
4. Select **Add property** to add all your existing properties to the action type, then choose **Next** to configure your action type's metadata.

[API Reference ↗](#)

[Send feedback](#)

Create a new action type

1 Action type

2 Mapping

3 Metadata

4 Submission criteria

STEP 2

Map action parameters

Map the action parameters that will be used as inputs to this action

Modify existing selected

Gaia Create Object

Or create a new object with

Auto-generated primary key

PROPERTY		MAP TO
Classification	→	Classification
Geoshape	→	Geoshape
Name	→	Name
Category	→	Category

+ Add property

Skip

Back

Next

i You will only need to map a **Classification** property if a restricted view backs your object type.

5. Name your action type, and optionally enter a description and update its default icon.
6. Select an **Organization**, **Group**, or **User** who may execute the action, then select **Create**.

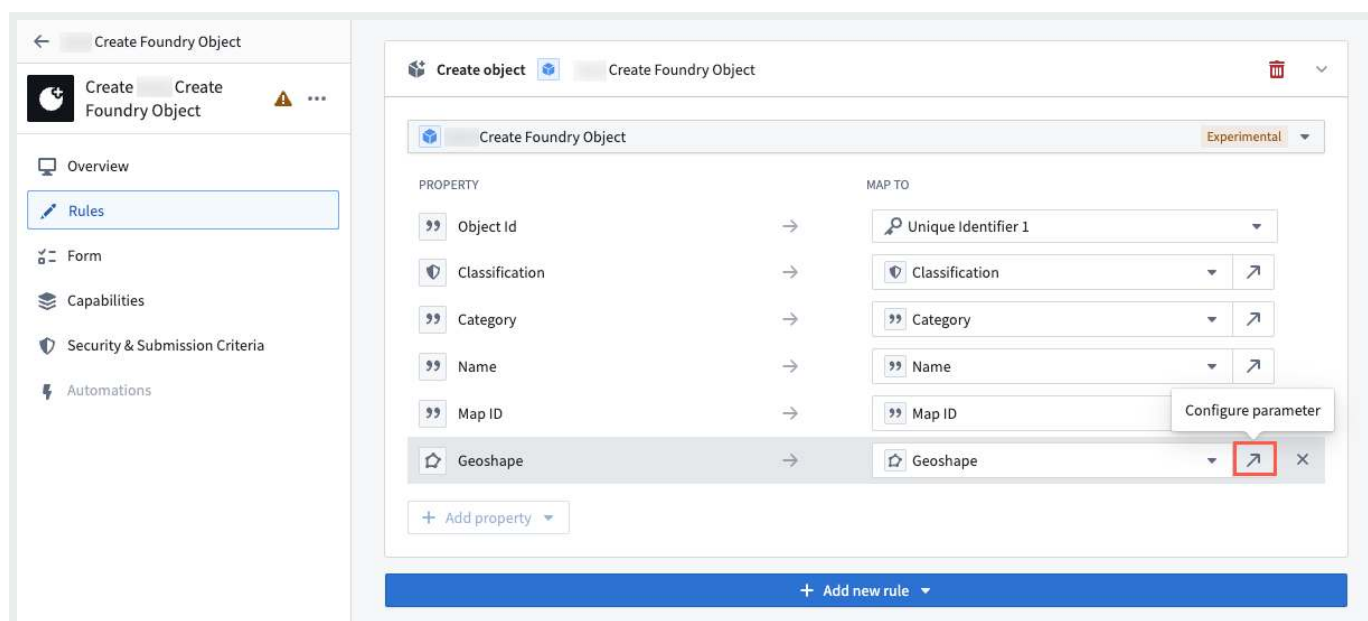
Select the green **Save** button at the top of the screen to publish incremental changes to your ontology before proceeding.

Next, you will configure your action type's **Rules** and **Parameters** by following the steps below:

1. Select **Rules** from the left side of the screen.
2. **API Reference** **Geoshape** property's configurations by selecting the arrow icon next to **Configure parameter**.

Send feedback

- Ensure the **Geoshape** property's type is either **Geoshape** or **String** from the **Type** dropdown menu on the right side of the screen.
- Ensure the **Disabled** option is selected in the **General** panel so a user cannot manually configure the location of a **Geoshape**.



3. Select your **Classification** property from the **Form content** panel on the left side of your screen and verify that its **Type** is **Mandatory control**. You will only need to configure a **Classification** property if a restricted view backs your object type.
4. Select **Back to Form** and remove **Object Id** from **Form content** by selecting the **X** icon on the far right side of the **Object Id** panel. Foundry will automatically generate a unique ID for each object created from Gaia.
5. Select the green **Save** button at the top of your screen to save your action type's configurations to the Ontology.

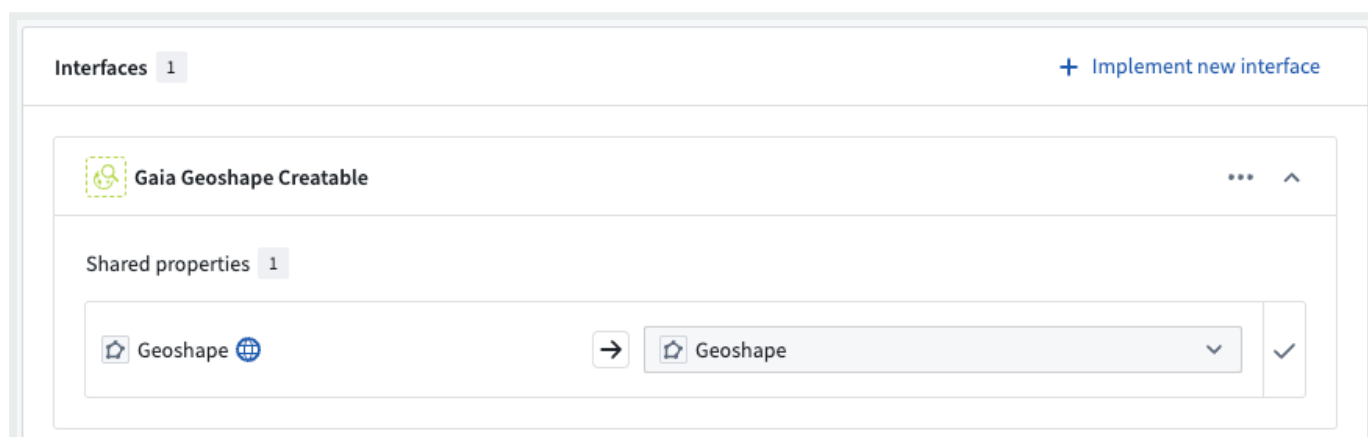
Implement the Gaia Geoshape Creatable interface

The final step in creating and configuring your object type in Ontology Manager is to implement the Gaia Geoshape Creatable interface that you [previously installed from Marketplace](#). Navigate back to your object type's **Overview** page and follow the steps below to implement the interface before saving changes to your ontology:

1. Select **API Reference**  **Interfaces** window beneath **Object Views** on the left side of your screen.
2. Select **Implement new interface** and search for **Gaia Geoshape Creatable**, choosing **Next**.

Send feedback

3. Select **Choose an option** > **Replace existing** to map your ontology's **Geoshape** shared property type to the object type implementing the interface.
4. Select **Confirm** to close the **Implement an interface** window and **Save** the newly configured interface to your ontology.



Create an object type that implements the Gaia Geopoint Creatable interface

To create an object type that implements the Gaia Geopoint Creatable interface, you can follow the same steps outlined in the [object type creation instructions for the Gaia Geoshape Creatable interface above](#) with the following distinctions in each section:

- [Create an object type-backing restricted view.](#)
 - Create a **Geopoint** instead of **Geoshape** column.
- [Create an object type and ensure it integrates with Gotham.](#)
 - In the object type creation window, ensure **Geopoint**'s **Property** is **geopoint**.
- [Configure an action type to enable object creation in Gaia.](#)
 - Ensure your **Geopoint** property's **Type** is **Geopoint** in your action type's **Parameters** window.
- [Implement the Gaia Geoshape Creatable interface.](#)
 - Search for the **Gaia Geopoint Creatable** instead of the **Gaia Geoshape Creatable** interface.
 - When mapping your ontology's shared property types, select **Replace existing** for the **API Reference**  head of the **Geoshape** property.

Send feedback

Create an object type that implements the MILSTD 2525C Symbol interface

 Unlike the Gaia Geoshape Creatable and Gaia Geopoint Creatable interfaces, you **must** use a restricted view to back an object type that implements the MILSTD 2525C Symbol interface.

To create an object type that implements the MILSTD 2525C Symbol interface, you can follow the same steps outlined in the [object type creation instructions for the Gaia Geoshape Creatable interface above](#) with the following distinctions in each section:

- [Create an object type-backing restricted view](#).
 - You will only need to create `Object ID`, `Classification`, and `Title` columns.
- [Create an object type and ensure it integrates with Gotham](#).
 - In the object type creation window, set `Object ID` as the **Primary key** and `Title` as the **Title**.

When [configuring an action type to enable object creation](#), ensure you map at least the `Classification`, `Symbol Anchor Points`, and `SIDC` properties in the **Create a new action type** window before you select **Next**.

[API Reference ↗](#)

[Send feedback](#)

Create a new action type

1 Action type

2 Mapping

3 Metadata

4 Submission criteria

5 Save location

STEP 2

Map action parameters

Map the action parameters that will be used as inputs to this action

Modify existing selected

MILSTD 2525C Interface Symbol

Or create a new object with

Auto-generated primary key

PROPERTY		MAP TO
Classification	→	Classification
Symbol Anchor Points	→	Symbol Anchor Points
SIDC	→	SIDC

+ Add property

Skip

Back

Next

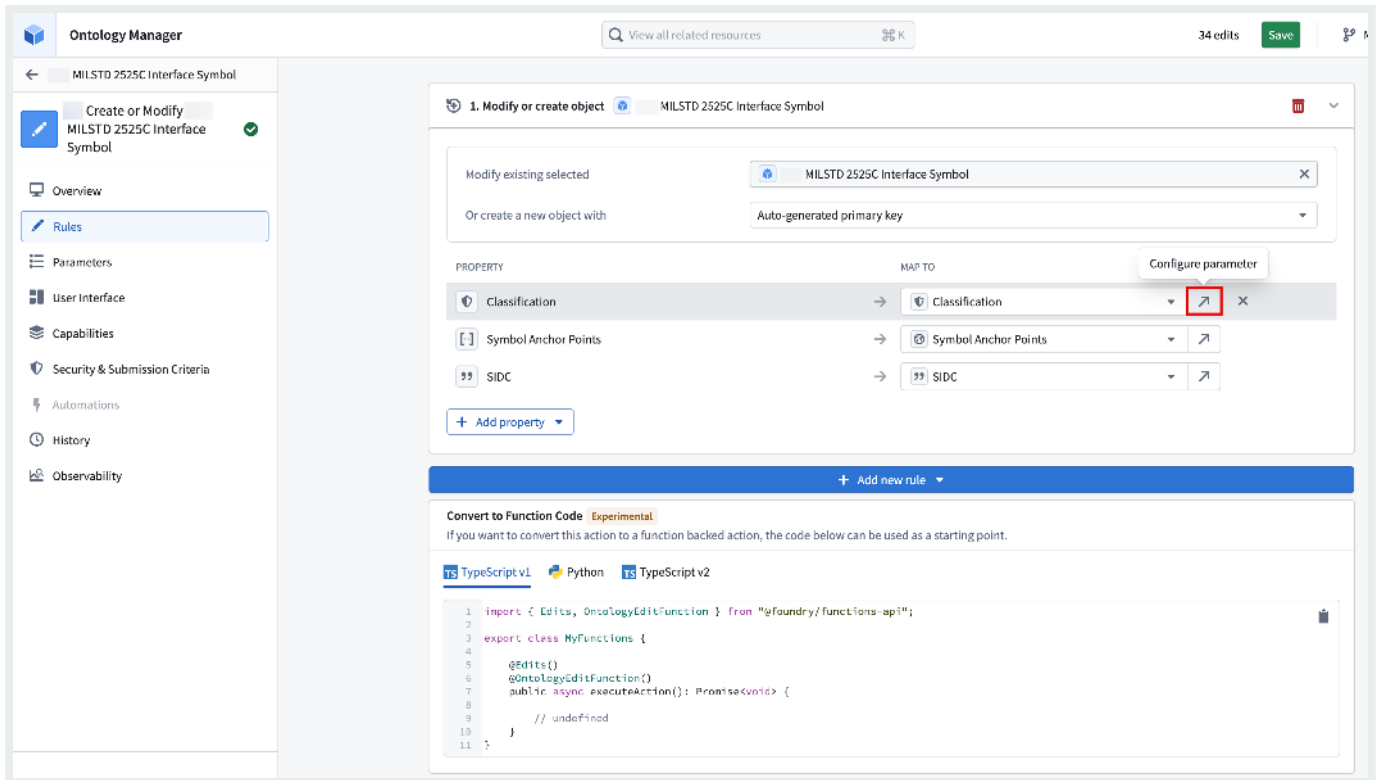
Select the green **Save** button at the top of the screen to publish incremental changes to your ontology before proceeding.

Next, you will configure your action type's **Rules** and **Parameters** by following the steps below:

1. Select **Rules** from the left side of the screen.
2. Select the arrow icon to the right of your **Classification** property to **Configure parameter**.

API Reference ↗

Send feedback



3. If the **Classification** property on your object type is fulfilled by the shared property type, then you should toggle on **Disabled** in the **General** section to ensure the object created automatically inherits its classification markings from your Gaia map. If you want to manually set the classification markings on creation, then do not toggle on **Disabled**.
4. Select **Add** under **Set a parameter max classification** to ensure the action type's maximum classification matches the object type's maximum classification.

[API Reference ↗](#)

[Send feedback](#)

Classification

No description

Value Display Overrides Details

Type

Mandatory control

General

Visible

Add override

Disabled

Add override

Required

Add override

Constraints

Classification input

Marking input

Set a parameter max classification ?

Add a maximum classification

Add

Default value

Add override

Set a default value for the input. It can be overridden by users.

Object parameter property

As static value

MILSTD 2525C Interface Symbol

Classification

Experimental

5. Select the green **Save** button at the top of the screen to publish the action type to your ontology.

⚠ Since the use of struct properties in actions is not currently supported, you will not be able to configure the **Speed Modifier (Z)** or **Altitude/Depth Modifier (X)** properties as action type parameters.

- Implement the Gaia Geoshape Creatable interface.
 - Search for the **MILSTD 2525C Symbol** instead of the **Gaia Geoshape Creatable interface**.

API Reference ↗ When configuring your ontology's shared property types, select **Replace existing** for your **Classification** property and **Create edit-only property** for **Classification**.

Send feedback

Register your ontology and its types

Now, you have object types that implement the Gaia Geoshape Creatable, Gaia Geopoint Creatable, and MILSTD 2525C Symbol interfaces and contain accompanying action types that enable a user to configure objects they create from shapes drawn, points dropped, and tactical graphics configured on a Gaia map. Next, you will register your ontology, object types, and action types either in Gaia's admin application or [Control Panel](#) extension, depending on your enrollment.

i To access Gaia's admin application, you must be a platform administrator. To access Gaia's Control Panel extension, you must be granted the **Organization administrator** role.

Contact Palantir Support with questions about access to the admin application or Control Panel extension if you are unable to access either.

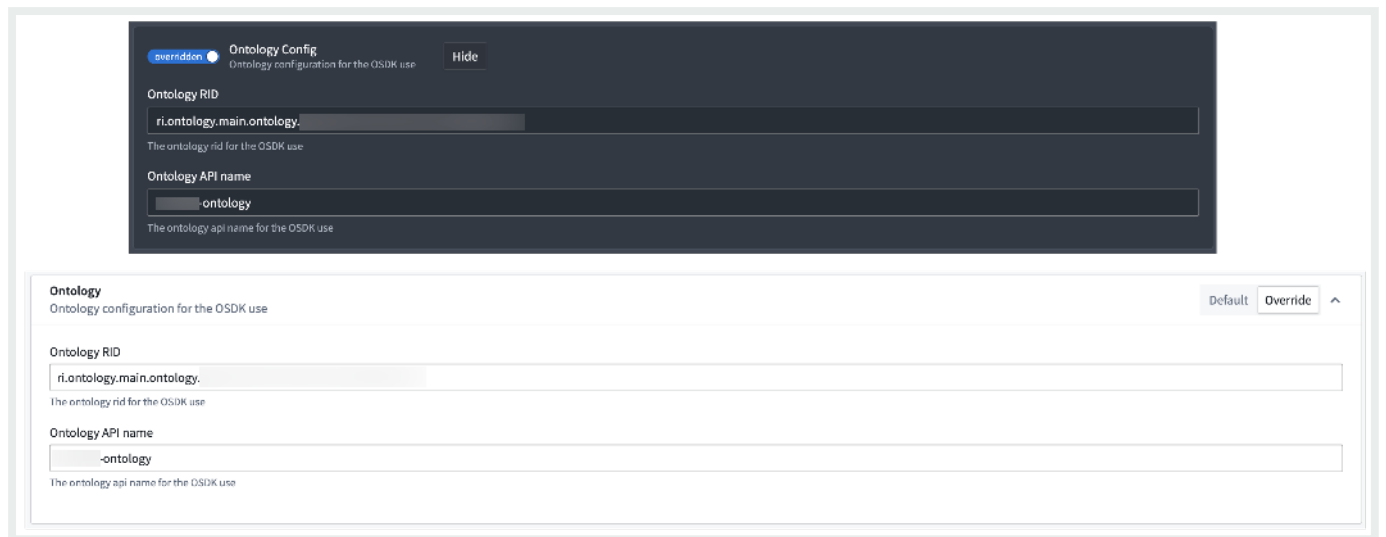
To register your ontology, object types, and action types, launch Gaia's admin application or Control Panel extension and follow the steps below:

1. Locate the **Ontology Config** (admin application)/**Ontology** (Control Panel extension) panel and verify your ontology's configuration. If your ontology is not configured, select the toggle on the left side of the panel to set **Ontology Config/Ontology** to **overridden** (admin application)/**Override** (Control Panel extension).
2. Enter your ontology's RID and API name into the **Ontology RID** and **API Name** text boxes.

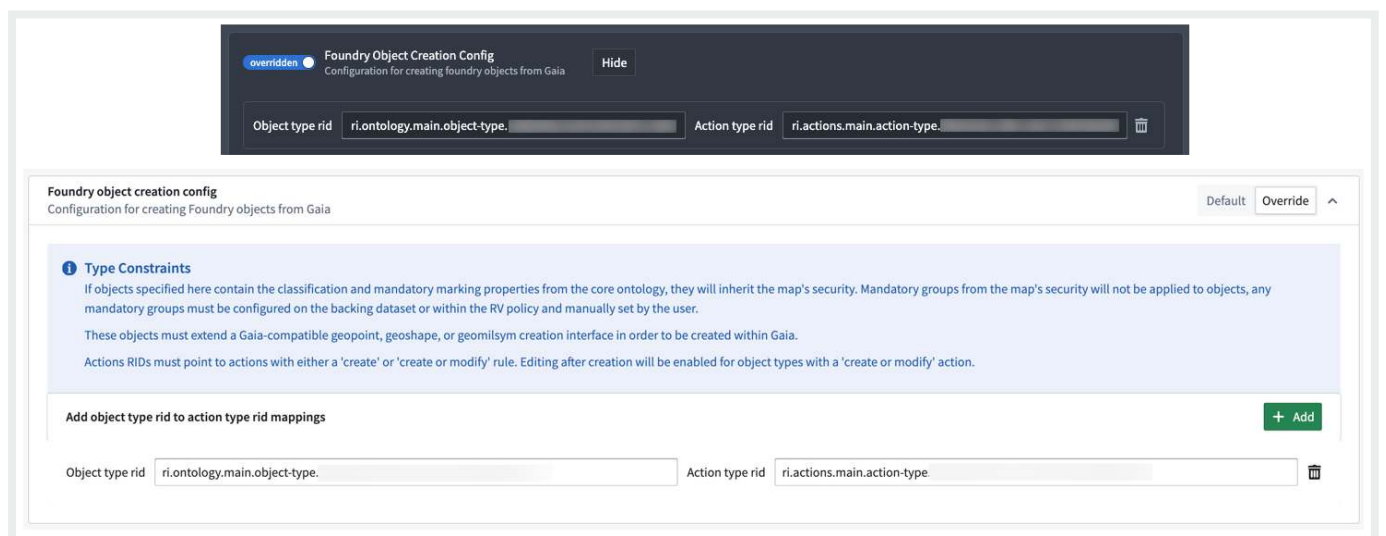
i To locate and copy your ontology's RID and API name, navigate to Ontology Manager and choose **Ontology configuration** from the bottom of its left panel to launch the **Ontology metadata** window.

[API Reference ↗](#)

[Send feedback](#)



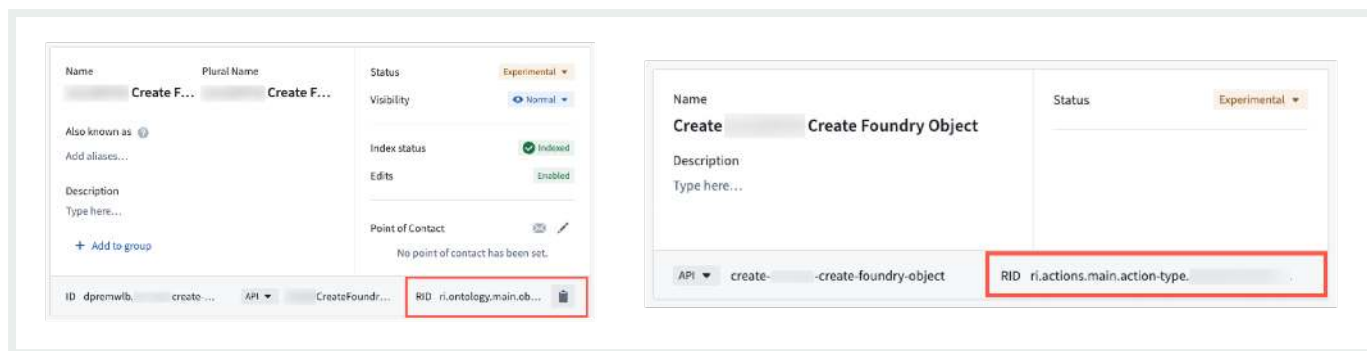
3. Locate the **Foundry Object Creation Config** panel and select **Show** (admin application)/**Override** (Control Panel extension).
4. Select **Add** at the bottom of your enrollment's existing object and action type list.
5. Copy and paste the RIDs for all three object types and their supporting action types into three separate **Object type rid** and **Action type rid** text boxes, respectively.
6. Select **Preview and save** in the top right ribbon (admin application)/**Save for {Organization}** in the bottom right corner (Control Panel extension).



You can access your object type's RID from the **Overview** window in Ontology Manager. Select the clipboard icon to copy the RID. Additionally, you can access your action type's RID by selecting **Create {object type name}** in the **Action types** section of the **Overview** window. The action type's RID can also be copied through the clipboard icon.

[API Reference ↗](#)

[Send feedback](#)



Next, you will launch Gotham's Gaia application to create objects from shapes you draw, points you drop, and tactical graphics you configure on a map using its **Add to map** menu.

Entry points to create objects on your Gaia map

Gaia provides two distinct entry points you can use to create an object on your map which determine the list of available object types that render in the **Object type** dropdown menu.

- **Select an object type first:** When you select an object type from the toolbar dropdown menu, Gaia lists every object type that implements a supported interface, including the Gaia Geoshape Creatable, Gaia Geopoint Creatable, MILSTD 2525C Symbol, and Gaia Milsym Creatable interfaces. After you select an object type, Gaia filters the available drawing tools to *only those* compatible with the interfaces that the object type implements. For example, selecting an object type that implements the Gaia Geoshape Creatable interface displays only the shape drawing tools. When you then select a drawing tool, Gaia opens the left panel with your object type already populated in the **Object type** dropdown menu.

i An object type can implement more than one interface. For example, if an object type implements both the Gaia Geopoint Creatable and Gaia Geoshape Creatable interfaces, then selecting it from the toolbar displays both the point symbol and all shape drawing tools while filtering out the symbol **Search** menu.

- **Select a drawing tool first:** When you select a drawing tool before selecting an object type, Gaia opens the left panel with no object type pre-selected. In this case, the **Object type** menu lists only the object types that implement the interface matching the tool you selected. For example, selecting the point symbol displays only the object types that implement the Gaia Geopoint Creatable interface.

API Reference ↗

Send feedback

Draw a new shape on your Gaia map and tag it to an object type

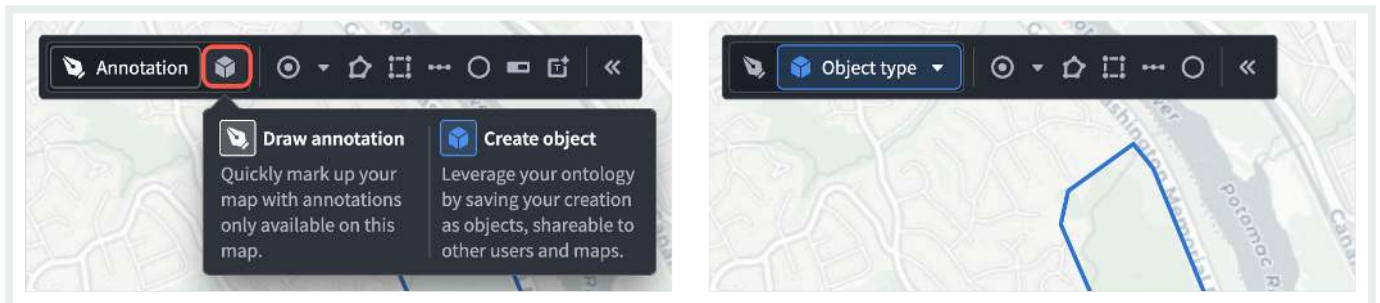
⚠ Current limitations around the International Date Line

While you can create map annotations that cross the 180th meridian, or antimeridian, on a Gaia map, you cannot draw shapes tagged as Foundry objects that do so.

Contact Palantir Support with questions about annotation creation, rendering, or Gaia's additional documentation present in Gotham.

Before you begin, review the [entry points for creating objects](#) to understand how the object type and drawing tool you select determine which object types are available to you.

With your Gaia map open, select the object icon from the menu in the top left region of your map to switch from **Draw annotation** to **Create object** mode.



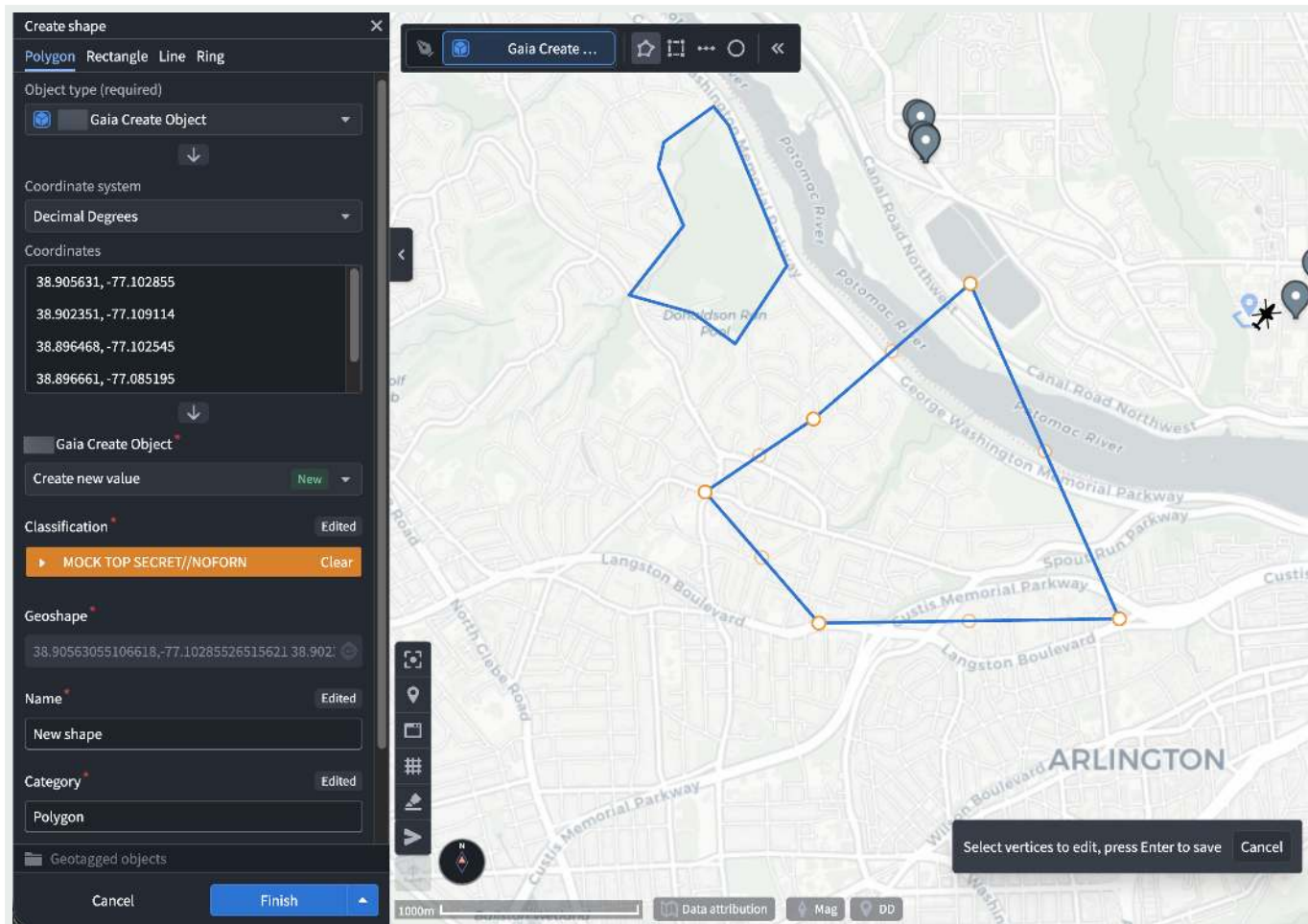
Next, select a shape to draw from the right side of the same menu. When you finish drawing your shape, the **Create shape** window will appear in Gaia's left panel. Follow the steps below to configure your shape and save it to your ontology as an object:

1. Search for and select your object from the **Object type** dropdown menu. If you selected an object type from the tool bar's dropdown menu on your map canvas, then Gaia will automatically populate the **Create shape**'s dropdown menu with that value.
2. Complete the required fields in the action form, such as **Category** and **Name**, for your

[API Reference ↗](#)

select **Finish**.

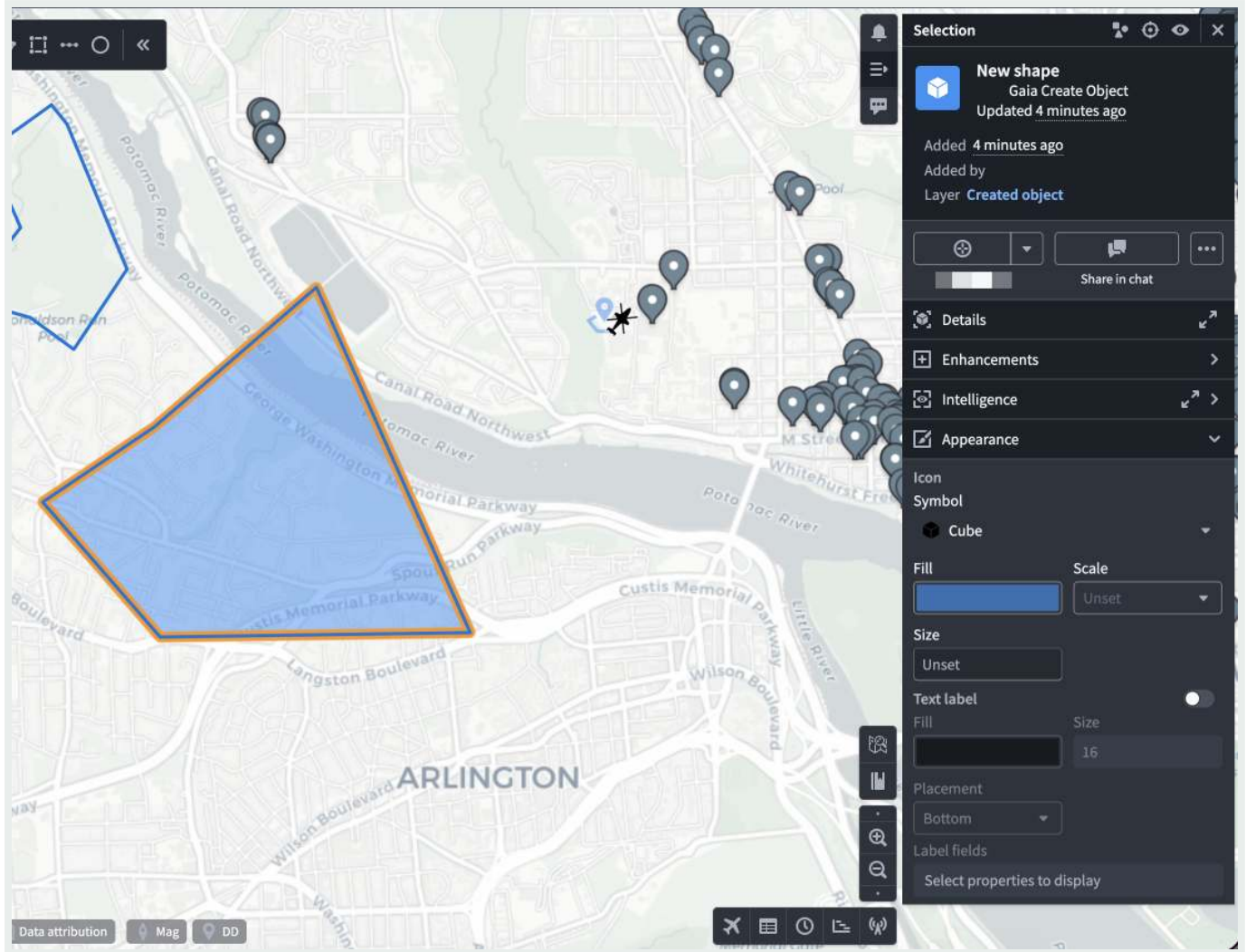
[Send feedback](#)



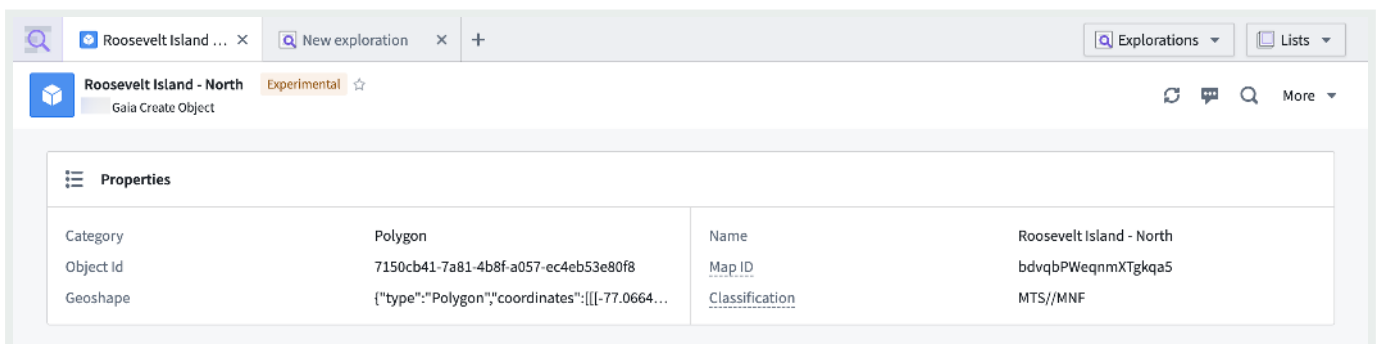
Once you save your shape as an object, it will render as one of the **Layers** in Gaia's left panel. You can select the object's name to launch the **Selection** panel on the right side of your screen, where you can view its properties and customize its appearance.

[API Reference ↗](#)

[Send feedback](#)



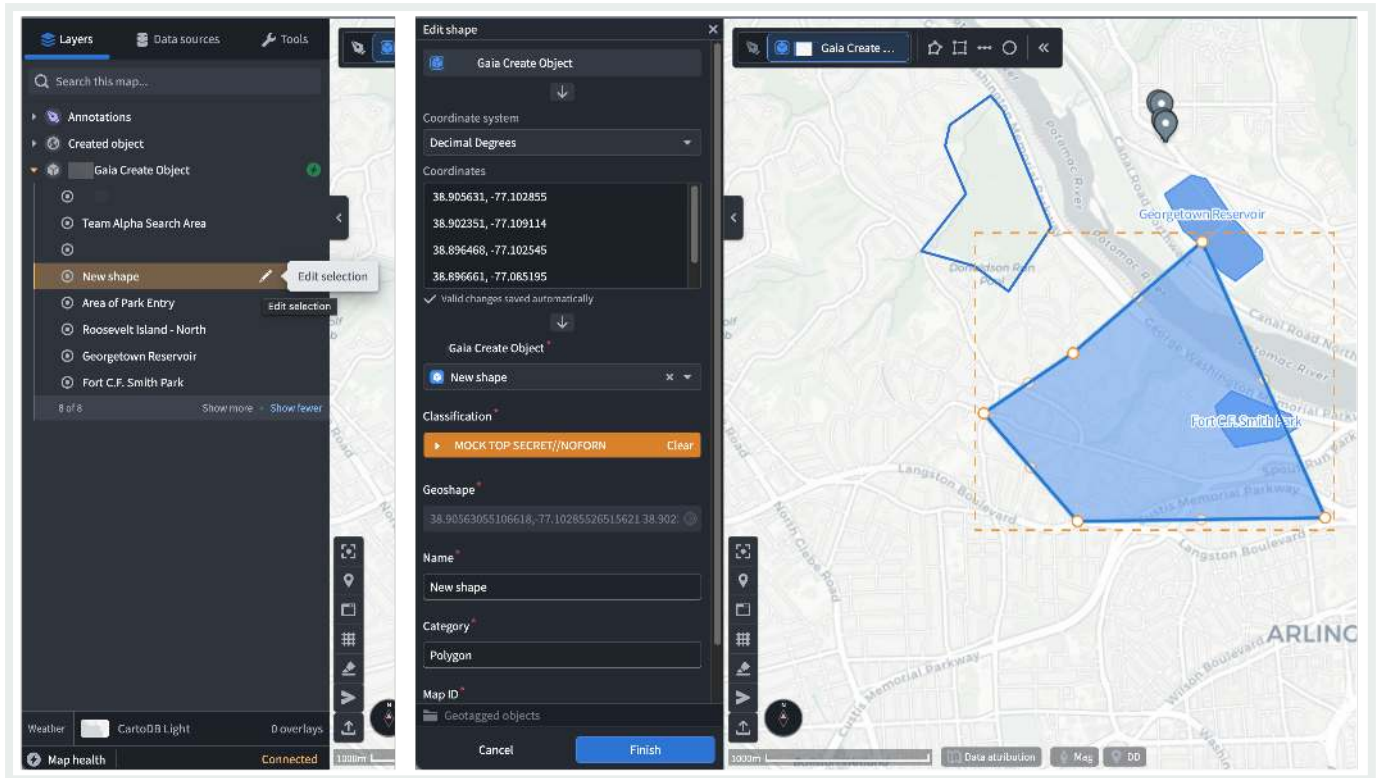
You can also view your object in Foundry's [Object Explorer](#).



Edit an existing shape object

To edit an existing shape object, hover your cursor over the object in the left panel and choose **Edit selection** to launch the **Edit shape** window, which you can also access by clicking the **API Reference** icon on your map.

Send feedback



Select and drag any vertex to adjust the shape, or select a point within the shape's boundary to drag the shape to another location on your map. When you are finished making changes, choose **Finish** at the bottom of the **Edit shape** window.

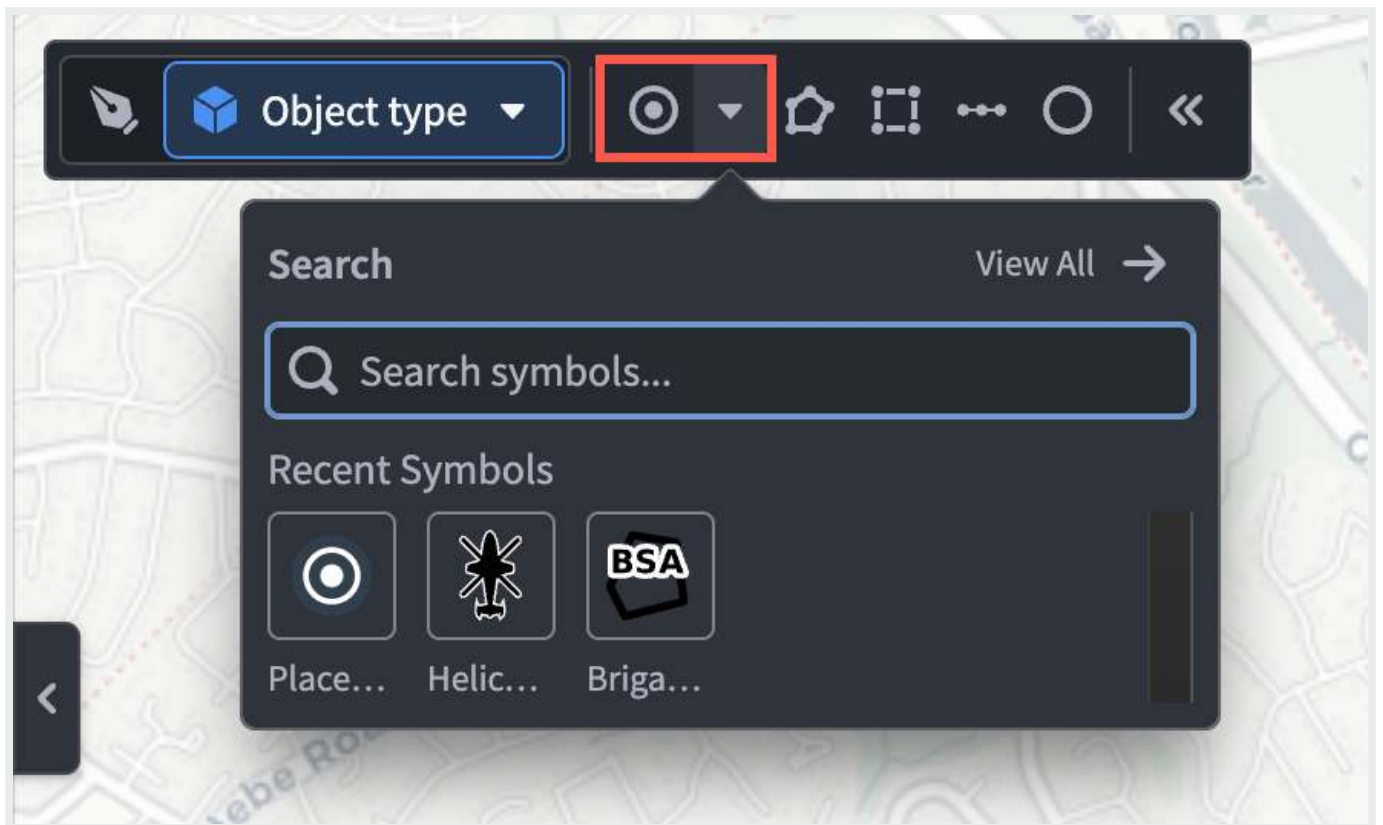
Drop a new point on your Gaia map and tag it to an object type

Before you begin, review the [entry points for creating objects](#) to understand how the object type and drawing tool you select determine which object types are available to you.

With your Gaia map open, select the down arrow to render the symbol **Search** menu, where you can choose an available symbol to drop anywhere on your map. If you select one of the available **Tactical Graphics** symbols, then you will only be able to create an object within an object type that implements the [MILSTD 2525C Symbol interface](#). Gaia will automatically populate the **Coordinates** input box based on your map's coordinate system preference.

API Reference ↗

Send feedback



i Select **File > Preferences > Coordinate system** from Gaia's top ribbon to update the default coordinate system.

The **Create symbol** window will appear in Gaia's left panel, where you can search for and select your object type that implements the Gaia Geopoint Creatable interface. Next, optionally adjust the **Bearing (mag)** of your geopoint before entering a **Title** for your object. Choose **Finish** to save the dropped symbol to your ontology as an object.

Once you save your geopoint as an object, you can interact with the new map layer and view the object in Foundry's Object Explorer [in the same manner you can for a shape drawn on your map](#).

Edit an existing geopoint object

You can edit an existing geopoint object [in the same manner you can for a shape drawn on your map](#).

[API Reference ↗](#)

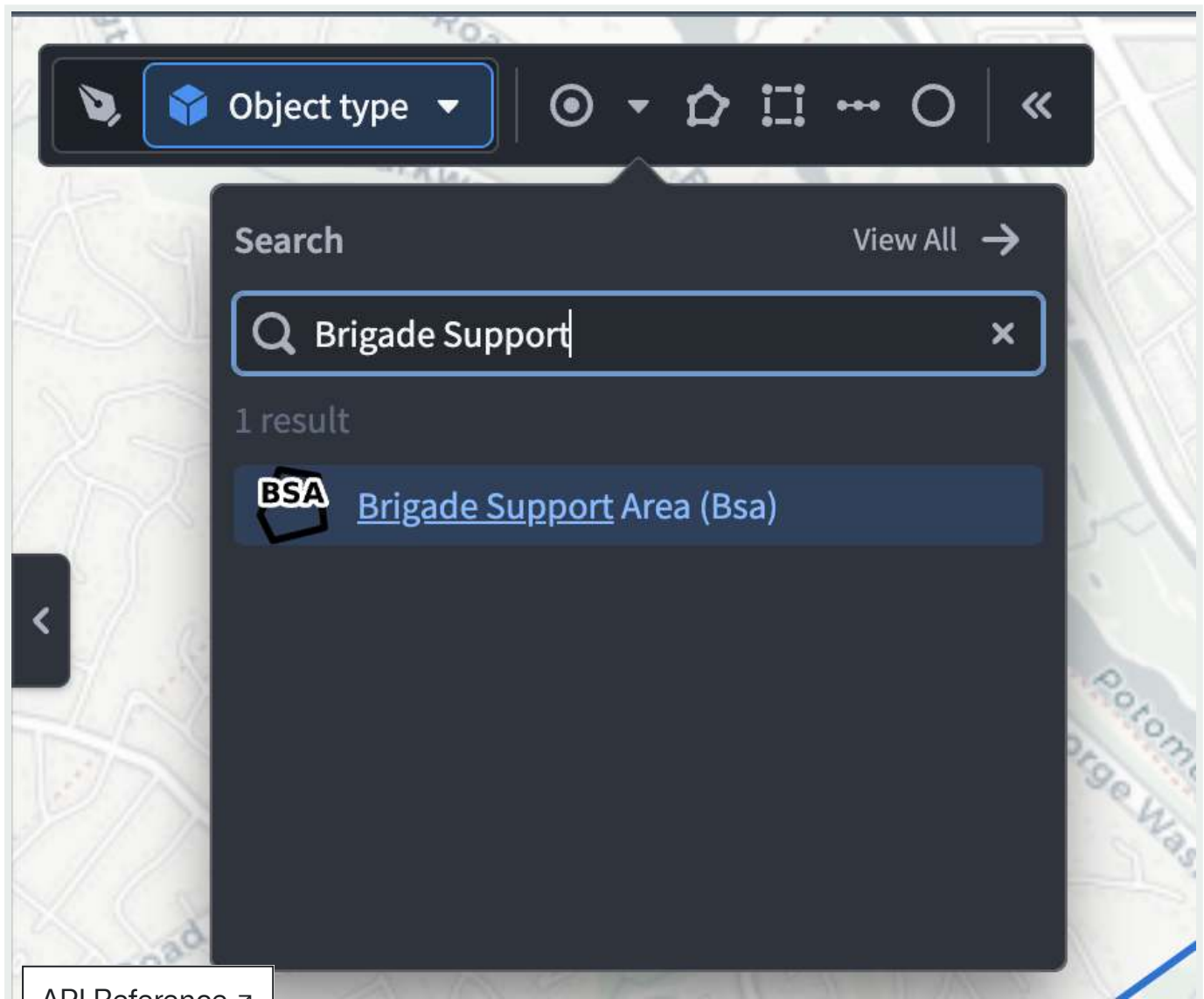
Configure a tactical graphic on your Gaia map
an object type

[Send feedback](#)

⚠ Your Foundry enrollment must contain MRS to tag a tactical graphic to an object type. Contact Palantir Support with questions about MRS installation or functionality.

Before you begin, review the [entry points for creating objects](#) to understand how the object type and drawing tool you select determine which object types are available to you.

[Similar to adding a new geopoint to tag as an object](#), select the down arrow to render the symbol **Search** menu, where you can choose an available tactical graphic to drop anywhere on your map. Type the name of your desired tactical graphic in the **Search symbols...** input box, such as `Brigade Support Area`.

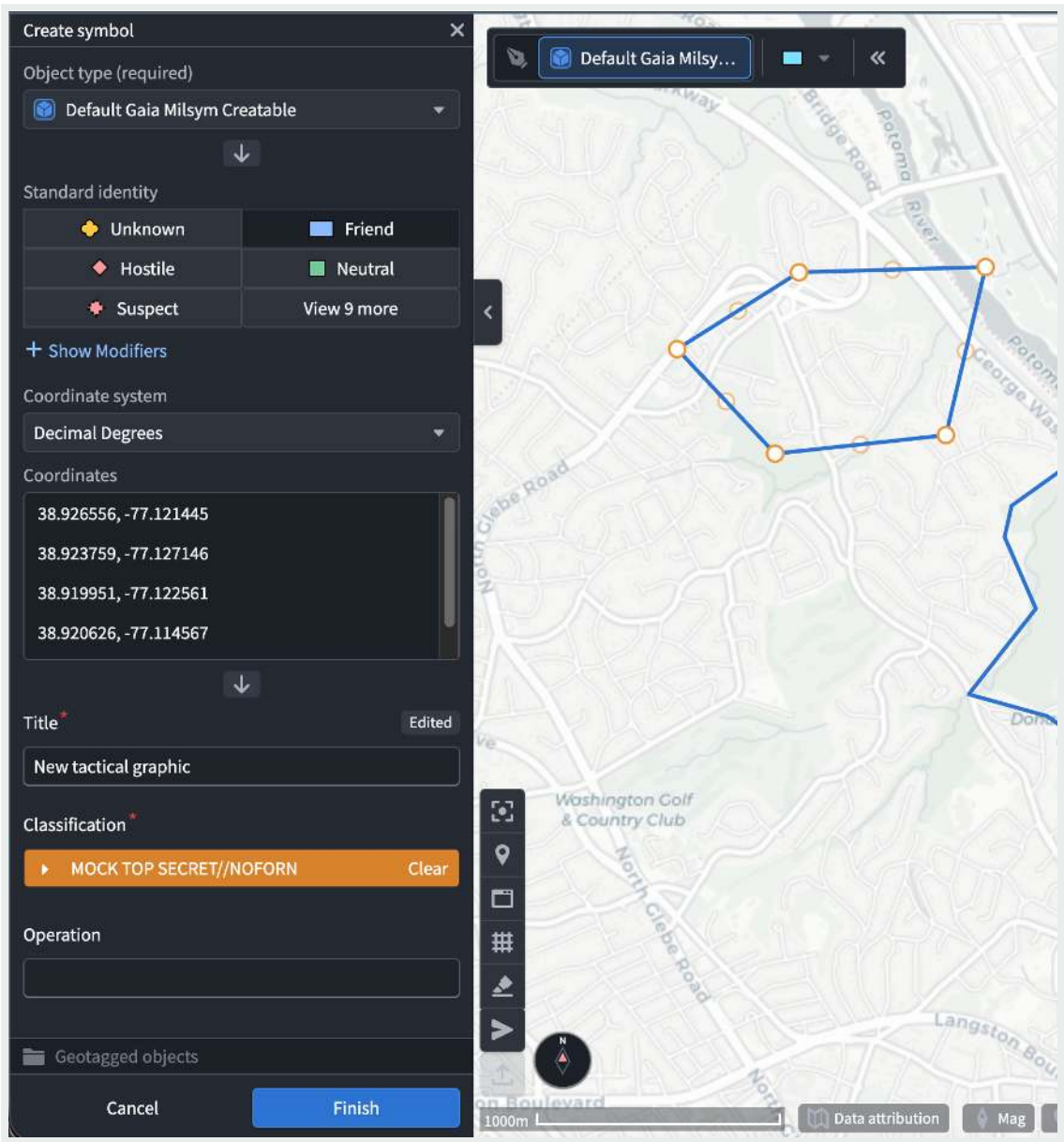


[API Reference](#) ↗

The **Create symbol** window will appear in Gaia's left panel, and you can find [shape creation instructions](#) above to add your tactical graphic to the map and tag it to an

[Send feedback](#)

object type.



Edit an existing tactical graphic object

You can edit an existing tactical graphic object by following the [shape object editing instructions above](#).

Promote an annotation to an object

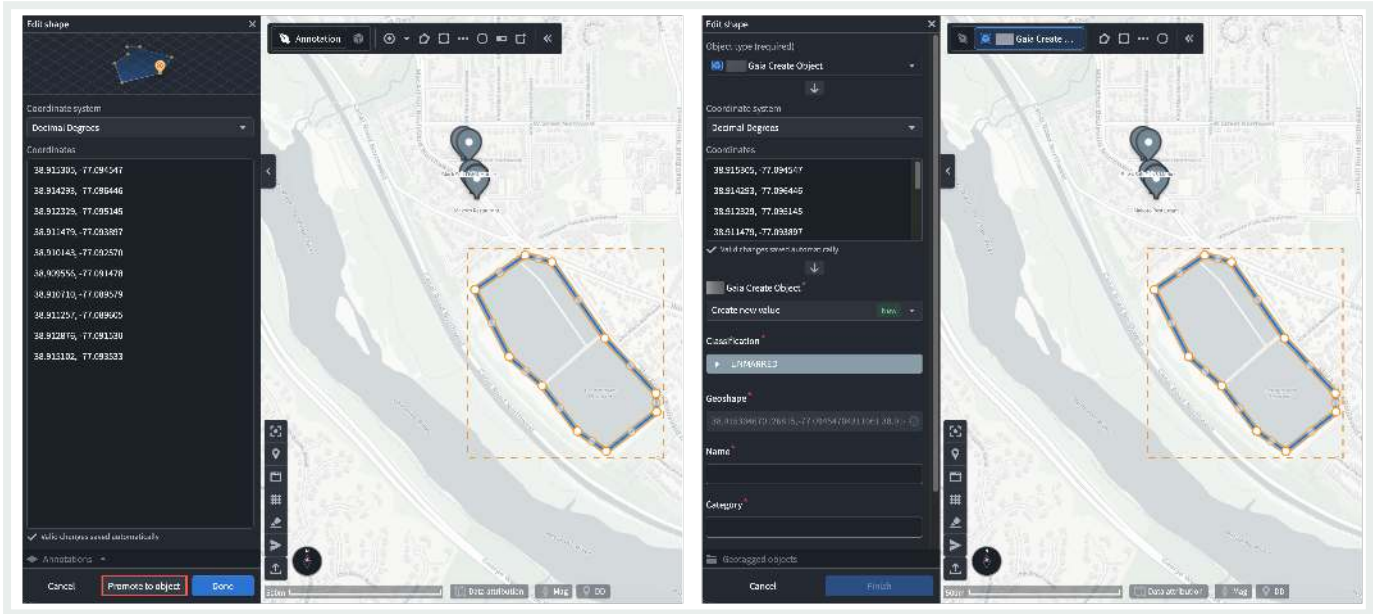
In

API Reference ↗

 is a shape, point, or symbol that is local to your current map. You can promote annotations on your map to objects in your ontology for use on other maps created by other users who can access their object type.

Send feedback

To tag an existing annotation as an object, double-click the annotation on your map or hover your cursor over the annotation's pencil icon in the **Layers** tab of the left panel to launch the shape, point, or symbol edit window. Next, select **Promote to object**, choose your object type from the **Object type** dropdown menu, and complete the action form in the shape, point, or symbol edit window.



To learn more about the various methods you can use to add data *from* your ontology to Gotham, review the existing [geospatial data integration documentation](#).

Metadata reference

An object type is represented in the Ontology by the following metadata:

- **ID:** A unique identifier of the object type, primarily used to reference objects of this type when configuring an application. For example, `employee` may be the ID of the `Employee` object type.
- **RID:** An automatically generated unique identifier for every resource in Foundry. An object type's RID will be referenced in error messages across the platform.
- **Icon:** A picture and color used as a visual identifier of the object type that will appear in user applications when a user views an object of this type. For example, the person icon may be used to depict the `Employee` object type.
- **Display name:** The name shown to anyone accessing an object of this type in user applications. For example, the display name for the `Employee` object type may be `Employee`.
- **Plural display name:** The name shown to anyone accessing multiple objects of this type in user applications. For example, the plural display name for the `Employee` object type may be `Employees`.
- **Description:** Explanatory text about the object type that anyone can read in user applications. For example, the description of the `Employee` object type may be `All full-time and part-time employees of Organization X`.
- **Groups:** A group is a label that helps you categorize your object types. For example, the `Employee` object type may belong to groups `HR` and `Employee 360`.
- **API name:** The name used when referring to the object type programmatically in code. For example, the API name of the `Employee` object type may be `Employee`. Read more about [API names](#).
- **Visibility:** An indication to user applications for how prominently to display the object type. An object type will lead applications to show this object type first to `normal`. A hidden object type will not appear in user applications. For example, a `HiddenEmployee` object type will have visibility `normal`.

API Reference ↗

Send feedback

- **Status:** A signal to users and other Ontology builders about where in the development process the object type stands. It can be `active`, `experimental`, or `deprecated`. By default, the `Employee` object type will have status `experimental`. Read more about [statuses](#).
- **Index status:** The status of the last reindex of the object type and its backing datasources. It can be `success`, `failed`, or `not started`. Read more about [index statuses](#).
- **Writeback:** An indication of whether the object type has a writeback dataset generated, and whether allowing end users to make edits to objects of this type is `enabled` or `disabled`. Read more about [writeback datasets](#).

[Learn more about creating and configuring an object type in the Ontology and about validation requirements for object type metadata.](#)

[Learn more about Properties \(characteristics of an object type\).](#)

PREVIOUS
← Create Ontology objects from Gaia

NEXT
Properties / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Properties

A **property** of an [object type](#) is the schema definition of a characteristic of a real-world entity or event. A **property value** refers to the value of a property on an object, or a single instance of that real world entity or event.

For example, in the Ontology Manager, an `Employee` object type may have properties `employee number`, `start date`, and `role`. The notional employee “Melissa Chang” may have property values “11502” for `employee number`, “October 9, 2016” for `start date`, and “software engineer” for `role`.

Similarly, in the Ontology Manager, a `Flight` object type may have properties `departure date`, `arrival date`, and `passenger count`. The object “JFK → SFO 24-02-2021” may have property values “24-02-2021” for `departure date`, “25-02-2021” for `arrival date`, and “150” for `passenger count`.

The concepts underpinning the Ontology have analogous concepts in the structure of a dataset. The definition of a property in the Ontology is analogous to that of a column in a dataset, while the definition of a property value is analogous to that of a field in the dataset. For example, an `Employee` dataset may have columns for `departure date`, `arrival date`, and `passenger count`. In this case, a single field will have the value “11502” for the `employee number` column of the row for employee “Melissa Chang.”

Rather than being an abstract data model, the Foundry Ontology maps each ontological concept to an organization's actual data, enabling this data asset to power real-world applications. Property values are created and displayed in user applications by adding backing datasources to an object type in the Ontology Manager. To create property values for `employee number`, `start date`, and `role` on objects of type `Employee`, an object type, add backing datasources to the `Employee` object type and feed their data from a data repository and other enterprise data into the Ontology.

API Reference ↗



Send feedback

Supported property types

Property base type	Valid as title key?	Valid as primary key?	Notes
Commonly used: String , Integer , Short	Yes	Yes	
Time-based: Date , Timestamp	Yes	Discouraged	Typically, time values are inappropriate as primary keys, due to potentially unexpected collisions / uniqueness based on the storage format differing from the display format. In most cases, we recommend using String instead.
Number-like: Boolean , Byte , Long	Yes	Discouraged	Boolean limits your object type to two object instances. Byte properties can only be assigned in Actions via an Integer parameter, so in most cases we recommend using Integer properties instead. Long has representational issues in Javascript ↗ , so not all frontend libraries and code work well with Long values greater than 1e15. In most cases, we recommend using String instead.
Float-like: Float , Double , Decimal	Yes	No	
Vector	No	No	
Array	Yes	No	Array properties cannot contain null elements. If the inner type of the Array is not a valid title property, the Array also cannot be used as the title property.
API Reference ↗			
☰			
			Send feedback

Property base type	Valid as title key?	Valid as primary key?	Notes
			Nested arrays are not supported in Object Storage v2.
Struct	No	No	Struct properties do not support nesting, and fields cannot be arrays. See the struct documentation for detailed information on supported field types.
Media Reference, Time Series, Geotemporal Series, Attachment	No	No	
Geopoint	Yes	No	Geopoint values are stored as a comma-separated string in the format latitude, longitude (for example, 57.64911,10.40744). See also Use geospatial data in the Ontology , Create GeoPoint , Convert to Ontology GeoPoint , and Is valid Ontology GeoPoint .
Geoshape	No	No	
Marking	No	No	
Cipher	Yes	No	

To learn more about base types, see [base types](#).

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Edit object type properties

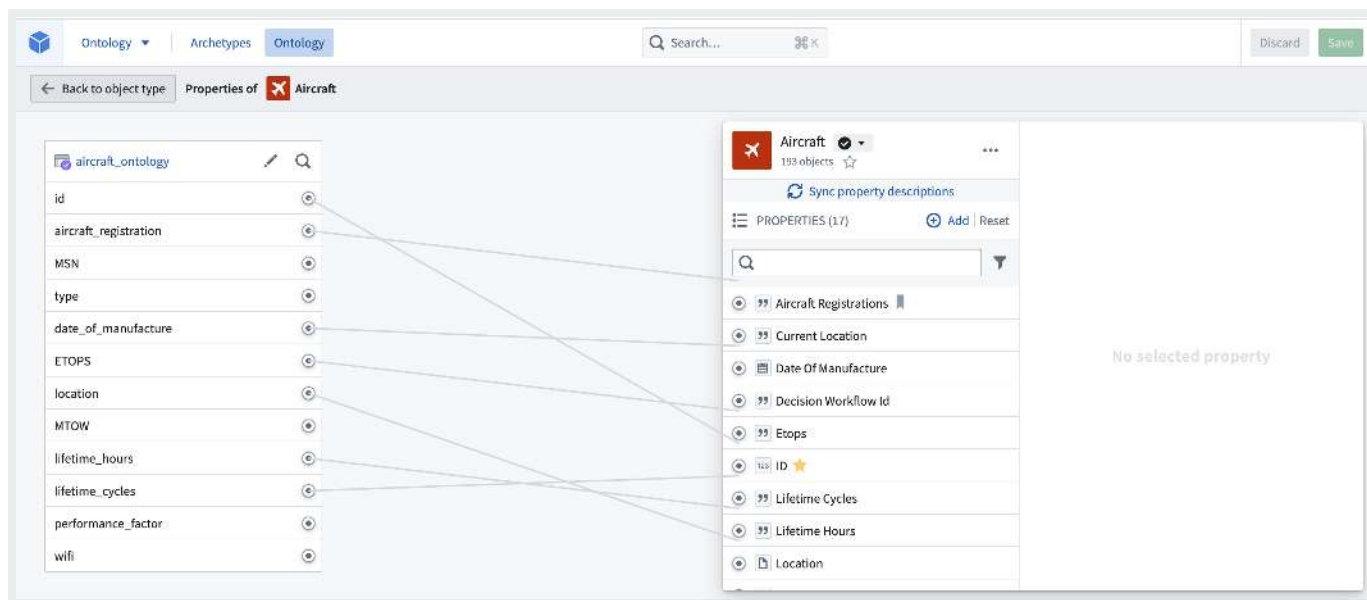
Delete a property

From within the property editor, in the properties pane, hover over the property you want to delete and select **Delete property**.

- Note that the deletion of the property only takes effect after you save your changes, and will break any views or applications referencing the property.
- Properties with an `active` status **cannot** be deleted.
 - Read more about [statuses](#).

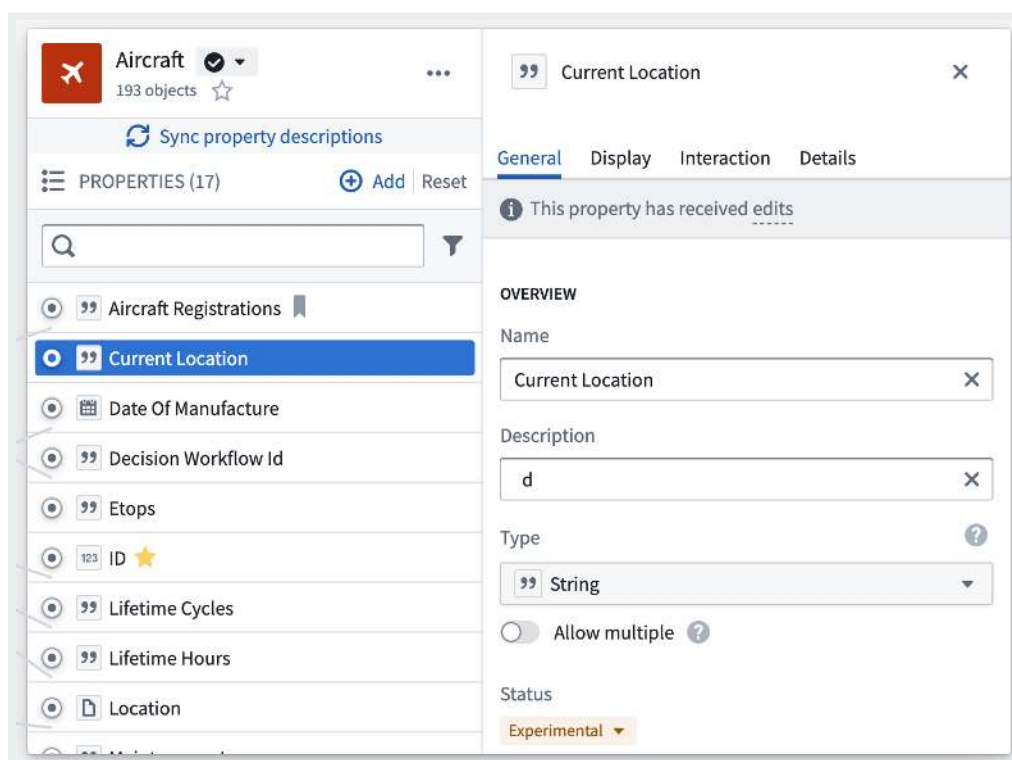
Change the column backing a property

From within the property editor, in the properties pane, hover over the property you want to unmap and select **Unlink property**. To link the property to a new column, hover over the property and select **Map to a column**.



Edit a property type's metadata

You can edit metadata for a property type by selecting the property type, as shown in the image below.



[API Reference ↗](#)

The available options for editing property metadata are clustered into... which give access to the following configurations:

[Send feedback](#)

1. **Display name and description:** Select into the existing display name or description to edit the text.
2. **Status:** Select the existing status to open a dropdown of available statuses. Choose from the `deprecated`, `experimental`, and `active` statuses.
 - Read more about [statuses](#).
3. **API name:** Select into the existing API name to change its value.
 - Note that you **cannot** change the API name for properties with an `active` status.
 - Read more about [statuses](#).
 - Read more about [valid API names](#).
4. **Keys:** Indicate whether a property is the object type's title key or primary key.
 - Note that you **cannot** change the primary key of an object type with an `active` status.
 - Read more about [keys](#) and about [statuses](#).
5. **Value formatting:** Apply a special formatter to the values of a property to make them more readable in applications.
 - Read more about [value formatters](#).
6. **Conditional formatting:** Apply rules to a property that dictate how it will be rendered in applications.
 - Read more about [conditional formatting](#).
7. **Property base type:** Select the property's base type from the dropdown. The type of the property constrains the possible set of operations that can be done on the property's values.
 - For example, a property with base type `timestamp` can be shown in a timeline widget in Object Explorer.
 - You will receive an error if the type of a property is not compatible with the type of its backing column.

 If you make a change to object property types, you must also update the type expected by Actions that interact with property on that object. To do this, open the Action in Ontology Manager and edit the expected type.

8. **Type classes:** Apply type classes as additional metadata that can be interpreted by applications.

API Reference ↗

[Classes documentation](#) for a list of available type classes.

Generate render hints from the supplied checklist that you want applied to a property in order to improve how a property value is rendered and [Send feedback](#)

Storage v1 (Phonograph).

- See the [render hints documentation](#) for descriptions of the available render hints.

10. **Visibility:** Select the existing visibility to open a dropdown of available visibilities. A **prominent** property will lead applications to show this property first to users. A **hidden** property will not appear in user applications.

After making a change to property metadata, initiate a re-index of the affected object to update the Ontology.

Bulk edit multiple properties

You can select multiple properties in the property editor by holding the **Cmd/Ctrl** key while selecting properties. Once multiple properties are selected, the following bulk editing actions become available:

- Changing base type.
- Adding/removing of type classes.
- Changing render hints.
- Changing visibility.
- Adding/removing value formatting.

PROPERTY NAME	STATUS	VISIBILITY	RENDER HINTS	BASE FORMATTER
☐ ID Primary key	Active	Normal	3 hints	Numeric formatting
☐ Aircraft Registrations Title	Experimental	Prominent	3 hints	No formatting
☒ Current Location	Experimental	Prominent	3 hints	No formatting
☐ Date Of Manufacture	Experimental	Prominent	3 hints	Date and time format
☒ Decision Workflow Id	Experimental	Normal	3 hints	No formatting
☐ Etops	Active	Normal	4 hints	No formatting
☐ Lifetime Cycles	Active	Normal	3 hints	No formatting

You can also bulk edit some of the above fields from outside of the property editor, by selecting the **Properties** page from the sidebar of an object type view. Simply select the [API Reference](#) the properties you want to edit and a new row will appear at the top of the table with options for bulk editing.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Add value formatting

Value formatting refers to applying a special formatter to the value of a property, transforming the raw value to a more readable version. In the image below, the left-hand side (**Before**) shows the `weight` and `value` columns without any formatting. The right-hand side (**After**) has a unit (“kg”) applied to the `weight` column and the `value` column is displayed in a more compact form with a currency sign (“\$100K”). These are both examples of numeric formatting. The Ontology also supports date and time formatting, as well as user ID formatting, resource RID formatting, and artifact GID formatting.

The image shows two side-by-side screenshots of a data table interface, labeled 'Before' and 'After'. Both screenshots show a table with two columns: 'WEIGHT' and 'VALUE'. The 'Before' table shows raw numeric values, while the 'After' table shows the same values formatted with units and currency symbols.

WEIGHT	VALUE
10,000	100,000
5,000	150,000
20,000	130,000
3,234	220,000
2,456	230,000
12,000	10,000,000,000
11,334	43,200,000

WEIGHT	VALUE
10,000 kg	\$100K
5,000 kg	\$150K
20,000 kg	\$130K
3,234 kg	\$220K
2,456 kg	\$230K
12,000 kg	\$10B
11,334 kg	\$43M

Supported value formatting

[API Reference ↗](#)[Send feedback](#)

Type	Description
Numeric formatting	Add currencies/units/prefixes, various types of notations (compact, scientific), and percentages. See the numeric formatting section for more details.
Date and time formatting	Render timestamps and dates in a specific format as well as in a specific timezone.
Foundry ID formatting	Display a Foundry ID as a user's first and last name or group name.
Resource RID formatting	Display a Foundry resource ID (RID) as an icon and resource name, with a clickable link that routes to that resource.
Artifact GID formatting	Display an artifact global ID (GID) as an icon and artifact name, with a clickable link that routes to that artifact.

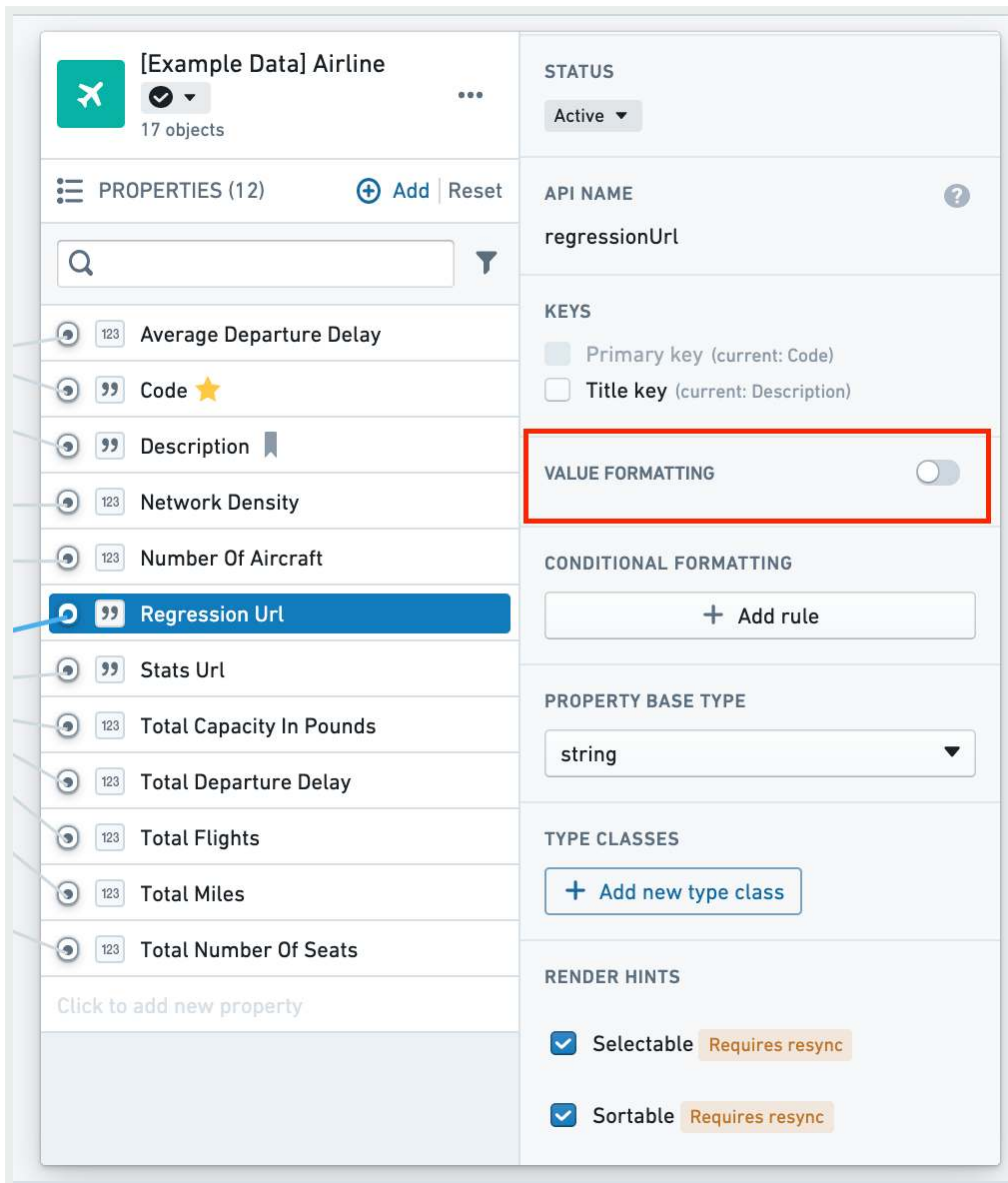
Add value formatting

In the property editor:

1. Select the property to which you want to add value formatting.
2. On the right hand side panel of the properties pane, you will see a type of formatting depending on the base type of the property (value formatting, numeric formatting, date and time formatting, etc.). Toggle on the formatting.

[API Reference ↗](#)

[Send feedback](#)



3. Additional formatting options are available for numeric formatting and date and time formatting, as described below:
 - [Numeric formatting options](#)
 - [Date and time formatting options](#)
4. As you select the available formatting options, you will see a preview for how values of the property will be rendered with the new formatting applied.

Numeric formatting options

[API Reference ↗](#)

[Send feedback](#)

✈️

[Example Data]

Aircraft

7,284 objects

PROPERTIES (14)

+

Add

Reset

🔍

🔼

🕒

Acquisition Date

🟢

🏷️

Aircraft Status

🟢

🔢

Capacity In Pounds

🏷️

Carrier Code

🏷️

Display Name

📌

🔢

Manufacture Year

🏷️

Manufacturer

🏷️

Model

🏷️

Model Type

🔢

Number Of Seats

🏷️

Operating Status

🏷️

Serial Number

🏷️

Tail Number

★

🏷️

Unique Carrier Name

NUMERIC FORMATTING

🔵

Base type

Standard unit

▼

Pound

▼

▶

▼ Additional options

Use grouping

📘

✕

🔵

Use grouping

+

Notation

+

Negative to parenthesis

+

Minimum integer digits

+

Minimum fraction digits

+

Maximum fraction digits

+

Minimum significant digits

+

Maximum significant digits

PREVIEW RESULT

123456

Capacity In Pounds

123,456 lb

Name	Description	Usage
Numeric formatting	On/Off toggle.	Toggle this to remove/add numeric formatting.
Base type	Contains various types of formatting available (Currency, Unit, Percentage, Prefix/Suffix, Fixed Values) as well as examples and descriptions of each type.	If Capacity in Pounds has an associated unit, select "Unit" from this dropdown.
Use grouping	Adds locale-aware comma separator.	To

Send feedback

Name	Description	Usage
		123,456.
Notation	Contains Compact/Scientific and Engineer notations.	Choose compact to approximate values, like 123K.
Maximum fraction digits	The most digits to show after the decimal point. Values with more decimals are rounded to this length.	Set to 2 to display 3.14159 as 3.14 .
Minimum fraction digits	The fewest digits to show after the decimal point. Shorter values are padded with trailing zeros.	Set to 2 to display 3.5 as 3.50 .
Maximum/Minimum significant digits	The most and fewest significant digits to display, counted from the first non-zero digit.	Set maximum significant digits to 3 to display 3.14159 as 3.14 .
Minimum integer digits	The fewest digits to show before the decimal point, padded with leading zeros.	Set to 2 to display 5 as 05 .
Preview result	View and test numeric formatting.	Add any number in the input that is similar to what you'd expect to see in your property's values for a preview of the formatting.

Date and time formatting options

[API Reference ↗](#)

[Send feedback](#)

[Example Data]

Aircraft

7,284 objects

✓

...

API NAME

acquisitionDate

KEYS

☐ Primary key (current: Tail Number)
 ☐ Title key (current: Display Name)

DATE AND TIME FORMATTING

Date and time

▼

DESCRIPTION

Both the date and time.

EXAMPLES

From 4/15/2021 to

Thu, Apr 15, 2021, 3:50:33 PM

From 1/1/2020 to

Wed, Jan 1, 2020, 12:00:00 AM

PROPERTIES (14)

+ Add | Reset

▼

Acquisition Date

Aircraft Status

Capacity In Pounds

Carrier Code

Display Name

Manufacture Year

Manufacturer

Model

Date

Date and time

Date and time, short

ISO instant

Relative to now

Time

Name	Description	Example
Date	Only the date (no time)	Wed, Jul 22, 2020
Date and time (long)	Both the date and time, in long form	Wed, July 22, 2020, 1:00:00 PM
Date and time (short)	Both the date and time, in short form	Jul 22, 2020, 1:00 PM
ISO instant	Both the date and time (ISO 8601 format)	2020-07-22T13:00:00.000Z
Relative to now	The date relative to right now	8 minutes ago
Time	Only the time (no date)	1:00 pm

i When formatting **Relative to now**, applications will only format in relative terms up to 24 hours ago. After this, it will render in **Date and time (short)** form with the day of the week: Wed, Jul 22, 2020, 1:00 PM.

API Reference ↗

Send feedback

DATE AND TIME FORMATTING

Relative to now

PREVIEW RESULT

4/12/2021, 4:33:39 PM

Acquisition Date
Mon, Apr 12, 2021, 4:33 PM

Time zones

If you are formatting a timestamp, you can specify which timezone to render the timestamp, either as a static timezone that you input, or as the application user's current timezone. When entering a static timezone, you can search for a timezone by inputting the UTC offset or the locale name.

User ID formatting

Value formatting can be applied to strings that are Foundry/Multipass user IDs or group IDs and convert them to display the user's first and last name or the group name by selecting the **Multipass username** option. This value formatting option is typically used when you have created an [Action](#) that edits a property and stores a user ID or group ID in one of the property fields. In the backing data, this information will be stored as the user's Foundry user ID or group ID, and the value formatting can be applied to render the user's name or the group instead of the ID.

FAQ

[API Reference ↗](#) with existing type classes?

Send feedback

Value formatting takes precedence over existing type classes in the UI. If you have both configured, value formatting will be displayed. You can however, use value formatting on one property and type classes on another.

Will this work with editable properties in Object Views?

Value formatting is supported for properties configured for [inline edits](#). Properties with the legacy `hubble:editable` type class will disable value formatting.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

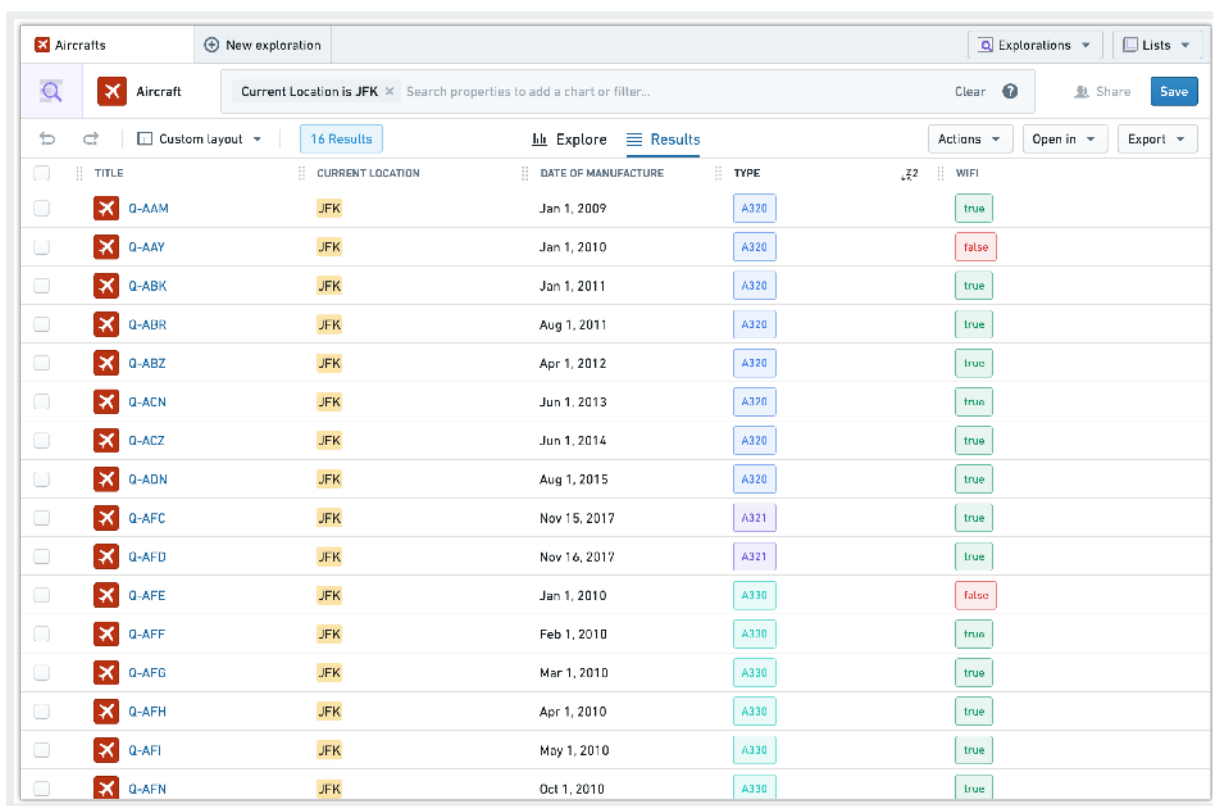
[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

Ontology building / Object and link types / Properties / Add conditional formatting

Conditional formatting enables the configuration of rules for any property and dictates how that property's values will be rendered (e.g. coloring, alignment, etc.) in user facing applications. When you configure conditional formatting in the Ontology Manager, the formatting rules will apply in Object Explorer, Object Views, Quiver, and Workshop.



The screenshot shows the Palantir Object Explorer interface for the 'Aircraft' object type. A filter 'Current Location is JFK' is applied, resulting in 16 results. The table displays columns for TITLE, CURRENT LOCATION, DATE OF MANUFACTURE, TYPE, and WIFI. The 'type' and 'wifi' columns are color-coded based on conditional formatting rules.

TITLE	CURRENT LOCATION	DATE OF MANUFACTURE	TYPE	WIFI
Q-AAM	JFK	Jan 1, 2009	A320	true
Q-AAY	JFK	Jan 1, 2010	A320	false
Q-ABK	JFK	Jan 1, 2011	A320	true
Q-ABR	JFK	Aug 1, 2011	A320	true
Q-ABZ	JFK	Apr 1, 2012	A320	true
Q-ACN	JFK	Jun 1, 2013	A320	true
Q-ACZ	JFK	Jun 1, 2014	A320	true
Q-ADN	JFK	Aug 1, 2015	A320	true
Q-AFC	JFK	Nov 15, 2017	A321	true
Q-AFD	JFK	Nov 16, 2017	A321	true
Q-AFE	JFK	Jan 1, 2010	A330	false
Q-AFF	JFK	Feb 1, 2010	A330	true
Q-AFG	JFK	Mar 1, 2010	A330	true
Q-AFH	JFK	Apr 1, 2010	A330	true
Q-AFI	JFK	May 1, 2010	A330	true
Q-AFN	JFK	Oct 1, 2010	A330	true

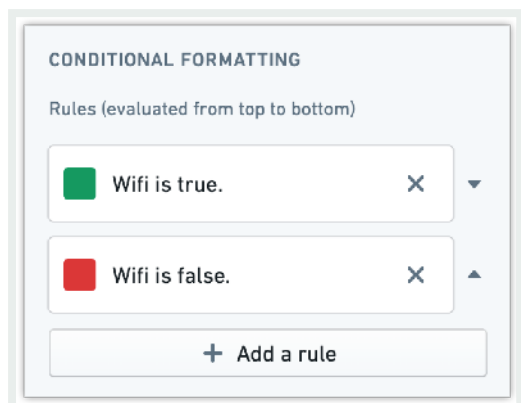
For the example `Aircraft` object type in Object Explorer, pictured above, the `type` and `wifi` properties have their values in colored boxes that are applied based on certain conditions. The main benefit of adding these is to make information easier to understand quickly. If an analyst was looking for all “A320” planes without wifi in “JFK”, just by glancing at the results above, we could tell that “Q-AAY” is the plane we’re after.

[API Reference ↗](#)

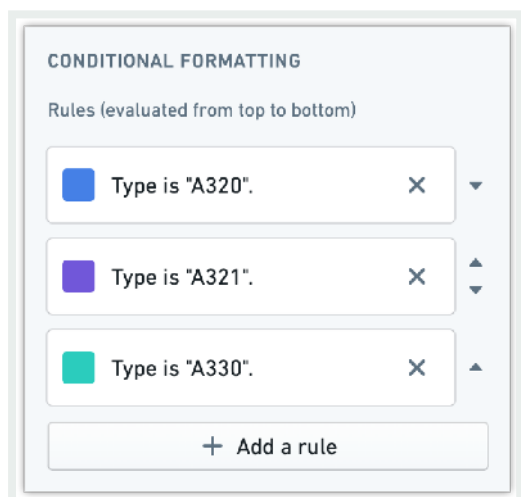
Learn how these conditions are applied.

[Send feedback](#)

- For property `wifi`, we assign green if the value of the property is “true” for each object in the table, and red if it is “false.”



- For property `type`, we assign colors based on exact match between “A320”, “A321” and “A330”.

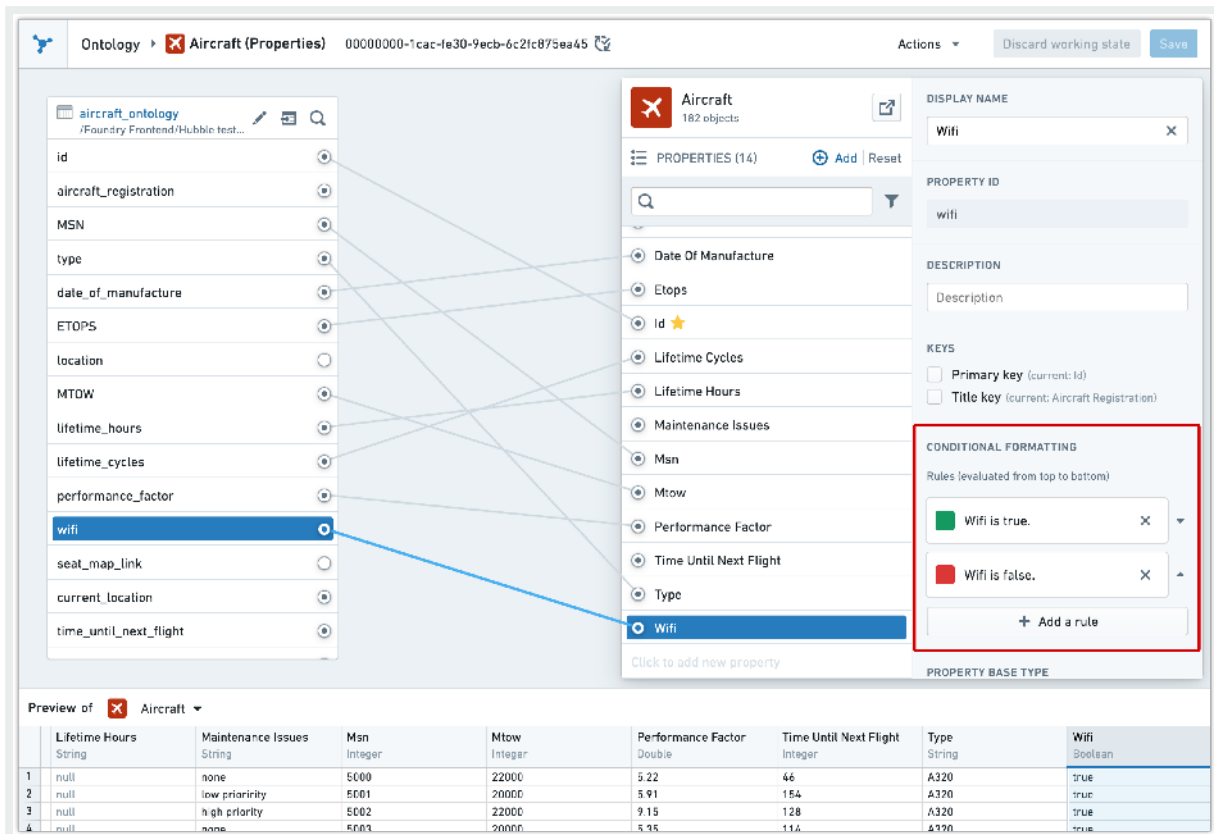


Add conditional formatting

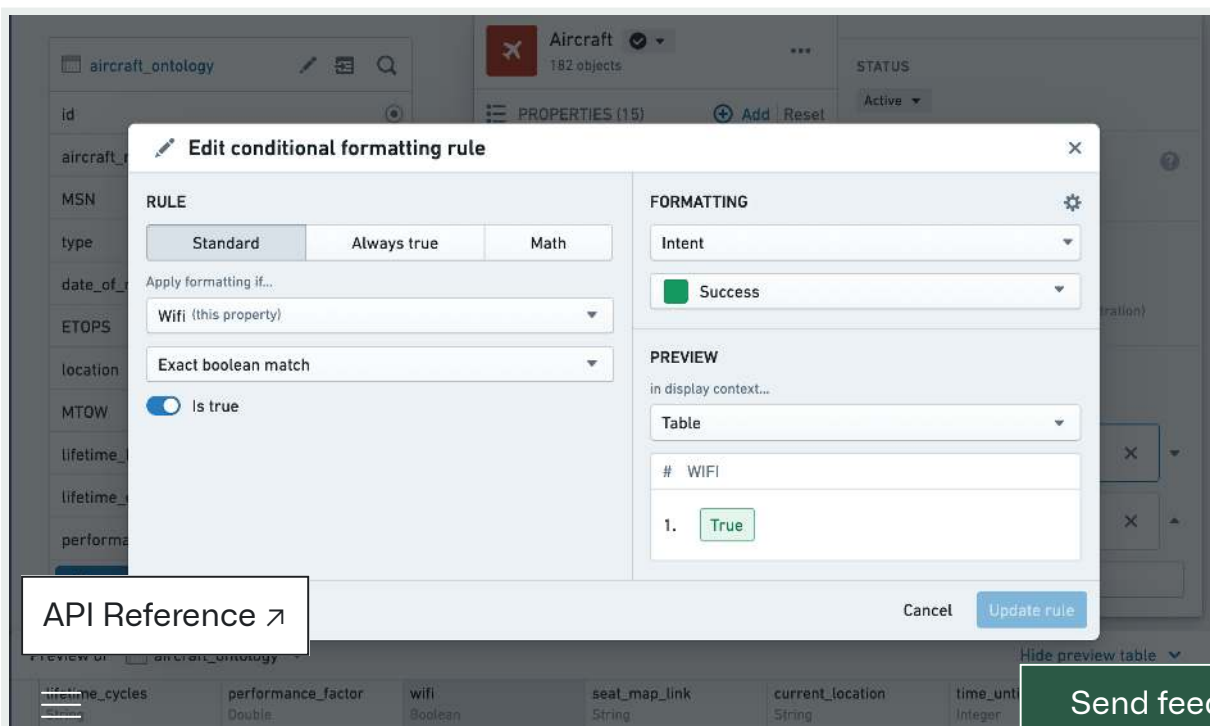
In the property editor:

1. Select the property to which you want to add conditional formatting.
2.  API Reference ↗ conditional formatting on the properties pane; select the **Add a rule** button.

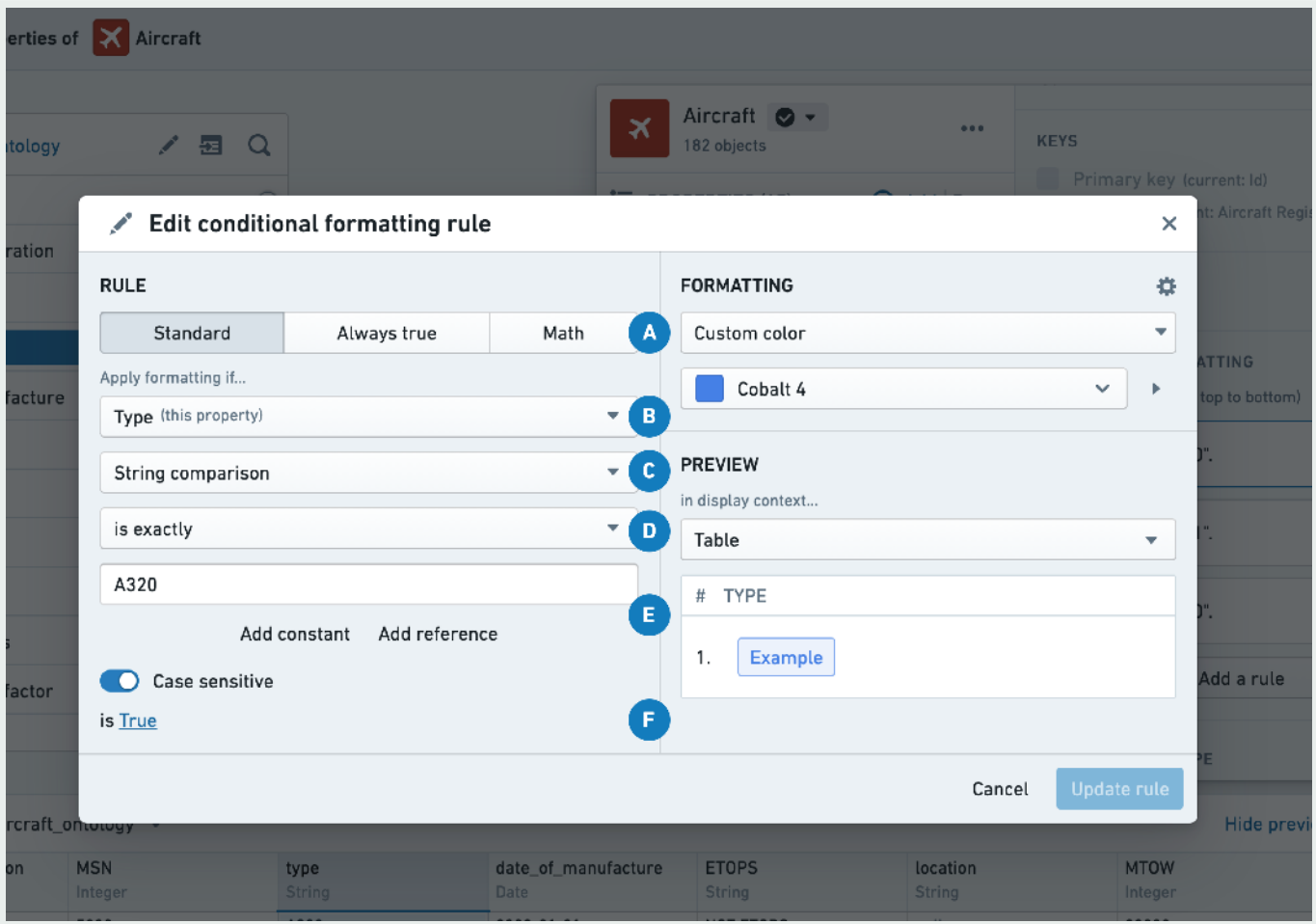
Send feedback



1. Click on the newly created default rule to open the **Edit conditional formatting rule** editor. [Read on for more information about the components of the Rule editor.](#)
2. Modify the rule.



Edit rules using the Rule editor



Label	Description	Usage
A	Switch between a Standard rule, an Always true rule, or a Math rule.	Use Always true as a fallback in case your other rules don't match. In the example above, we could have grey as the fallback case when neither of the <code>type</code> values match. Use a Math rule when you want to run math operators on some of your properties.
API Reference ↗	rule will always be applied to the property from which you selected Add a rule ; however, this dropdown allows you to	In the case above, assume we want to color the value for <code>Type</code> in red when the value <code>factor</code> drops underneath a certain

Send feedback

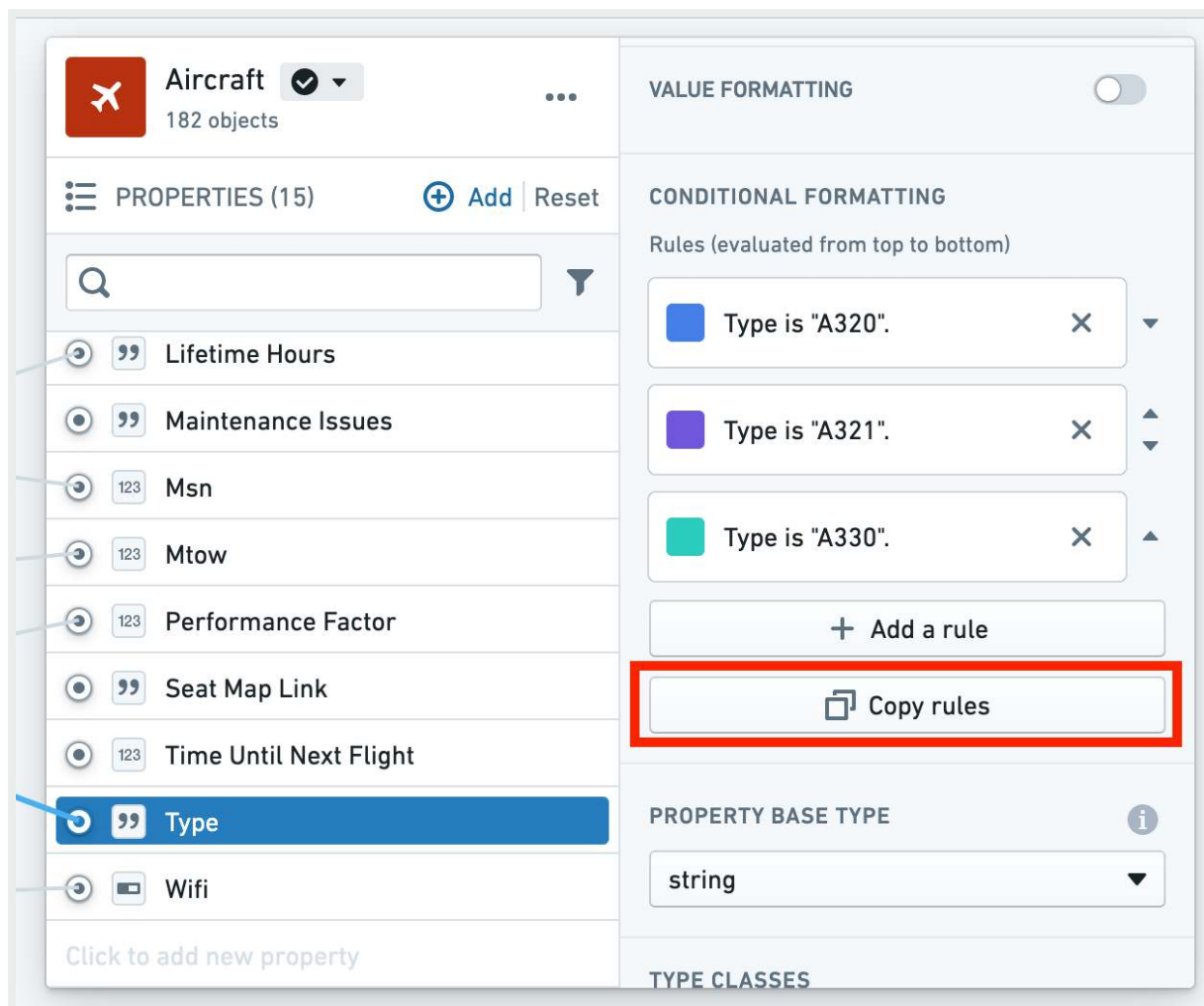
Label	Description	Usage
	choose to apply the rule based on the value of another property.	threshold. We would choose Performance factor in our logic instead of Type ; however, the color would still show on Type .
C	Types of comparisons available are based on the type of the property. For example, for strings String comparison and Is null are available. For numeric types, Numeric range or Exact numeric match are available.	To color the type in grey if the value is null, select this dropdown and choose Is null instead of String comparison .
D	Subtypes of comparisons, String comparison has Is exactly , Contains , Starts with , etc.	Use this to color all plane type values that Start with "A32".
E	Compare against a constant or a property reference.	In this case, we are specifically looking for the constant "A320", but we could also add a reference from another property from the same object type.
F	Toggle between a True or False rule.	To color all planes in blue that are not A320, switch this to False .
Formatting	Use Blueprint colors and intents or add your own custom color. You can also switch alignment.	Switch between hex, RGB or Blueprint colors based on need; you can also align the boxes on the right hand side for easier readability for numbers.
Preview	View how conditional formatting appears in various contexts.	Preview an Objects table or a Property card .

 [API Reference ↗](#)

In the property editor:

[Send feedback](#)

1. Select the property from which you want to copy the conditional formatting rule.
2. You will see conditional formatting on the properties pane; select the **Copy rules** button to open the **Copy rule** dialog.

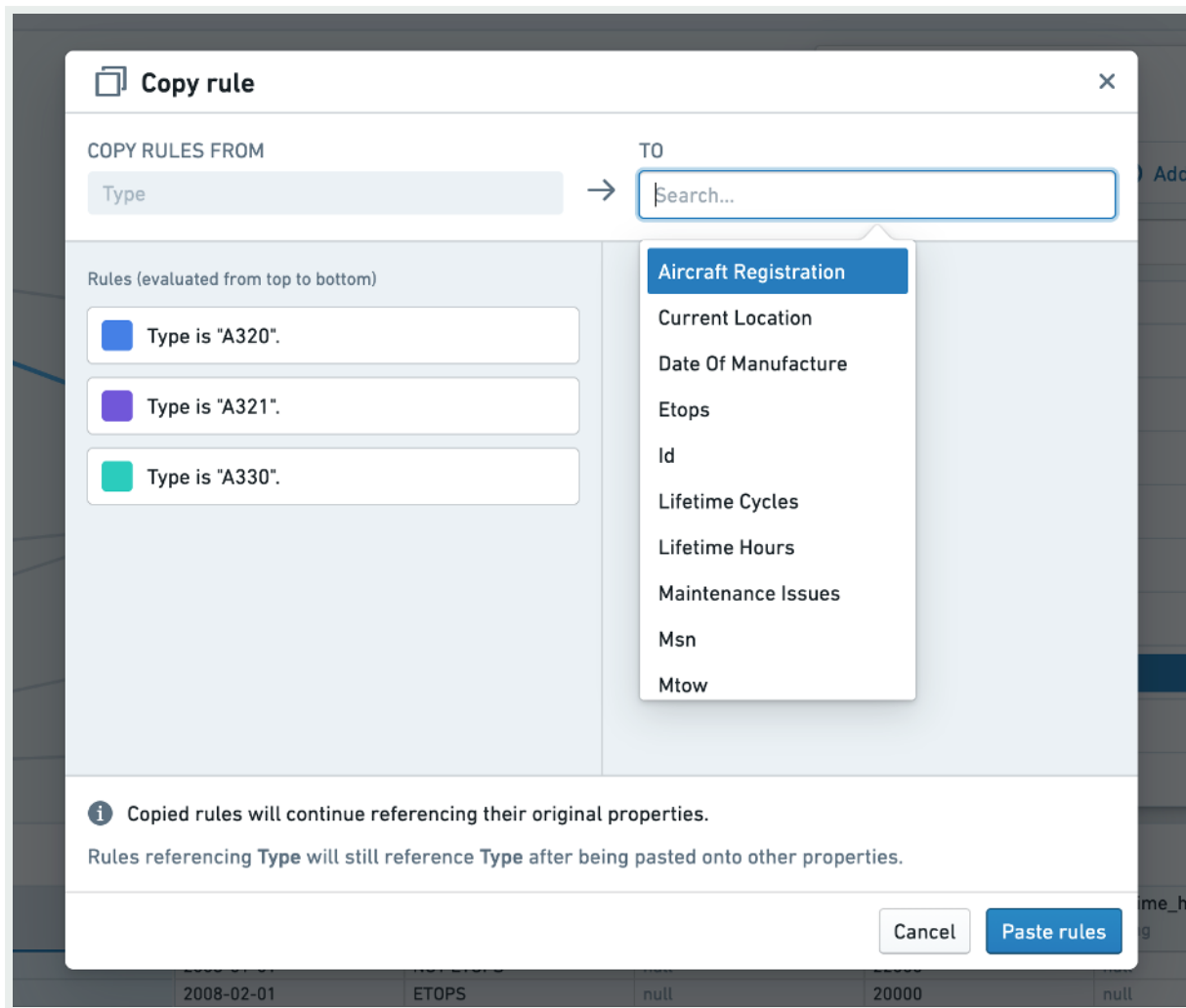


3. Select the properties to which you want to copy the conditional formatting rules.

i If the properties you are copying to already have their own conditional formatting rules, they will be overwritten by the new rules.

[API Reference ↗](#)

[Send feedback](#)



Copied rules will continue referencing their original properties. For example, if a rule states that `wifi` values should appear green when “true,” and that rule is copied to the `customer experience` property, values of the `customer experience` property will also be green when the object’s `wifi` value is “true.” To change the property a rule references, simply select the rule and choose a new property from the **Property** dropdown in the rule editor.

FAQ

Will this work with existing type classes?

Conditional formatting takes precedence over existing type classes (with one exception detailed in the [following question](#)). If you have both configured, conditional formatting will be displayed. You can however, use conditional formatting on one property and not on another.

API Reference ↗

Send feedback

Will this work with editable properties in Object Views?

Conditional formatting is supported for properties configured for [inline edits](#). Conditional formatting is disabled for properties with the legacy `hubble:editable` property type class.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

Metadata reference

A property is represented in the Ontology by the following metadata:

- **ID:** A unique identifier of the property, primarily used to reference the property when configuring an application. For example, `start-date` may be the ID of the start date property.
- **Display name:** The name shown to anyone accessing property values for this property in user applications. For example, the display name for the `start date` property may be `Start date`.
- **Description:** Explanatory text about the property that anyone can read in user applications. For example, the description of the `start date` property may be `The day the employee began new hire training`.
- **RID:** An automatically generated unique identifier for every resource in Foundry. A property's RID will be referenced in error messages across the platform.
- **Status:** A signal to users and other Ontology builders about where in the development process the property stands. It can be `active`, `experimental`, or `deprecated`. By default, the `start date` property will have status `experimental`. Read more about [statuses](#).
- **API name:** The name used when referring to the property programmatically in code. For example, the API name of the `start date` property may be `startDate`. Read more about [API names](#).
- **Keys:** An indication of whether the property is the object type's title key or primary key.
 - The **title key** is the property that acts as a display name for objects of this type. For example, setting the `full name` property as the title key of the `Employee` object type will use the values of that property, such as the notional employees "Melissa Chang" and "Diego Rodriguez" as the display names for each respective `Employee` object.

API Reference ↗

Key is the property that acts as a unique identifier for each instance of an

object type, meaning that each row in the backing datasource is identified by this

for this property. For example, the value of the `employee` property

be used to identify "Melissa Chang" as a unique employee within the organization.

Send feedback

- **Base type:** Indicates the type of values for this property and determines the set of operations available in user applications. For example, the `start date` property will have base type `date`. User applications will allow you to configure a timeline widget with this property.
- **Value formatting:** Depending on the base type of the property, numeric formatting, date and time formatting, user ID and resource ID formatting are available to apply to the property, transforming its raw values into more readable versions in user applications. Read more about [value formatting](#).
- **Conditional formatting:** Rules set on a property that dictate how that property value will render (e.g coloring, alignment, etc.) in user facing applications. For example, you may set a rule on the `full name` property that colors its values green if the value of the `start date` property was less than 2 weeks ago, in order to indicate a new hire in user applications. Read more about [conditional formatting](#).
- **Type classes:** Additional metadata that are interpreted by user applications. Read more about [type classes](#).
- **Render hints:** Indications to user applications about how to render the property that may be different than most properties of the same base type. Many render hints can be used to impact the performance of reindexes of the object type the property is defined on. For example, if you don't expect any users to search or sort on the `start date` property in user applications, you can deselect the `searchable` and `sortable` render hints and improve the reindex performance of the `Employee` object type. Read more about [render hints](#).
- **Visibility:** An indication to user applications for how prominently to display the property. A `prominent` property will lead applications to show this property first to users. A `hidden` property will not appear in user applications. By default, the `start date` property will have visibility `normal`.

[Learn more about creating and configuring properties in the Ontology and about validation requirements for property metadata.](#)

Property base types with limited support

Some property base types have limited support. These types are indicated with the `Limited` status icon visible in the property base type picker.

API Reference ↗

- `byte`:
 - Properties of this type cannot be used within action types.

Send feedback

- `decimal` :
 - Properties of this type cannot be used within action types as the precision cannot be guaranteed when updating this data type due to the conversion between JSON and Java.
 - This type is also not supported in Object Storage v2.
- `float` :
 - Properties of this type cannot be used within action types.
- `short` :
 - Properties of this type cannot be used within action types.
- `vector` :
 - Vectors can only be queried by [KNN](#).
 - The max vector dimension is 2048.

For more information on the limitations of property base types in action types, see [the documentation on supported property types](#).

PREVIOUS
← Add conditional formatting

NEXT
Edit-only properties →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Edit-only properties

Edit-only properties allow you to define Ontology properties that are not directly mapped to a column in the backing dataset of the object type. This is particularly useful for situations where you may want to store additional information alongside your object types without modifying the underlying dataset.

Summary of edit-only properties

When working with edit-only properties, keep the following in mind:

- **No mapping to a column in the backing dataset:** Edit-only properties are not required to be mapped to a specific column in the backing dataset. This allows you to easily create new properties before the backing column exists, or create properties that will only be edited through the ontology.
- **Permissioned to one of the datasets backing the object type:** To ensure data consistency and security, edit-only properties must be permissioned to one of the datasets backing the object type.
- **Available only in Object Storage v2:** Edit-only properties are a feature that is exclusively available for object types leveraging Object Storage v2.

Creating edit-only properties

1. Navigate to the **Ontology Manager**.
 2. Choose the object type to which you want to add an edit-only property.
- API Reference ↗ **property** and fill in the required details, including the property name, type, description.



Send feedback

4. Under the **Data** section, toggle on the **Edit-only property** toggle and choose a dataset to permission to (if you have more than one dataset backing the object type).
5. **Save** your changes to create the edit-only property.

The screenshot shows a configuration panel for a 'Current owner' property. It has tabs for 'General', 'Display', 'Interaction', and 'Details'. The 'General' tab is active. Under 'General', there is an 'Allow multiple' toggle (off), a 'Value type' dropdown (set to 'Select an option...'), and a 'Status' label 'Experimental'. The 'CONFIGURATION' section has two rows: 'Title key' with a toggle (off) and 'Current: Name', and 'Primary key' with a toggle (off) and 'Current: Name'. The 'DATA' section has an 'Edit only property' toggle (on) and a 'Permissioned to' dropdown menu showing 'vehicles'.

Mapping edit-only properties to dataset columns

If you later decide to add a column to your backing dataset that corresponds to a property that is currently edit-only, you can easily map that property to the new column.

1. Navigate to the **Ontology Manager**.

2. [API Reference](#) ↗ object type with the edit-only property you want to map.

3. Select the edit-only property to open its details.

4. Under the **Data** section, untoggle the edit-only property and choose one of the backing datasets.

[Send feedback](#)

5. **Save** your changes.

PREVIOUS
← Metadata reference

NEXT
Required properties →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Required properties

Required properties are object type properties that must have a value. You can use this object type property to validate that there are no objects that have a null value for this property, or an empty array if it is an array property. This validation applies to data from the backing datasource and edits via actions.

Summary of required properties

When working with required properties, keep the following in mind:

- **Validation happens when data is being indexed into the object:** The check for null values happens as backing datasources are indexed into Object Storage. This means that the ontology modification itself will succeed if the column backing a required property contains null values.
- **Array properties cannot be empty:** Setting an array property to required ensures the presence of at least one item.
- **Changes via actions are validated at apply time:** If you attempt to write a null or empty value to a property via an action, the action will fail to execute.
- **Available only in Object Storage v2:** Required properties are only supported by object types leveraging Object Storage v2.

Create a required property

1. Navigate to **Ontology Manager**.

2. **API Reference** ↗ object type, and then the property you want to set as required.

3. **Create Property** and fill in the required details, including the description.

Send feedback

4. Under the **Configuration** section, toggle on the **Required** toggle.
5. **Save** your changes to the Ontology and wait for the reindex to be completed.

Note that if there is any null value currently set on the backing column for the property, the reindex will fail. To fix this, you must either update the backing datasource to no longer have nulls in the column, or unset the property as required.



Configuration

Title key ☐

Current: » Name

Primary key ☐

Current: » Id

Require values ?  ☒

Required properties that allow empty arrays

You can configure your required property to allow empty arrays. This means that the property will still reject null values, but will accept empty arrays. This is useful for mandatory control properties which can never be null, but can be configured to allow empty arrays. Learn more about [mandatory control properties](#).

It is important to note that Actions will write an empty array to any property that is mapped to a parameter, but the parameter is not set. This means that if you have a required property that allows empty arrays, and you leave the parameter blank in an Action, the Action will succeed and write an empty array to the property. If you do not want this behavior and want to enforce that users always set a value for this property via Actions, you should not allow e

[API Reference ↗](#)

required property.

[Send feedback](#)



Required properties for object types with multiple backing datasources

You may occasionally encounter situations where null values appear in required properties of object types backed by multiple datasources. This phenomenon occurs when a record for the object is present in some datasources, but absent in the datasource that supplies the required property.

The following example illustrates this behavior. Assume there is a `Movie` object type with two backing datasources and a `Genre` property that is required.

Datasource 1

Movie Id	Title	Box office	Regions
101	The Adventure Begins	200m	["USA", "Canada", "UK"]
102	Love in the City	75m	[]
103	Galactic Battles	500m	["USA", "UK", "Australia"]

Datasource 2

Movie Id	Budget	Genre (Required)	Director
101	50m	Adventure	Michael John Smith
API Reference ↗		Romance	Jane Doe
103	150m	Sci-Fi	

Send feedback

If a new `Movie` is added to both backing datasources without providing a value for `Genre` , it will fail to index to the Ontology.

However, suppose only Datasource 1 had a row for the new `Movie` object.

Datasource 1

Movie Id	Title	Box office	Regions
101	The Adventure Begins	200m	["USA", "Canada", "UK"]
102	Love in the City	75m	[]
103	Galactic Battles	500m	["USA", "UK", "Australia"]
104	Happy Ending	150m	["UK", "FRANCE"]

Datasource 2

Movie Id	Budget	Genre (Required)	Director
101	50m	Adventure	Michael John Smith
102	20m	Romance	Jane Doe
103	150m	Sci-Fi	

The example above will successfully get indexed into the Ontology, despite the fact that the resulting object would have no value for the required property.

Property	Value
Movie Id	104
Title	Happy Ending
Box Office	150m
Regions	["UK", "FRANCE"]
Budget	
Genre (Required)	
API Reference ↗	

The same applies to objects created via Action edits. `Movie` objects modified successfully, if they only contain properties tied to columns in `Datasource 1`.

Send feedback

However if the Action adds a property to the object that is sourced from `Datasource 2` , such as `Budget` , then the Action will be invalid and will fail to execute. This is because the object will now be present on `Datasource 2` and thus `Genre` must be set.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

Ontology building / Object and link types / Properties / Mandatory control properties

Mandatory control properties

Mandatory control properties are object type properties that allow for granular access control to the data stored in objects. You can use mandatory control properties to restrict access to all other properties in the same datasource for a given object, making those properties viewable only by users who satisfy the mandatory controls.

Note: Mandatory control properties are **only available on Object Storage v2**.

How to use mandatory control properties

1. First, create your marking-backed restricted view (RV). Learn more about [creating marking-backed restricted views](#).
2. Navigate to the Ontology Manager.
3. Choose the object type for which you want to restrict property access, then create or select the property you want to set as a mandatory control property.
4. On the property sidebar, ensure the property is mapped to the corresponding **marking column** on your restricted view.
5. Set the base type of the property type to **Mandatory Control**.
 1. By default a mandatory control property supports **markings** and/or **organizations** to restrict access.
 2. If you have CBAC enabled, you will have the option to choose **classification** based mandatory controls.
6. Select the **Allowed markings** and/or **Allowed organizations** on the datasource. For **classifications**, select the **Max classification**.

API Reference ↗

has multiple datasources, select a mandatory control property for each other datasources to secure their properties as well.
8. Save your changes to the ontology and wait for the reindex to be completed.

Send feedback

Types of mandatory control properties

There are three types of mandatory controls that can be set on a property:

- [Markings](#)
- [Organizations](#)
- [Classifications](#)

Markings

Markings are mandatory controls that restrict access by requiring a user to have a particular Marking in order to access data. If a resource has multiple markings, the user must have all of them to access the resource. Learn more about [markings](#).

To use markings, you are required to provide a set of allowed markings. Only markings in this set will be permitted on any mandatory control property on the datasource.

Organizations

Organizations are access requirements that enforce strict silos between groups of users and resources. Every user is a *member* of only one organization but can be a *guest member* of multiple organizations. To access data marked with an organization, a user must be a member of that organization. If a resource has multiple organizations, the user must be a member of at least one of the organizations applied to the resource. Learn more about [organizations](#).

To use organizations, you are required to provide a set of allowed organizations. Only organizations in this set will be permitted on any mandatory control property on the datasource.

Markings and organizations can be used together on the same mandatory control property. In this case, a user must satisfy all the markings and at least one of the organizations to

^a [API Reference ↗](#)

Classifications

[Send feedback](#)

Classification markings are mandatory controls used to protect sensitive government information. They are used to restrict access to sensitive information where sensitivity of information is defined in a hierarchical way. Every user can only access data that is classified at or below their own classification level.

You can only configure CBAC markings if you have CBAC enabled on your enrollment. Learn more about [CBAC \(classification based access controls\)](#).

To use classifications, you need to provide a max classification. Only markings that satisfy this max classification will be permitted on any classification based mandatory control property on the datasource.

Classifications can not be used together with markings or organizations on the same mandatory control property.

Datasource-level permissioning

A mandatory control property secures all other properties in the same datasource. For object types with a single datasource, this means that a user will only be able to view an object if they satisfy the value in the mandatory control property.

However, for multi-datasource-backed object types (MDOs), each datasource could have its own mandatory control property. Only the properties backed by a specific datasource will be secured by the mandatory control in that datasource.

This means that it is possible for a user to only have permission to see a subset of properties on an object. In this case, the user will only be able to see the properties mapped from those datasources. Other properties will appear as null when displaying an object instance to the user.

To use mandatory control properties effectively, the backing datasources should be structured in such a way that only properties that should share a mandatory control are in the same datasource.

API Reference ↗

Validations

Send feedback

The following validations are enforced on mandatory control properties:

- Mandatory control properties must be mapped to a **marking column** on a **restricted view**. The mandatory controls are enforced by backing the object type with a restricted view which has a policy that requires users to satisfy the markings in the mapped column to be able to view a row. See [Restricted Views](#) for more information.
- **Mandatory control properties must be required.** This ensures that if an object with a mandatory control property is present on a datasource, the mandatory control must be defined to help maintain data consistency and integrity. All mandatory control properties must not be null. However, markings and organization values can be set to an empty array. In such cases, all users will meet the marking requirements and be able to view the row. Learn more about [required properties](#).
 - If you want to add a mandatory control property to an edit-only object type that already has edits, you cannot create the property directly because mandatory control properties cannot be empty. To work around this:
 1. Add a nullable string array property.
 2. Backfill its values using an [Action](#).
 3. Change the property's base type to **Mandatory Control**.
- Every datasource that contains a mandatory control property must define a constraint on what values can be added to those properties. These constraints come in the form of a max classification for classification based mandatory controls, or a set of allowed markings and/or allowed organizations. Any edits made to the mandatory control properties, as well as the values gotten from the backing dataset, must adhere to the constraint set on the datasource.
 - This constraint is enforced on the object storage level, so even though you may be able to use Ontology Manager to save an object type that violates this constraint, the object type will fail to index if existing values in the dataset do not satisfy the constraints, or if the values in the dataset are updated to include invalid values for the mandatory controls. Also, any edits made that try to set an invalid value to the mandatory control property will be rejected and the Action will fail to submit.
 - These allowed markings, allowed organizations or max classification will be used to mark any exported dataset that is materialized from this Object type. This ensures that only users who can view all rows on the Object type will be able to view the materialized dataset.

Note that mandatory control properties are set to **Hidden** by default. This is because mandatory control properties are meant to be used as markings for other fields, so there is no need for mandatory control properties to appear in object views or tables.

However, mandatory control property visibility can still be enabled if

API Reference ↗

Send feedback

Mandatory controls in actions

You can add a mandatory control parameter to your action type. This can be a marking parameter, or a classification parameter if CBAC is enabled. Organization parameters are currently not supported.

Mandatory control parameters are commonly used to set a mandatory control property on an object that the action creates. In this case, the values provided must adhere to the property's allowed values, if an invalid value is provided, action submission will fail.

You can also add a max classification at the parameter level, for classification based mandatory control parameters. This is an action type validation, and so will prevent the action from being submitted if the provided value does not satisfy the max classification, as opposed to relying on the datasource validation which will allow the action to be submitted but will fail to complete.

Objects created by actions will be secured by the provided value for the mandatory control property, just like objects derived from a backing datasource.

Marketplace usage

Object types with mandatory control properties and action types with mandatory control parameters can be packaged and installed through Marketplace.

When packaging an object type with mandatory control properties, the allowed markings or max classification are declared as installation inputs for that product.

Similarly, if packaging an action type with a classification based mandatory control parameter with max classification set, the max classification is declared as installation inputs.

When installing the product, you will be prompted to select the allowed markings or max classification for each mandatory control property. The selected values will be set as a max classification of the mandatory control properties upon install.

[API Reference ↗](#)

[Send feedback](#)

Markings

For more detailed information on the effect of a marking, click 'More details'

Allowed Markings

→

test-marking

✓

Allowed markings or allowed organizations for data under a mandatory control property type.

Affecting Mandatory Marking

More details

Max Classification

→

MTS//MNF

Edit

✓

The maximum classification for data under mandatory control property types and/or the maximum classification allowed on an action type classification parameter.

Affecting CBAC Marking

More details

Max Classification

→

MS//REL TO USA, FVEY

Edit

✓

The maximum classification for data under mandatory control property types and/or the maximum classification allowed on an action type classification parameter.

Affecting CBAC Marking CBAC Marking

More details

Note that packaging multiple mandatory control properties and/or parameters with the same values would results in only one mandatory control input being declared.

Base types

Property base types define the kind of data that can be stored in a property. For a complete reference of all supported property base types, see the [properties overview](#).

Base types are used to define properties on objects. The base type of a property determines the set of operations available for that property in user applications. All field types are valid base types except for `Map` and `Binary` types.

Base types also include the following advanced types:

- **Vector:** A type for storing [vectors](#) on objects for use in a semantic search.
- **Geopoint:** A type for defining properties that represent geographic [points](#).
- **Geoshape:** A type for defining properties that represent geographic [shapes](#).
- **Attachment:** A type for storing files on objects for use with [functions on objects](#).
- **Time series:** A type for defining a property as a [time series](#).
- **Geotemporal series:** A type for defining a property as a reference to a [geotemporal series](#).
- **Media reference:** A type for defining a [reference to a media file](#).
- **Cipher text:** A type for storing a string value encoded with [Cipher](#).
- **Struct:** A type for defining schema-based properties with [multiple fields](#).

All base types may be used in arrays to represent multiple values for a property, excluding the `Vector` and `Time series` types.

Complex property base types

API Reference ↗

Some property base types require additional configuration or have special considerations below for information on the following property base types.

Send feedback

- **Media references**: Reference media items stored in media sets.
- **Struct types**: Complex structured data with multiple fields.

Media references

A **media reference** property type allows you to have media on your objects, such as images, videos, audio files, and documents. A [media reference](#) points to a specific media item within a [media set](#). The media reference contains information about the media file, which means Foundry can display the media wherever the media reference is used.

Media reference format

Below is an example media reference:

```
1  {
2    "mimeType": "image/png",
3    "reference": {
4      "type": "mediaSetViewItem",
5      "mediaSetViewItem": {
6        "mediaSetRid": "ri.mio.main.media-set.00000000-0000-0000-0000-000000",
7        "mediaSetViewRid": "ri.mio.main.view.00000000-0000-0000-0000-000000",
8        "mediaItemRid": "ri.mio.main.media-item.00000000-0000-0000-0000-000000"
9      }
10   }
11 }
```

The media reference includes the following:

- **mimeType**: The file's media type.
- **reference**: A reference containing the media set RID, view RID, and specific media item RID.

Configure media reference properties

[API Reference ↗](#) Media reference properties are backed by a dataset. The backing dataset must include a media reference column, which will map to the media

Send feedback

This column type is specifically designed to store media reference values and ensures proper integration between your ontology objects and media sets.

Media Reference

General **Source** Display Interaction Dependents

Source type

☒ Datasource
Back this property with a dataset, restricted view or stream

☐ User edits
Back this property exclusively with edits from user inputs

☐ Linked objects
Use a property from another object type

Data

Backing column

mediaReference

Additionally, a media reference property must have a **media source**, which can be configured in the **Capabilities** tab of the object type. This media source should be the media set that the media references point to.

[API Reference ↗](#)

[Send feedback](#)

Media Reference Properties

Set up a media reference property.

Media Reference

+ Add media source



images

master

+ Add property

Structs

A **struct** is an ontology property base type that allows users to create schema-based properties with multiple fields. Struct properties are created from struct type dataset columns. To learn more about structs, refer to the complete [struct](#) documentation.

PREVIOUS

← Mandatory control properties

Property reducers

→ NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

API Reference ↗

Send feedback

Property reducers [Beta]

Beta

Property reducers are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development.

A **property reducer** enables you to transform an [array property](#) into a single value in the array for display and interface implementation purposes. Reduction does *not* change the underlying property type or property data stored; instead, it provides access to the reduced value in the array when reading the property value.

For example, applying a reducer to an array with multiple inspection dates allows you to display only the most recent date when viewing the property in a table or application but ensures the full array remains accessible for queries and other operations. Applications that support reducers, such as [Workshop](#), also enable you to view the complete array on hover or in expanded views.

Reducers work with array properties containing numeric, temporal, string, and boolean base types. You can reduce [struct](#) arrays based on any struct field that uses a supported base type. Review the [tables below](#) to learn more about a property reducer's supported base types.

When to use property reducers

Use property reducers when:



Historical or temporal data is stored in arrays, such as inspection updates, or measurement readings.

Send feedback

- You want to display the latest/earliest, highest/lowest, or first/last array value in a table or application while preserving the property's full history.
- Array properties require non-array types to satisfactorily implement [interface properties](#).

Supported and unsupported base types

Reducers work with array properties based on the underlying base type. The following tables categorize which array subtypes support reducers and which do not.

Supported base types

Property reducers are available for arrays containing the following subtypes:

Category	Base types	Reducer options
Numeric	Byte , Short , Integer , Long , Float , Double , Decimal	Highest, lowest
Temporal	Date , Timestamp	Most recent (latest), Least recent (earliest)
String	String	First, last (lexicographically)
Boolean	Boolean	True first, false first
Struct	By any supported struct field outlined below	Depends on the base type of the struct field

Unsupported base types

Property reducers are *not* available for arrays containing the following subtypes:

- Attachment
- Cipher Text
- Geohash
- API Reference ↗
- Geotemporal Series Reference
- Marking

Send feedback

- [Media Reference](#)
- [Time Dependent](#)
- [Vector](#)

Support for struct arrays

Reducers function on struct arrays based on a specific **field** within the struct, not the struct itself. You can only reduce by struct fields that use one of the [supported base types](#) listed above. You can also configure multiple reducers using different struct fields to handle tie-breaking scenarios.

Configure a property reducer

1. Navigate to **Ontology Manager**.
2. Search for and select your object type by choosing **Object types** under **Resources** in the left panel.
3. Select the array property to configure from your object type's **Properties** tab.
4. Choose the **Interaction** tab in the property editor panel that opens on the right.
5. Scroll to the **Reduce array** section.

[API Reference ↗](#)

[Send feedback](#)

General Source Display Interaction Dependents

...

Editing

Inline edit ?



Cannot use inline action on property of this type

Render hints

☐

Keywords

☒

Searchable

Requires reindex

Limited support

☐

Selectable

Requires reindex

☒

Sortable

Requires reindex

☐

Low cardinality

Requires reindex

Reduce array ?



Add array reducer

Type classes ?

API Reference ↗



Add new type class

Send feedback

6. Select **Add array reducer**.

7. Select your desired [reducer option](#) based on the property's base type.



8. **Save** your changes.

Configure a property reducer for struct arrays

Struct arrays offer the most flexibility for reducers. You can reduce based on any field in the struct that uses a [supported base type](#) and configure multiple reducers for tie-breaking scenarios.

Example: Customer review history

Consider a `Product` object type with a `customerReviews` struct array property that contains the following fields:

- `rating ( Integer )`: A one to five star rating.
- `API Reference ↗ e )`: The date the customer posted the review.
- `reviewerName ( String )`: The reviewer's name.
- `verifiedPurchase ( Boolean )`: Indicates whether or not the purchase was verified.

Send feedback

Sample data for a `Product` object's `customerReviews` property resembles:

rating	reviewDate	reviewerName	verifiedPurchase
5	2024-11-20	Alice Chen	true
3	2024-11-15	Bob Smith	false
4	2024-11-22	Carol Lee	true

Configure a single reducer



Tip

Review the [property reducer configuration instructions](#) and the [supported base types](#) table before proceeding.

You can configure a single reducer on a struct field, which Foundry uses to reduce the array of structs to a single struct, sorting the values to display by the field with the configured reducer.

- **Single reducer:** Render the latest `reviewDate`.
- **Result:** Displays the struct containing Carol Lee's four star review from `2024-11-22`.

Users can still query for and access the full review history.

Configure multiple reducers

You can configure multiple reducers to handle tie-breaking scenarios:

- **Primary reducer:** Sort the structs and display the struct with the latest `reviewDate`.
- **Fallback reducer:** Render the highest `rating`.
- **Use case:** If two reviews are posted on the same day, render the higher-rated review.

[API Reference ↗](#) only applies when the primary reducer results in multiple items. The primary reducer is always evaluated first, and only items that tie on the primary field are further reduced by the fallback reducer. This reduction behavior is repeated for any additional configured reducers.

Send feedback

Reduce array ?

Array reduction logic

Reduce to

⋮

Most recent

🕒

Date collected

▼

×

↓ If multiple values match

Further reduce to

⋮

Largest

1.0

Confidence level

▼

×

↓ If multiple values match

Further reduce to

⋮

Select value...

▼

×

↓ If multiple values match

+

Add array reducer

How reducers appear in applications

Applications that support reducers display the reduced value in compact views like tables or lists, while providing access to the full array through an expanded view or on-hover mechanism. This approach allows users to quickly scan data without seeing verbose array representations, while still being able to inspect the complete array when needed.

For example, an application might display only `2024-11-22` (the most recent date) in a table cell, but reveal the full array of dates when the user hovers over or expands that cell.

API Reference ↗

Use property reducers with interfaces

Send feedback

Use property reducers to ensure an array property can implement non-array interface properties. This allows object types with array data to be mapped to [interface properties](#) that expect single values.

Example: Render the most recent equipment maintenance date through an interface implementation

Interface: `Asset`

- Property: `lastMaintenanceDate` (`Date`)

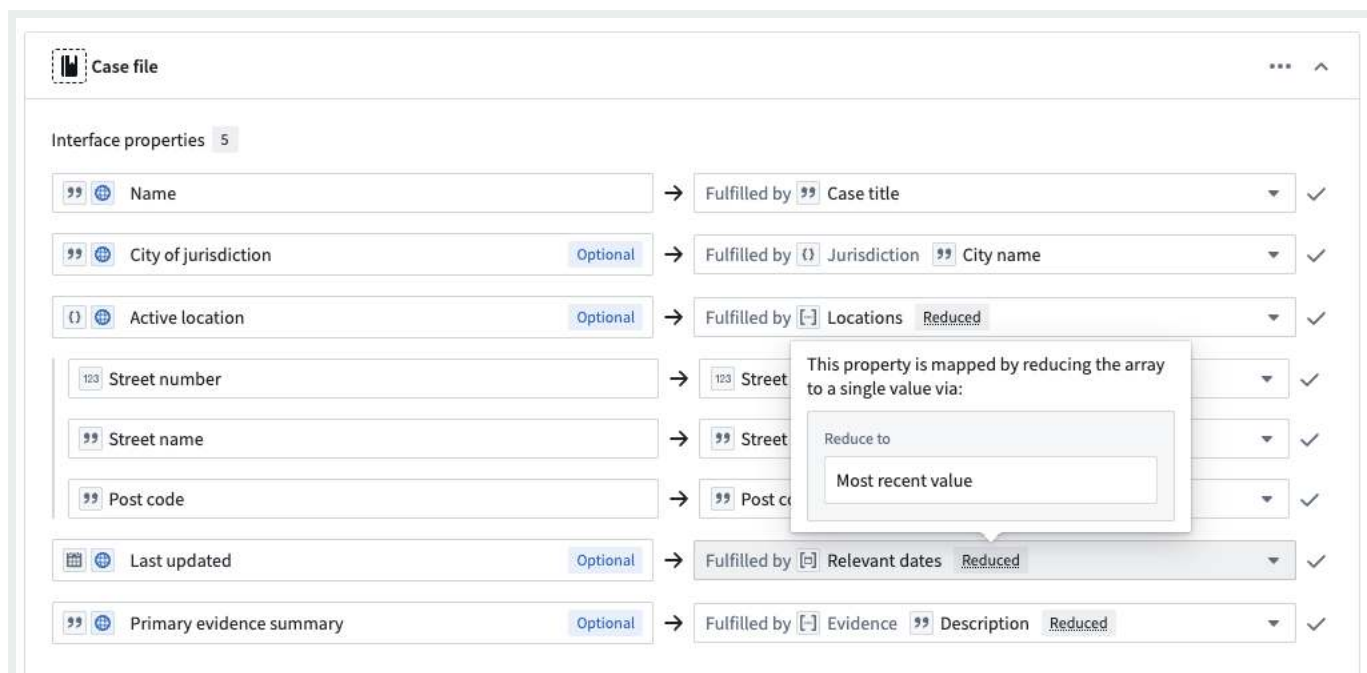
Object type: `Equipment`

- Property: `maintenanceHistory` (`Date` array)
 - Values: `[2024-01-15, 2024-03-22, 2024-11-01]`
- **Reducer:** Latest `maintenanceHistory` date.
- **Implementation:** `Date` array property implements a single `Date` through the configured reducer. A user will see `2024-11-01` when viewing the property via the interface.

The `Equipment` object type can implement the `Asset` interface because the reducer allows its `maintenanceHistory` array property to be represented as a single `Date` value. When users view an `Equipment` object through the `Asset` interface, they see only the most recent maintenance date.

[API Reference ↗](#)

[Send feedback](#)



⚠ When using a reduced array value on an object type to satisfy an interface property that is a parameter of an [interface action](#), the action will return an error when called on objects of that type. However, you can still use the interface action for objects that do not implement the interface through a reduced array value.

Combine property reducers with struct main fields

When you combine property reducers with [struct main fields](#), you enable property display and expand the number and shape of interfaces they can implement.

Consider the `Event` object type below, which contains a `Locations` property that is an array of structs:

Object type: `Event`

- Property: `locations` (`Struct` array)

API Reference ↗ `streetName` (`String`), `dateCollected` (`Date`), `numberOfGuests` (`Integer`)

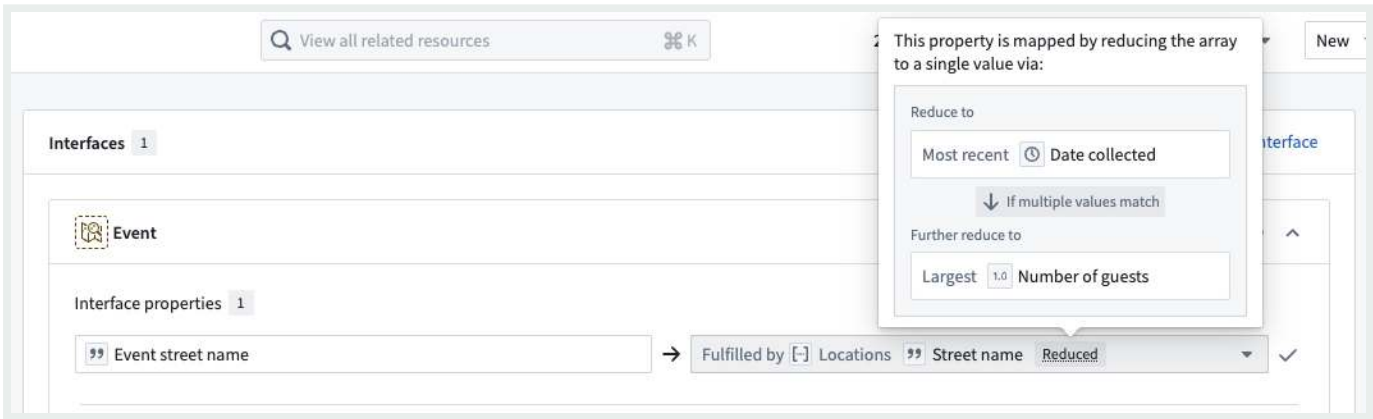
- Main field configured:** `streetName`
- Reducer configured:** Most recent by `dateCollected`.

Send feedback

With the `streetName` field configured as the main field and most recent `dateCollected` as the property reducer, this enables multiple interface implementation options:

Configuration	Can implement
Neither feature configured	Struct Array only
Main field only	Struct Array , String Array
Reducer only	Struct Array , Struct
Both configured	Struct Array , String Array , Struct , String

As an example, with both a struct main field and property reducer configured, you can use the `locations` property to fulfill a *single* string property from an interface, such as `Event street name` from the notional `Event` interface in the image below.



When you configure both a struct main field and a property reducer, the transformation:

- Applies the configured property reducer, fetching the most recent location based on its date.
- Extracts the configured main field and returns its value as a string.

This means a single property can implement interfaces requiring any of these types: `Struct Array` , `String Array` , `Struct` , or `String` .

API Reference ↗ and considerations

Send feedback

- **No direct querying of reduced value:** You cannot filter or query based on the reduced value. Queries operate on the full array, not the reduced representation.
- **Interface actions are not supported for reduced or struct main field implementations:** [Interface actions](#) that edit a property implemented through a property reducer or [struct main field](#) will return an error when called on objects of that type. This is because reduced and struct main field values cannot be translated back to the underlying object property. For example, a reducer that selects one element from an array has no inverse operation to reconstruct the full array; struct main fields that extract a subset of fields have no way to populate the remaining ones. Interface actions that do not target these properties, or that target objects implementing the interface without reducers or struct main fields, work as expected.

FAQs

Can I change or remove a reducer after configuration?

Yes, you can modify or remove reducers at any time in Ontology Manager. Changes do not require a reindex and take effect immediately.

What happens if multiple items tie for the reducer criteria?

For struct arrays, you can configure fallback reducers to handle ties. The fallback reducer is only evaluated for items that tie on the primary reducer. For primitive arrays or structs without fallback reducers, one of the tied values is returned in a deterministic but unordered manner.

Can I use reducers with edit-only properties?

Yes, you can configure reducers for [edit-only properties](#). The reducer configuration is independent of whether the property has a backing column.

API Reference ↗

Do reducers function across all applications?

Send feedback

Applications are progressively rolling out support for reduced properties. If you configure a reducer on a property and use an application that doesn't yet support reducers, it should not break any existing functionality, the property will simply continue to be displayed as an array. Only applications that support reducers will have the ability to display the reduced value.

PREVIOUS

← [Base types](#)

NEXT

[Derived properties](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Configure derived properties

Beta

Derived properties are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development.

Overview

Derived properties are properties that are calculated at runtime based on values from linked objects. Instead of storing data directly, a derived property pulls information from objects connected through link types, optionally applying aggregations like averaging, counting, or collecting values into lists.

For example:

- A `Department` object type could have a derived property for "Average employee salary" that aggregates salary values from all linked `Employee` objects.
- A `Project` object type could have a derived property for "Lead engineer name" that retrieves the name from a single linked `Engineer` object.
- An `Order` object type could have a derived property for "Product names" that collects all product names from linked `Product` objects into a list.

Derived properties are **read-only** and cannot be edited by functions or actions. These properties use the security context of all objects involved in the calculation, ensuring users [API Reference ↗](#) for which they have access authorization.



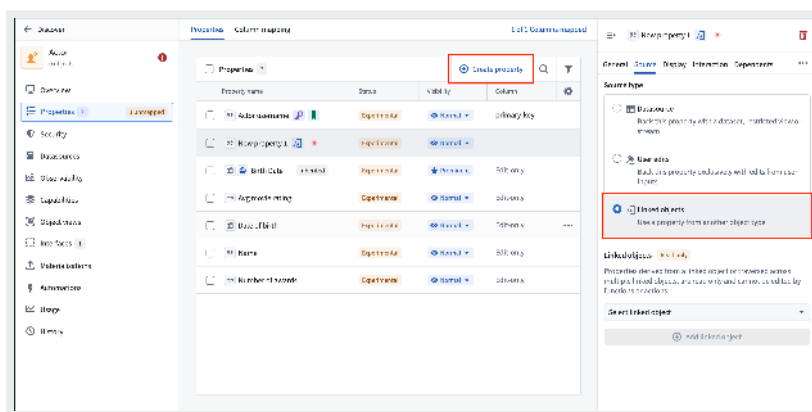
Send feedback

Configure a derived property

To configure a derived property on an object type, follow the steps below.

1. Open the property configuration panel

From the **Properties** tab of your object type, select **New property** or click on an existing property to edit it. This opens the property configuration side panel.



2. Navigate to the Source tab

In the property configuration side panel, select the **Source** tab to configure where the property gets its values.

3. Select the Linked objects source type

In the **Source type** section, choose the **Linked objects** option. This enables derived property configuration.

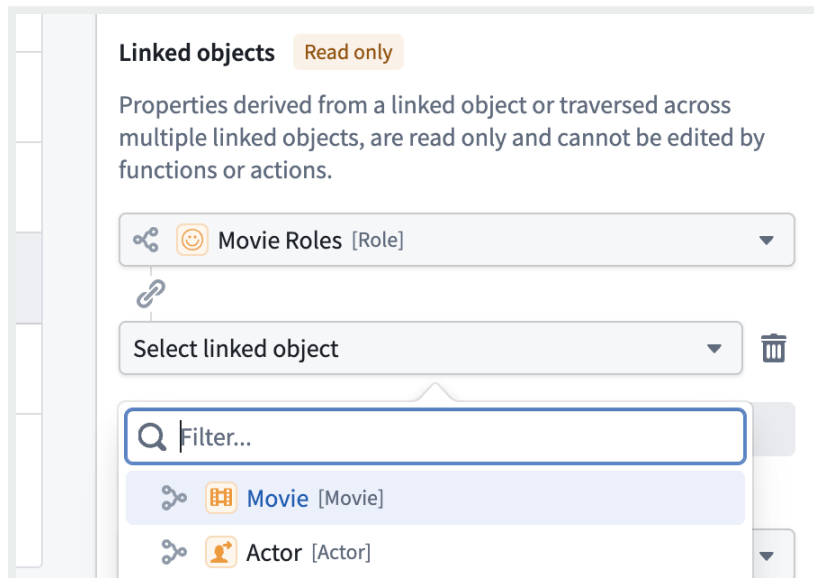
4. Select a link type

API Reference ↗

In the **Linked objects** section, select a link type from the dropdown. objects the property will derive values from.

Send feedback

- The dropdown menu shows all available link types from your current object type.
- After selecting a link type, you can optionally add additional link types to traverse multiple levels of connections (up to 3 levels).
- Use **Add linked object** to traverse through another level of linked objects.



5. Configure aggregation (if needed)

If any link in your chain has a "many" cardinality (one object linking to multiple objects), you must select an **Aggregation** to combine the values:

Available aggregations:

- **Count:** Count the number of linked objects.
 - **Average:** Calculate the average of numeric values.
 - **Sum:** Calculate the sum of numeric values.
 - **Minimum:** Select the minimum value.
 - **Maximum:** Select the maximum value.
 - **Approximate cardinality:** Estimate the number of unique values.
 - **Exact cardinality:** Count the exact number of unique values.
- API Reference ↗
- all values into an ordered list (preserves duplicates).
- all unique values into an unordered set.

Send feedback

6. Select the property to derive

After selecting a link type (and aggregation if needed), choose which property from the linked object type you want to derive:

- The dropdown menu shows all available properties from the final object type in your link chain.
- For **Count** aggregation, you do not need to select a property as objects are automatically counted.

Linked objects Read only

Properties derived from a linked object or traversed across multiple linked objects, are read only and cannot be edited by functions or actions.

  Movie Roles [Role]



  Movie [Movie]



 Add linked object

Aggregation

Collect set

Property

 Title  

Limit

10  

7. Configure collection limit (for collect aggregations)

If you select **Object list** or **Collect set** as your aggregation, you can optionally set a limit on the number of items collected. The default limit is 10 items.

[API Reference ↗](#)

Send feedback

Multi-hop derived properties

Derived properties support traversing up to **3 levels** of linked objects. This allows you to derive properties from objects that are indirectly connected to your starting object type.

For example:

- A **Department** object type could derive "Project names" by traversing through **Employee** objects to their linked **Project** objects.
- The link chain would be: Department → Employee → Project

Linked objects Read only

Properties derived from a linked object or traversed across multiple linked objects, are read only and cannot be edited by functions or actions.

  Employee [Employee] ▼



  Document [Project] ▼ 

 Add linked object

Aggregation

Collect list ▼

To configure multi-hop:

1. Select your first link type.
2. Select **Add linked object** to add another level.
3. Select the next link type from the newly-connected object type.
4. Repeat up to 3 levels total.

[API Reference ↗](#)

Known limitations

[Send feedback](#)

- **OSv1 support:** Queries with derived properties may not contain any object types indexed using [OSv1](#).
- **Text search:** Derived properties cannot be used in text search or keyword filters.
- **Structs in OSDK:** In current versions of the TypeScript OSDK, queries with derived properties may not contain any [struct](#) property types. You can use a `$select` operation to exclude struct properties.
- **Inline actions:** Properties with inline actions configured cannot be converted to derived properties.
- **Value types:** Properties with value types cannot be converted to derived properties.
- **Required properties:** Derived properties cannot be marked as required (non-nullable).
- **Property type constraints:** Derived properties cannot have property type constraints.
- **Display formatting:** Derived properties cannot have rule set bindings or base formatters.
- **Primary keys:** Primary key properties cannot be derived properties.
- **Ontology condition:** Derived properties are not supported for the Default ontology.

PREVIOUS
← [Property reducers](#)

NEXT
[Structs / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Structs

A **struct** is an Ontology property [base type](#) that allows users to create schema-based properties with multiple fields. Struct properties are created from struct type dataset columns. Struct property fields can have different data sources as long as the property is transformed into a single struct type column before being defined in the Ontology.

Many common object properties can be modeled as structs. For example, a `Full Name` property with the fields `First Name` and `Last Name`, or an `Address` property that includes `Street`, `City`, `Postal Code`, and `Country` fields.

Struct configuration

The following is a list of struct property constraints and allowed configurations:

- Structs have a depth of one and cannot be nested.
- Structs must have at least 1 field.
- Only the following field types are currently supported:
 - `BOOLEAN`
 - `BYTE`
 - `DATE`
 - `DECIMAL`
 - `DOUBLE`
 - `FLOAT`
 - `GEOPOINT`

API Reference ↗

Send feedback

- `STRING`

- **Timestamp**

Query semantics

Structs are indexed similarly to [ElasticSearch object field types](#), which means that arrays can have unintuitive behavior. For example, if you have an array of **Full Name** properties, an object containing [{"firstName": "Harvey", "lastName": "Dent"}, {"firstName": "Two", "lastName": "Face"}] would match a query for "firstName": "Harvey" AND "lastName": "Face". The two conditions are treated independently, rather than requiring to match on the same struct within the array.

Current levels of support

As support for struct property types expands, availability will vary across the Palantir platform.

Structs are currently supported in the following applications and services:

- **[Ontology Manager](#)**: Define and edit structs.
- **[Actions](#)**: Use actions to [create and modify struct property values](#).
- **[Pipeline Builder](#)**: Define and edit structs.
- **[Workshop](#)**: Display and use struct properties as variables.
- **[Marketplace](#)**: Package and install struct properties.
- **[Object Explorer](#)**: Search for objects by their struct property values (struct field search is under development).
- **[Ontology SDK](#)**: Load struct properties and search for objects by their struct property values in Ontology SDK applications. Not all Ontology SDKs support struct properties. Refer to the OSDK [unsupported property types](#) for more information.
- **[Functions](#)**: Struct parameters and struct property edits are supported in TypeScript v2 and Python functions.

Structs will not be supported in Object Storage v1 (Phonograph) and can currently only be c

[API Reference](#)

s and Restricted Views.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

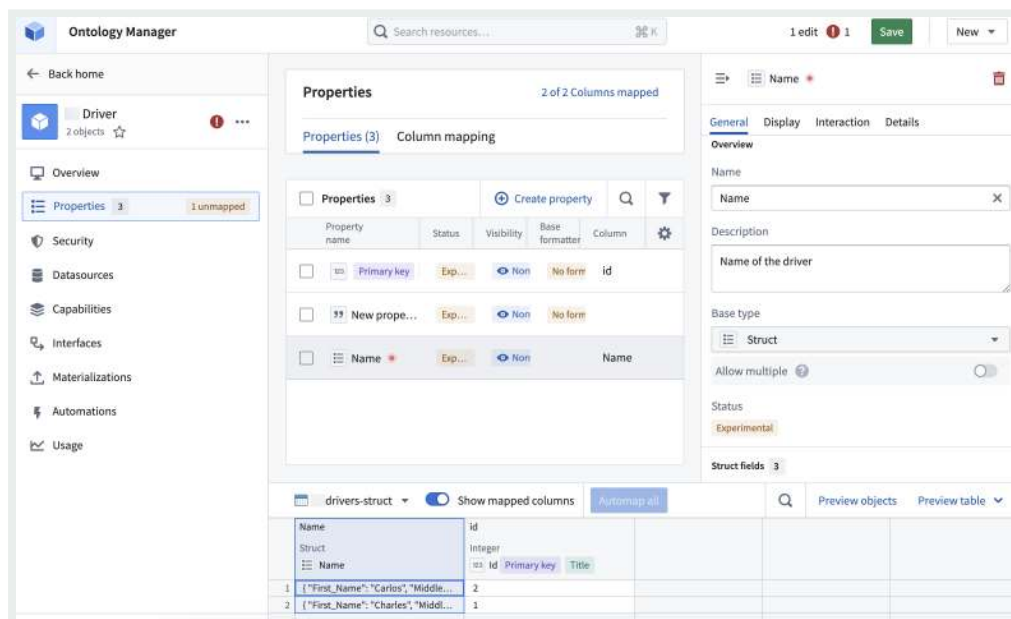
[Send feedback](#)

Ontology building / Object and link types / Structs / Create a struct type

Create a struct property type

Create and configure a new struct property from the **Object types** page in Ontology Manager. For more information about struct properties, see the [overview](#).

1. In Ontology Manager, open the **Object types** tab in the left sidebar and select an existing object type.
2. In the object type details page, open the **Properties** tab in the left sidebar, and select the **Create property** button on the top right of the **Properties** table.



3. In the **Property editor** panel, add a name and description, and select **Struct** from the **Base type** dropdown menu.

API Reference ↗

Send feedback

 Name 

[General](#) [Display](#) [Interaction](#) [Details](#)

Description

Name of the driver

Base type

 Struct ▼

Allow multiple 

Status

4. Scroll down to the **Data** section and select a **Backing column** from the dropdown.

[API Reference ↗](#)

[Send feedback](#)

Current: Id

Primary key

☐

Current: Id

Data

Edit-only property ?

☐

Backing column

Name

▼

Shared property

Assign (optional) ?

Select a shared property

▼

5. In the **Struct fields** section, select **Add field**, then **New field**.

[API Reference ↗](#)

[Send feedback](#)

Struct fields

3

”

First Name

”

Middle Name

”

Last Name

+

Add field

- 6. Name the new struct field and optionally add a description.
- 7. Lastly, map a column from a datasource to the new struct field.

Edit a struct property type

1. In Ontology Manager, open the **Object types** tab in the left sidebar and select an existing object type.
2. In the object type details page, open the **Properties** tab in the left sidebar, and select the relevant struct property type from the **Properties** table.
3. In the **Property editor** panel, scroll to the **Struct fields** section and select the struct field you would like to edit. The number of edits made will appear on the top right of the Ontology Manager interface.

1 edit  1

Save

New ▾

  Name 



General

Display

Interaction

Details

Allow multiple 

☐

Status

Experimental

Struct fields 3

  First Name

  Middle Name

  Last Name

 Add field

Configuration

Title key

☐

4. Make the necessary edits in the **Edit struct field** dialog, and select **Confirm**.

API Reference ↗

Send feedback

Properties2 of 2 columns mapped

EditName struct field

” Middle Name

Name

Middle Name

Description

Type

” STRING

Backing column field

Middle_Name

API name

middleName

Close

Confirm

i Note that changing a struct field's API name will result in a new struct field RID being generated. This will override the existing index, similar to the behavior of changing the property ID of a property type. Any applications that reference the updated struct field will need to be updated as well.

PREVIOUS
← Create a struct type

NEXT
Automapping struct types →

API Reference ↗ Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

Send feedback

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Structs / Automapping struct types

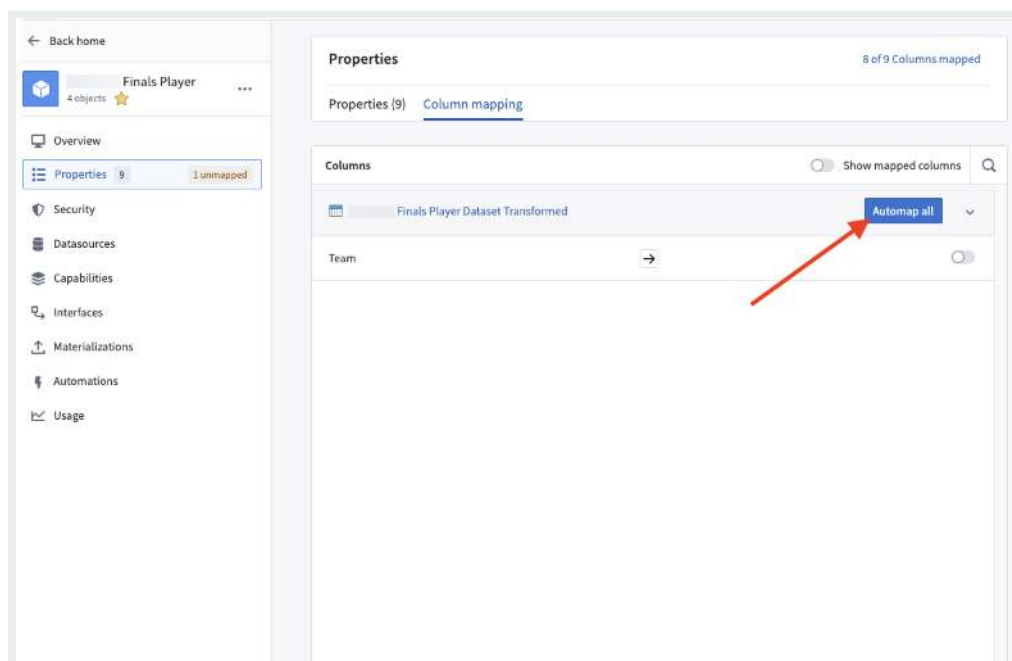
Automapping struct properties

Automapping allows users to map all columns automatically rather than manually.

Automap struct types in Ontology Manager

If the object has already been created, users can automap all columns by using the **Automap all** feature.

1. In Ontology Manager, enter the **Properties** tab and select the desired property.
2. Under the **Column mapping** tab, select the desired column.



[API Reference ↗](#)



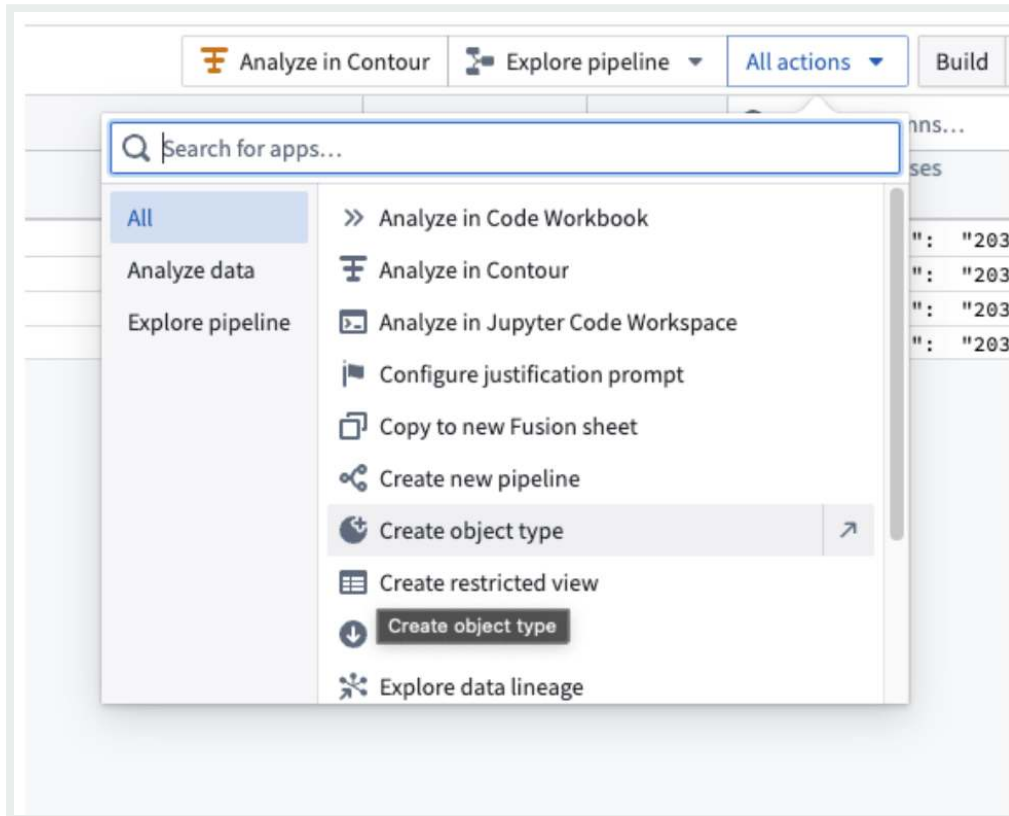
Automap all.

[Send feedback](#)

Automap struct types in Pipeline Builder

If the object has not yet been created, automapping can be done on initial object creation with the object type creation wizard.

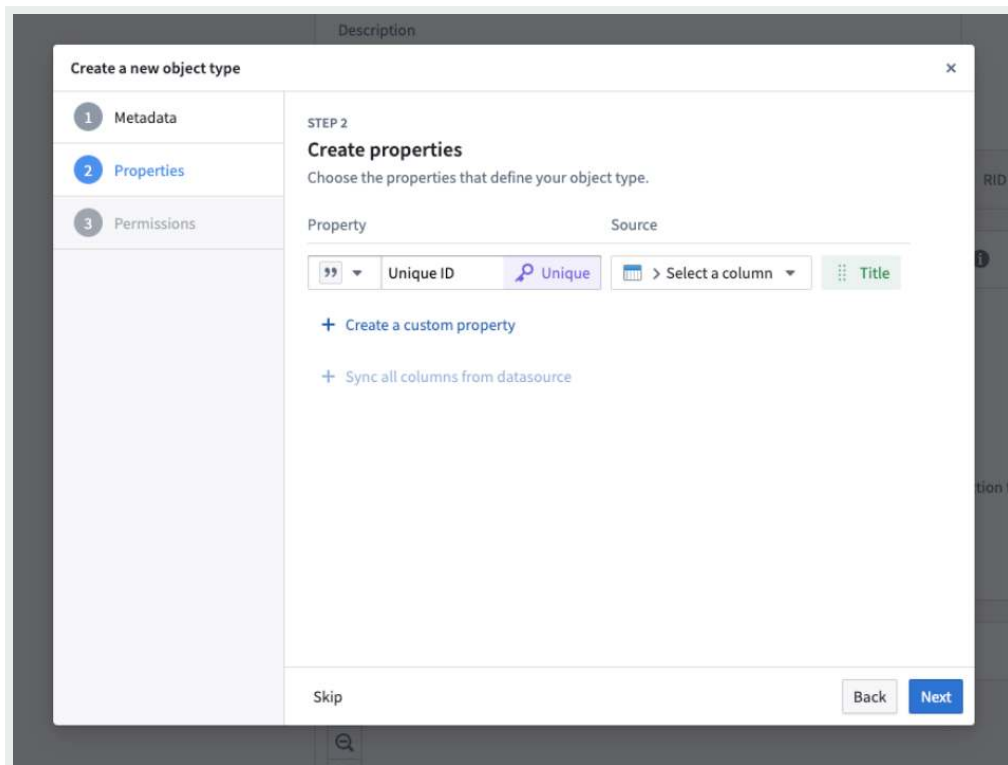
1. In your Pipeline Builder pipeline, open the relevant dataset and select the **All Actions** dropdown in the top right.



2. Select **Create object type** to create a new object.

[API Reference ↗](#)

[Send feedback](#)



3. Under **Properties**, add the desired properties to be mapped.

4. Select **Next** and complete the remaining steps to create an automapped object type.

PREVIOUS
← [Edit struct types](#)

NEXT
[Structs and shared properties](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Struct properties and shared property types

Struct properties can be used by local and shared property types. When converting, or promoting a local property type to a shared property type, struct fields need to be re-mapped. Local struct property types backed by shared property types will inherit shared property type fields except for the struct field resource identifiers (RIDs). Struct field metadata (display name, description, aliases) will then be inherited from the shared property type, but struct fields will keep their original RIDs.

Create a struct type shared property

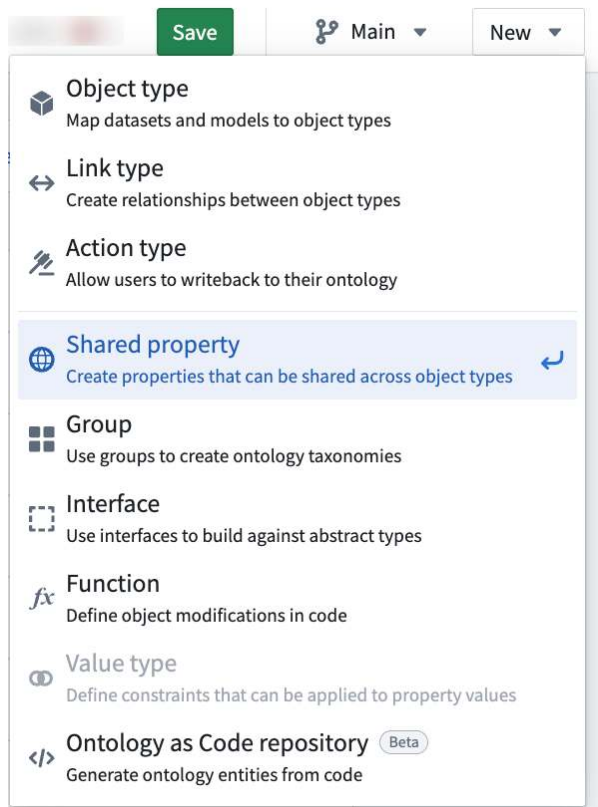
1. In Ontology Manager, you can create a struct type shared property in two ways:

API Reference ↗

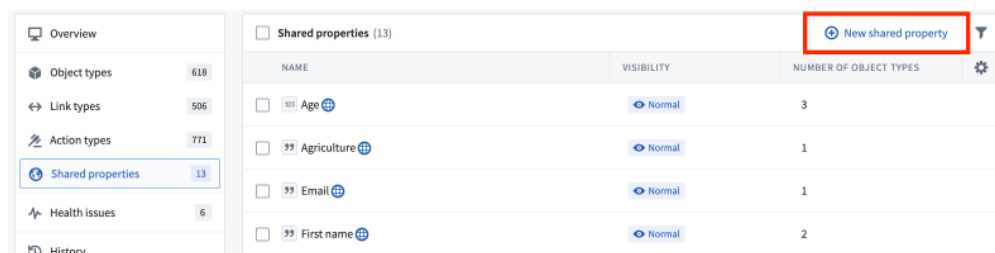


Send feedback

- From the top right of the homepage, select **New > Shared property**



- Select **Shared properties > + New shared property**



2. In the main panel, select the **New shared property** button. This will open a helper where you can configure metadata of the shared property including the name, description, aliases, and API name. Then select **Next** to proceed.

Create shared property ×

1 Metadata

2 Configuration

3 Save location

STEP 1

Metadata

Give your shared property meaningful metadata that will help others make best use of it

Name

Description

Aliases ?

API Name ?

Skip

Next

3. Configure the base type, value type, visibility, and whether to require values for the property.

Create shared property ×

1 Metadata

2 Configuration

3 Save location

STEP 2

Configuration

Base type

Allow multiple ?

☐

Value type ? (optional)

Visibility

Require values

If enabled, this property cannot receive null values from its backing column or user edits. Actions with parameters containing null values will also fail to submit.

☐

Skip

Back

Next

4. Select a project to save this shared property to.

API Reference ↗

Send feedback

Create shared property ×

1 Metadata

2 Configuration

3 Save location

STEP 3

Save location

Select a project to save this shared property to

A - Everyone is an Owner5

Browse

Permissions

Skip

Back

Create shared property

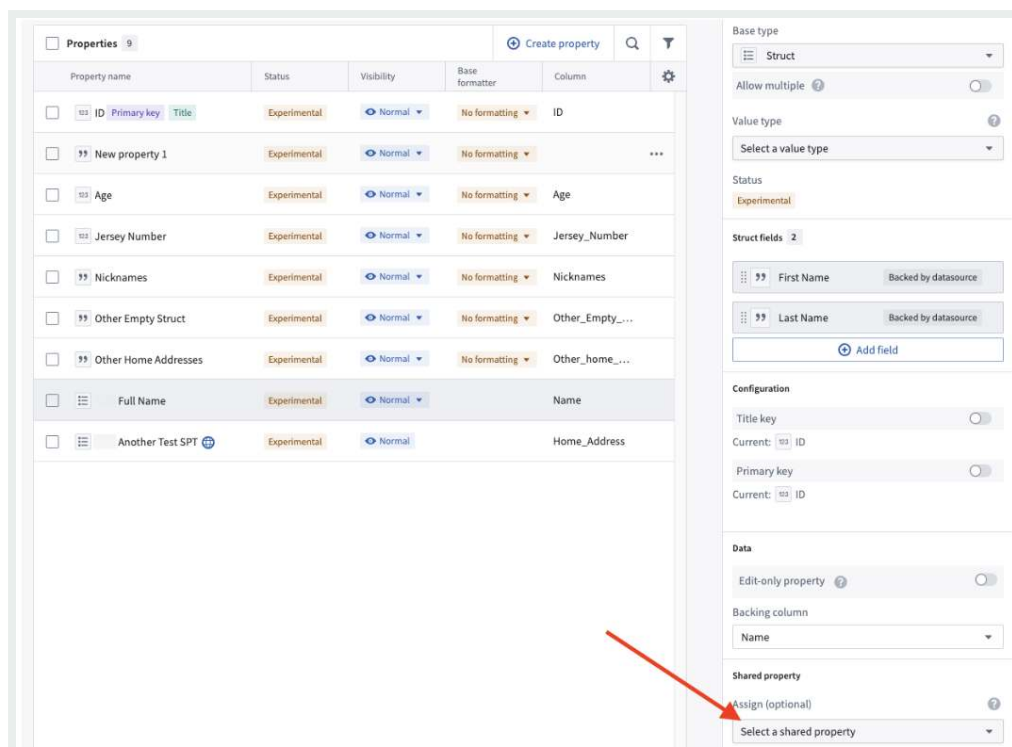
5. Back in Ontology Manager, select **Save** in the upper right corner to [make the change to your ontology](#).

Attach a struct type shared property

1. In Ontology Manager, open the **Properties** tab and select the desired property from the **Properties** table.
2. In the **Property editor** to the right, scroll down to **Shared property** and select a shared property under **Assign**. This will share property metadata among both properties.

API Reference ↗

Send feedback



Note: To add new struct fields after assigning a shared property type to a local struct property type, you must add the new struct fields to the shared property type and map them to datasource columns for all local struct property types that are backed by the shared property.

Convert a struct property type into a shared property

The following instructions detail how to convert a struct property into a struct property backed by a shared property type.

1. In Ontology Manager, open the **Properties** tab and select the desired property from the **Properties** table.
2. In the **Property editor** to the right, scroll down and select **Convert to a shared property**, which backs the struct property by a shared property type.

[API Reference ↗](#)

[Send feedback](#)

Edit-only property ?☐

Backing column

Select an option... ▼

Shared property

Assign (optional) ?

Select a shared property ▼

 Convert to a shared property

Struct main fields [Beta]

Beta

Struct main fields are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development.

Struct main fields enable you to designate a [struct's](#) core value and supplementary metadata. For example, an `Address` struct may contain fields capturing `streetName` and `postalCode` as its main values, while other fields like `collectionDate` and `collectorName` represent metadata that describe how the `Address` was obtained.

Many struct properties follow this pattern: one or more fields contain the primary data you care most about displaying in applications, while other fields provide context, tracking information, or audit details.

You can designate any of the [supported struct field types](#) as main fields.

When to use struct main fields

Use struct main fields when:

- Your struct contains core value fields in addition to fields that represent supplementary metadata.
- You want cleaner table displays without losing access to metadata.

[API Reference](#) ▸ [element interfaces](#) using only a single field or subset of fields from a

Configure struct main fields

1. Navigate to **Ontology Manager**.
2. Search for and select your object type by choosing **Object types** under **Resources** in the left panel.
3. Select the struct property to configure from your object type's **Properties** tab.
4. Choose the field to designate from the **Struct fields** section of the **General** tab that appears in the property editor panel on the right.

Locations

General

Source

Display

Interaction

Dependents

Overview

Name

Locations

Description

Base type

Structs do not yet have full platform wide support.

[See more details](#)

Struct

Allow multiple

Value type (optional)

Select a value type

Struct fields

6

+

 Add field

123

Street number

”

Street name

”

Post code

Date collected

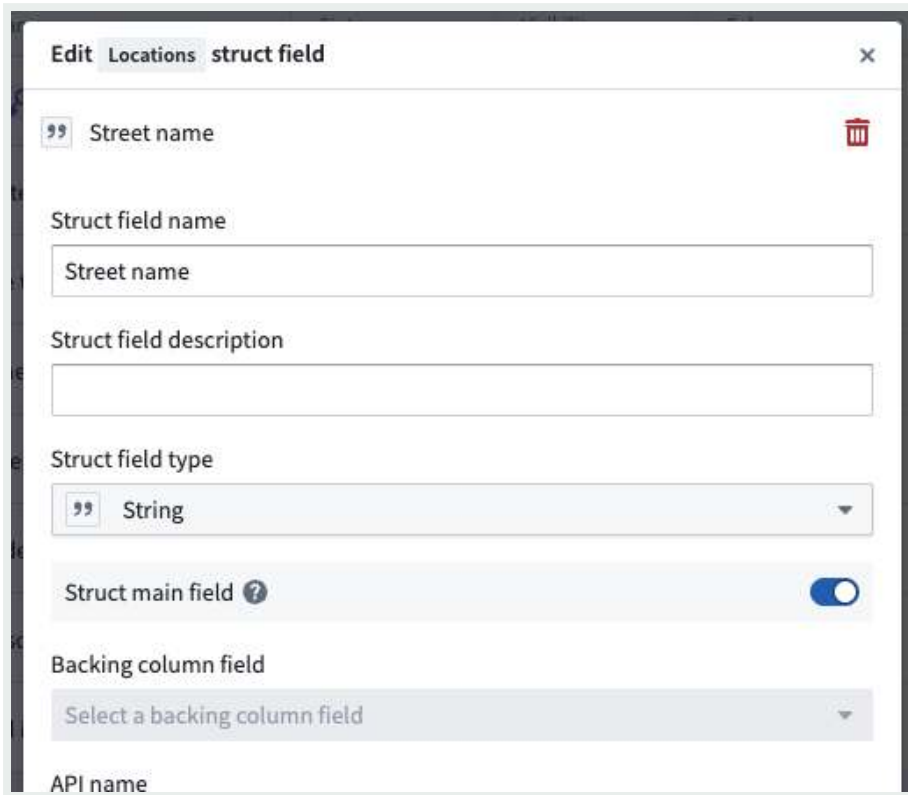
1.0

Number of guests

API Reference

Send feedback

5. Toggle on **Struct main field** in the **Edit {propertyName} struct field** popup window before you select **Confirm**. You can designate multiple main fields and reorder them for clarity by clicking and dragging a field's panel.



The screenshot shows a modal window titled "Edit Locations struct field". It contains a list of fields, with "Street name" selected. Below the list, there are several configuration options: "Struct field name" (text input with "Street name"), "Struct field description" (text input), "Struct field type" (dropdown menu with "String"), "Struct main field" (toggle switch, currently turned on), "Backing column field" (dropdown menu with "Select a backing column field"), and "API name" (text input).

6. **Save** your changes. Configured main fields display a **Struct main field** tag in the **Struct fields** list.

[API Reference ↗](#)

[Send feedback](#)

Applications that support struct main fields display only the main fields in compact views, like the [Object Table](#) and [Object List](#) widgets in [Workshop](#), while providing access to the full struct through an expanded view or upon hover. This enables you to quickly scan the most important data without seeing every struct field, while still being able to inspect the complete struct when needed.

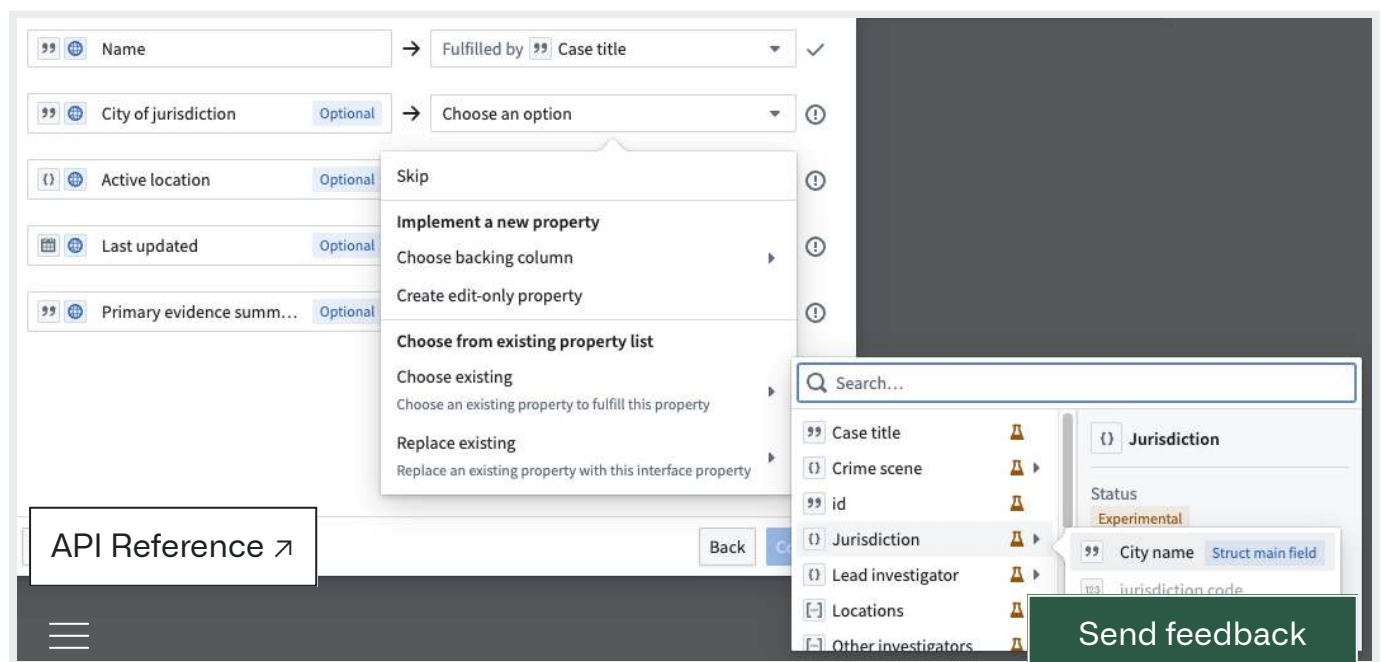
Applications support struct main fields in several ways:

- **Main fields only:** Display only main fields in compact views like tables or summary cards.
- **Main fields with hover:** Show main fields by default and reveal metadata fields on hover.
- **Full struct:** Display all fields in detailed views or forms where complete information is needed.

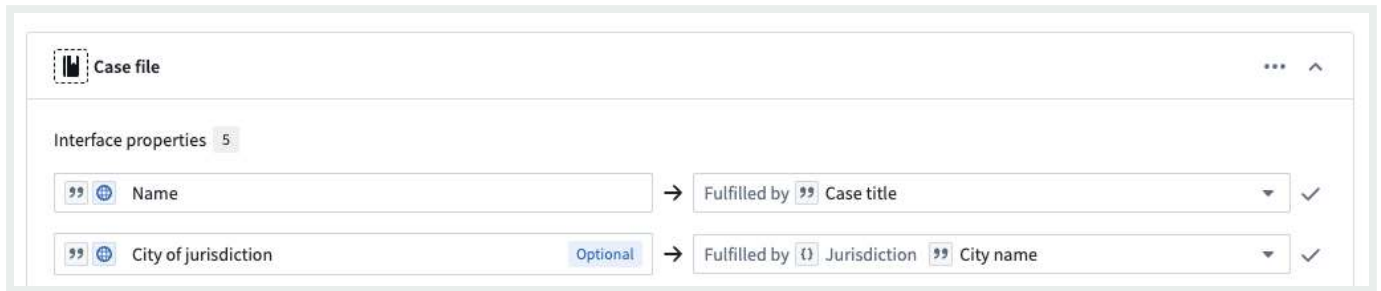
Use struct fields with interfaces

You can map any struct field to an interface property. When implementing an interface, you can select a specific field from a struct property to satisfy the interface contract and fulfill the property requirement.

For example, if an interface requires a `String` property called `cityName`, you can fulfill it using a struct property's `city` field. The interface picker displays all available struct fields with their types, allowing you to choose the appropriate field.



After mapping a struct field to an interface property, the implementation shows the struct property and the specific field being used.



The screenshot shows a window titled 'Case file' with a header bar containing a document icon and a close button. Below the header, there is a section labeled 'Interface properties' with a count of '5'. Two rows of mappings are visible:

Interface property	Mapping	Status
Name	Fulfilled by Case title	✓
City of jurisdiction	Fulfilled by Jurisdiction City name	✓

The 'City of jurisdiction' row has an 'Optional' label next to it. Each mapping row includes a small icon on the left and a checkmark on the right.

Main fields are indicated with a **Struct main field** tag in the picker, making them easy to identify. However, you can select any field from the struct regardless of whether it is designated as a main field.

Combine main fields with property reducers

Combine struct main fields with [property reducers](#) to enable additional flexibility in struct array property representation and interface implementation. A single struct array property can implement interfaces requiring `Struct Array`, the main field's array type, `Struct`, or the main field's base type.

See the [property reducers documentation](#) for detailed examples and implementation options.

Limitations and considerations

- **Query behavior:** Queries operate on all struct fields, not just main fields. You can search and filter based on any field in the struct.

FAQs

[API Reference ↗](#)

Can I change which fields are main fields later?

[Send feedback](#)

Yes, you can reconfigure main fields in Ontology Manager at any time. This may render some interface implementations invalid, which you will need to update.

Do main fields affect how Foundry stores data?

No, main fields only affect how Foundry displays data and implements interfaces. The underlying struct contains all fields with full fidelity, so all fields remain queryable and accessible.

Can I use main fields with struct arrays?

Yes. Main fields work with *both* single struct properties and struct array properties. When combined with [reducers](#), you get maximum flexibility in how Foundry represents the property across applications.

Do I need to configure main fields to use struct fields with interfaces?

No. You can map any struct field to an interface property regardless of whether it is designated as a main field. Main fields simply provide a visual indicator in the interface picker and affect how applications display the struct in compact views.

← PREVIOUS
[Structs and shared properties](#)

NEXT →
[Shared properties / Overview](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[API Reference ↗](#)

[Send feedback](#)

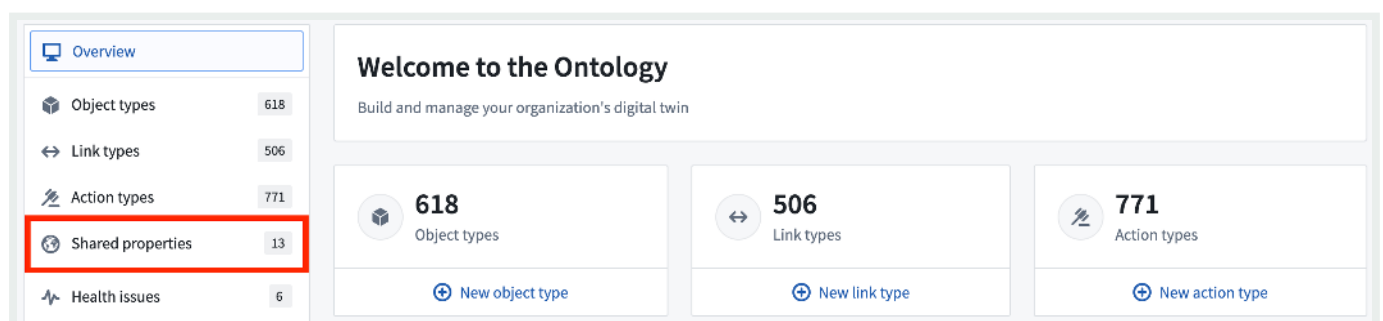
Shared properties

A **shared property** is a [property](#) that can be used on multiple [object types](#) in your ontology. Shared properties allow for consistent data modeling across object types and centralized management of property metadata. While property metadata is shared across objects, the underlying object data is not.

For example, in Ontology Manager, you may have `Employee` and `Contractor` object types that both have the property `start date`. By creating a `start date` shared property and using it for both object types, you can model your data using a consistent property and update `start date` metadata in one place instead of on each object type.

Shared properties can be [created directly](#), or existing properties on object types can be converted into shared properties. Once added to your ontology, shared properties can be [used](#) on object types as part of ontologizing your data and [edited](#) in a manner similar to regular properties.

Shared properties on objects are denoted with a globe icon next to their name.

[API Reference ↗](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

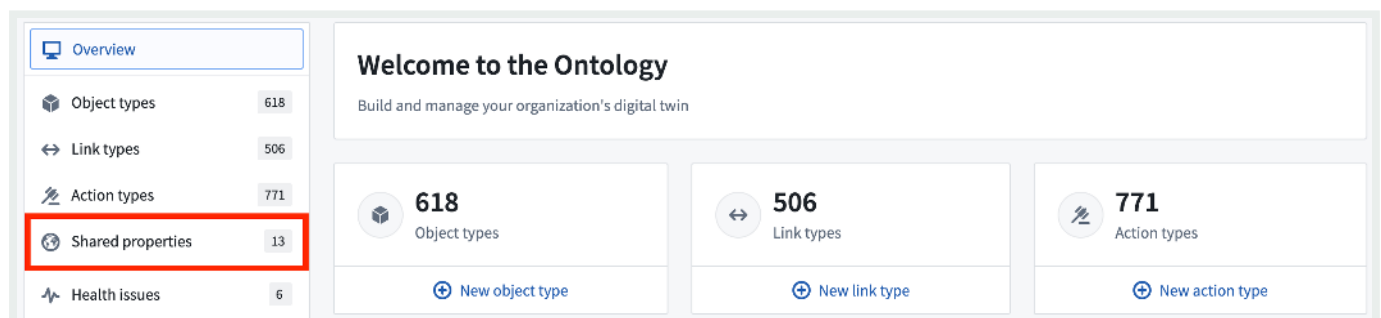
Ontology building / Object and link types / Shared properties / Create shared properties

Create a shared property

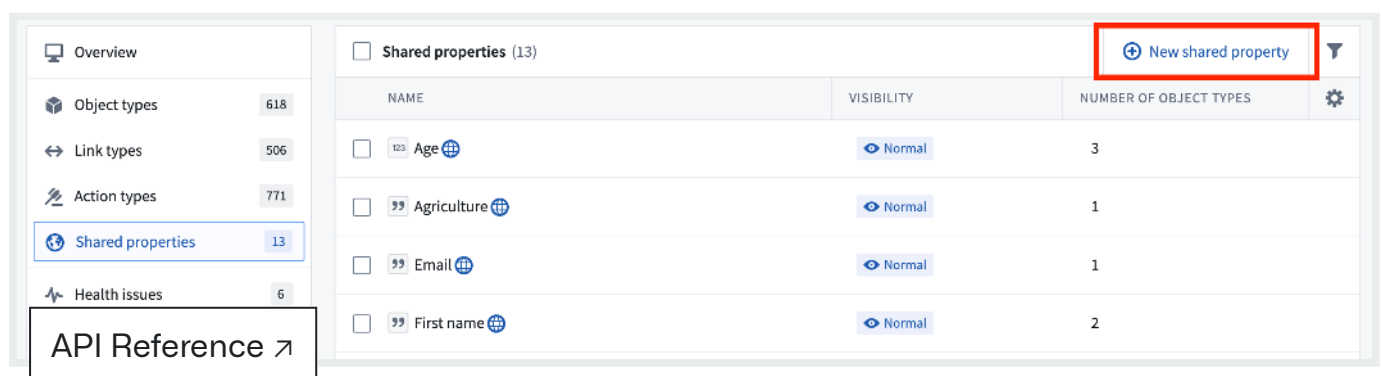
Create and configure a new shared property from the shared property page in the Ontology Manager application.

To access the page, follow the steps below:

1. Select the **Shared properties** menu option in Ontology Manager.



2. Once on the shared property page, select **New shared property** at the top right.



Send feedback

3. This will open the shared property creation modal, where you can configure the name, description, type, and other metadata to create the shared property.

The screenshot shows a modal titled "Create shared property" with a close button (X) in the top right corner. On the left, there is a sidebar with three steps: "1 Metadata" (selected), "2 Configuration", and "3 Save location". The main content area is titled "STEP 1 Metadata" and includes the instruction "Give your shared property meaningful metadata that will help others make best use of it". Below this, there are four input fields: "Name" (placeholder: "Give your shared property a name"), "Description" (placeholder: "Give your shared property a description"), "Aliases" (placeholder: "Give your shared property alternative names", with a help icon), and "API Name" (placeholder: "Specify how to refer to this shared property in code", with a help icon). At the bottom, there are two buttons: "Skip" and "Next".

A shared property can be configured with a subset of regular property metadata:

- **Name:** The name for the shared property.
- **Description:** Explanatory text about the shared property. For example, the description of the `start date` shared property may be `The day the employee or contractor began working`.
- **Base type:** Indicates the type of values for this property and determines the set of operations available in user applications. For example, the `start date` property will have base type `date`. User applications will allow you to configure a timeline widget with this property. Base types are related to the underlying column type and must match the column type in order to be applied on an object type
- **Value formatting:** Depending on the base type of the property, numeric formatting, date and time formatting, user ID, and resource ID formatting are available to apply to the property, transforming its raw values into more readable versions in user applications.

Learn more about [value formatting](#).

API Reference ↗

Additional metadata that are interpreted by user applications. Learn more about [type classes](#).

Send feedback

- **Render hints:** Indications to user applications about how to render the property that may be different than most properties of the same base type. Many render hints can be used to impact the performance of reindexes of the object type on which the property is defined. For example, if you do not expect any users to search or sort on the `start_date` property in user applications, you can deselect the `searchable` and `sortable` render hints and improve the reindex performance of the `Employee` object type. Learn more about [render hints](#).
 - **Visibility:** An indication to user applications for how prominently to display the property. A `prominent` property will lead applications to show this property first to users. A `hidden` property will not appear in user applications. By default, the `start_date` property will have `normal` visibility.
4. To persist the shared property to the Ontology, select **Save** in the upper right of the Ontology Manager.

PREVIOUS
← Overview

NEXT
Edit shared properties →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Shared properties / Edit shared properties

Edit shared properties

Edit shared property metadata

You can edit metadata for a shared property by first selecting the shared property to edit from the **Shared property** page of the Ontology Manager.

Edit shared property

General

Display

Interaction

Details

Usage

Permissions

Name

Jersey number

Description

Number of a player

Type

123

Integer

☐ Allow multiple

API Reference ↗

Done

Send feedback

The available options for editing shared property metadata are clustered into four different tabs: **General**, **Display**, **Interaction**, and **Details**. These tabs contain the following configurations:

- **Name:** The name for the shared property.
- **Description:** Explanatory text about the shared property. For example, the description of the `start date` shared property may be `The day the employee or contractor began working`.
- **Base type:** Indicates the type of values for this property and determines the set of operations available in user applications. For example, the `start date` property will have base type `date`. User applications will allow you to configure a timeline widget with this property.
- **Value formatting:** Depending on the base type of the property, numeric formatting, date and time formatting, user ID, and resource ID formatting are available to apply to the property, transforming its raw values into more readable versions in user applications. Learn more about [value formatting](#).
- **Type classes:** Additional metadata that are interpreted by user applications. Learn more about [type classes](#).
- **Render hints:** Indications to user applications about how to render the property that may be different than most properties of the same base type. Many render hints can be used to impact the performance of reindexes of the object type on which the property is defined. For example, if you do not expect any users to search or sort on the `start date` property in user applications, you can deselect the `searchable` and `sortable` render hints and improve the reindex performance of the `Employee` object type. Learn more about [render hints](#).
- **Visibility:** An indication to user applications for how prominently to display the property. A `prominent` property will lead applications to show this property first to users. A `hidden` property will not appear in user applications. By default, the `start date` property will have `normal` visibility.

Additionally, you can view the object types that use this shared property on the **Usage** tab and update the permissions on the shared property on the **Permissions** tab.

Delete a shared property

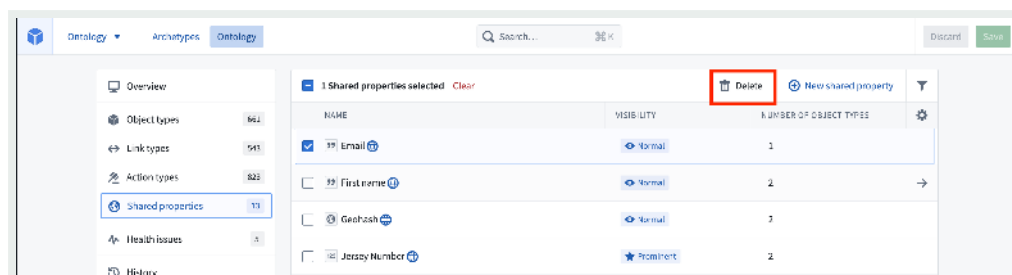
API Reference ↗

To delete a shared property, complete the following steps:

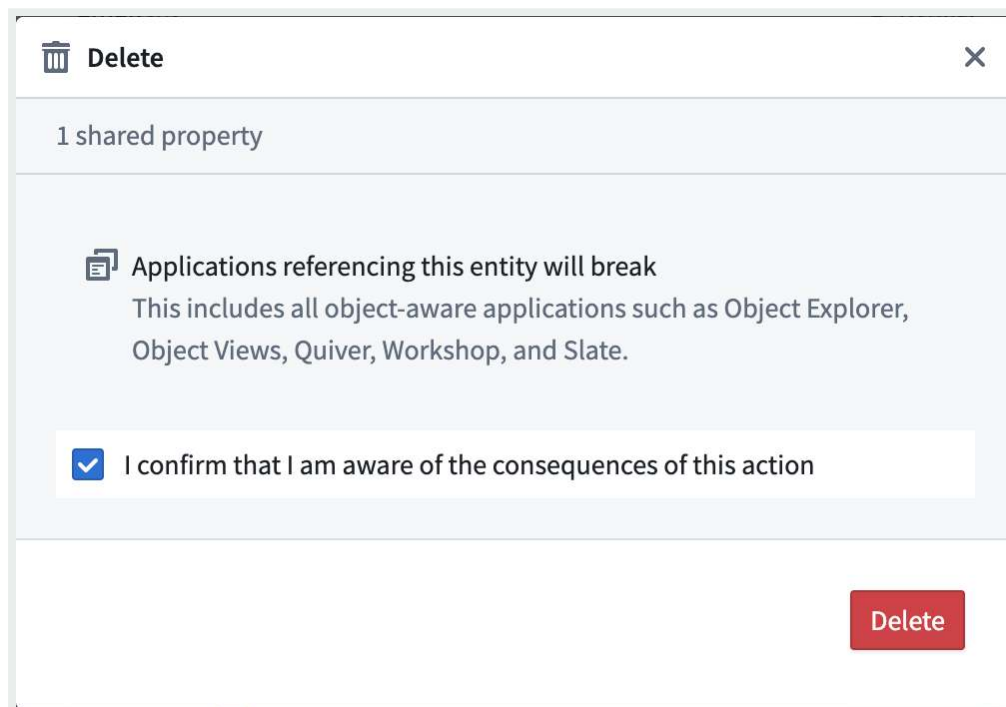
Send feedback

1. Navigate to the **Shared property** page of the Ontology Manager.

2. Select one or more shared properties for deletion, then select **Delete property**.



3. Confirm the delete action in the modal.



4. Select **Save** in the upper right.

⚠ When a shared property is deleted, all object types using this shared property will revert to regular properties.

[API Reference ↗](#)

← [Create shared properties](#)

[Use shared properties](#)

[Send feedback](#) →

NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

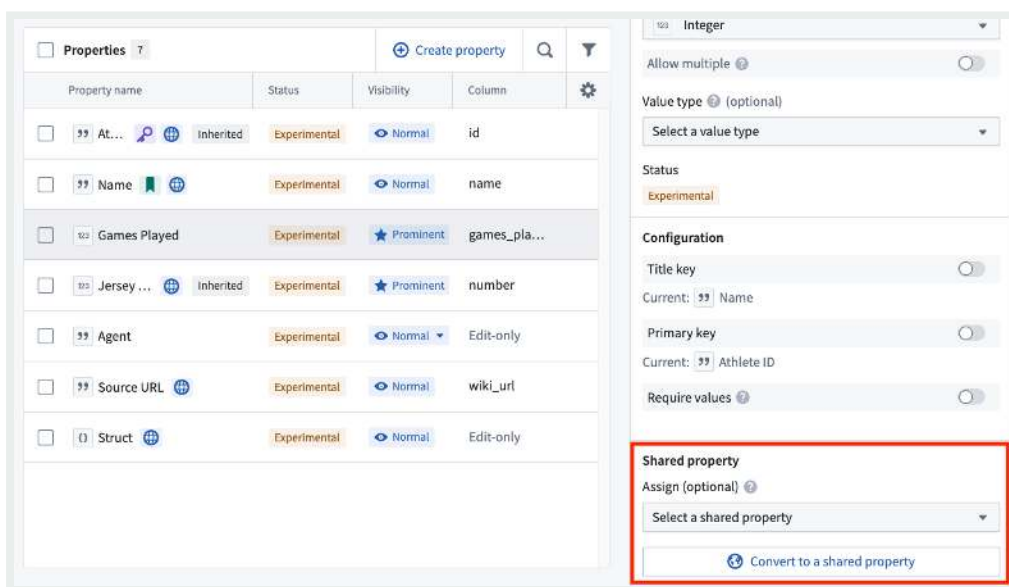
[Send feedback](#)

Ontology building / Object and link types / Shared properties / Use shared properties on object types

Use shared properties on object types

To update a property on an object type to a shared property, complete the following steps:

1. Navigate to the object type in the Ontology Manager.
2. Select the property on the panel that you want to update, then scroll down to the **Shared Property** section of the configuration.



3. Use the dropdown menu to select an existing shared property to use, or convert the property to a new shared property with the [shared property creation](#) modal.

API Reference ↗

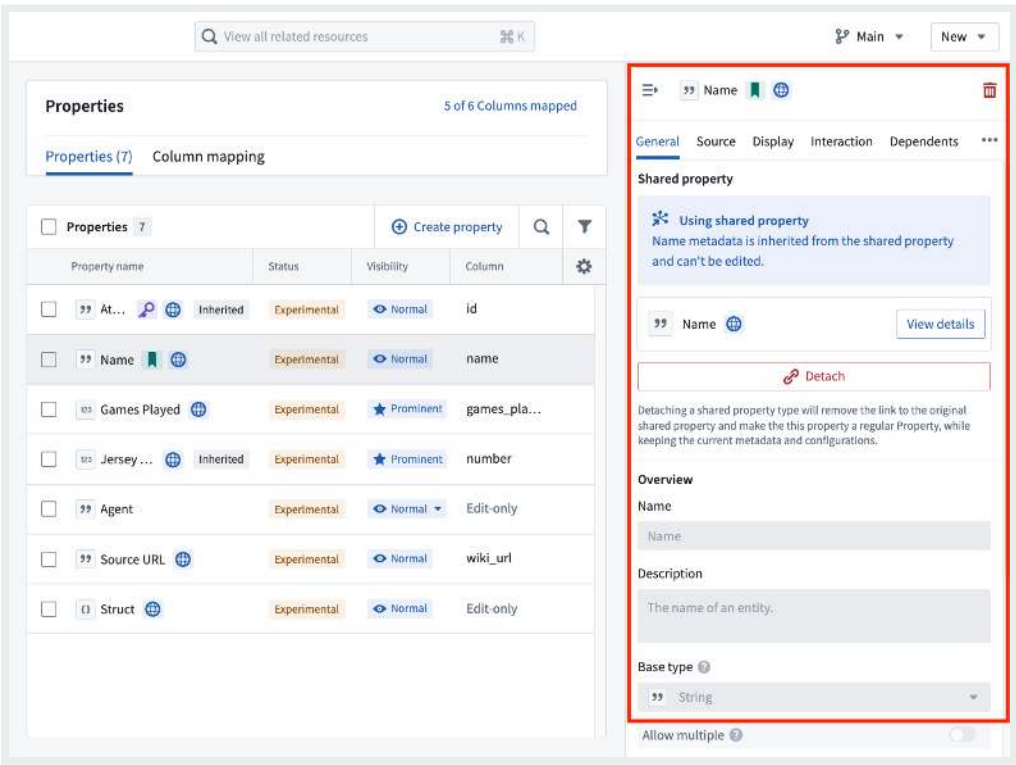
The property will then display as a shared property. To persist the use of the shared property to the ontology, select **Save** in the upper right.

Send feedback

- When using a shared property on an object, the property ID and API name of the object-specific property will remain unchanged so as to not break existing downstream workflows that leverage them.
- While associated with a shared property, direct edits to property metadata that is inherited from the shared property will be disabled. You can still add, delete, or edit type classes. When the property is loaded, the resulting set of type classes will be a union of those from the property and its associated shared property.
- If the shared property you use has different [render hint](#) configuration values than the selected property, using the shared property will override the configuration values of the selected property. Make sure your shared property is configured with the proper render hints for your use case.

Detach a shared property from an object

To detach a property from a shared property, use the same property panel on an object type in the Ontology Manager and select **Detach**.



API Reference ↗

Detaching a shared property removes the association between the property and the shared property.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Shared properties / Metadata reference

Metadata reference

A shared property is represented in the Ontology by the following metadata:

- **Name:** The name for the shared property.
- **Description:** Explanatory text about the shared property that anyone can read in user applications. For example, the description of the `start_date` shared property may be `The day the employee began new hire training`.
- **RID:** An automatically generated unique identifier for every resource in Foundry. A property's RID will be referenced in error messages across the platform.
- **Base type:** Indicates the type of values for this property and determines the set of operations available in user applications. For example, the `start_date` property will have base type `date`. User applications will allow you to configure a timeline widget with this property.
- **Value formatting:** Depending on the base type of the property, numeric formatting, date and time formatting, user ID, and resource ID formatting are available to apply to the property, transforming its raw values into more readable versions in user applications. Learn more about [value formatting](#).
- **Type classes:** Additional metadata that are interpreted by user applications. Learn more about [type classes](#).
- **Render hints:** Indications to user applications about how to render the property that may be different than most properties of the same base type. Many render hints can be used to impact the performance of reindexes of the object type on which the property is defined. For example, if you do not expect any users to search or sort on the `start_date` property in user applications, you can deselect the `searchable` and `sortable` render hints and improve the reindex performance of the `Employee` object type. Learn more

API Reference ↗

Visibility: An indication to user applications for how prominently to display the property. The `visible` render hint property will lead applications to show this property.

Send feedback

`hidden` property will not appear in user applications. By default, the `start date` property will have `normal` visibility.

- **Usage:** The object types on which a shared property is used. For example, the `start date` property can be in use by the `Employee`, `Contractor`, and other object types within the Ontology.

PREVIOUS

← [Use shared properties on object types](#)

NEXT

[Link types / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Link types

A **link type** is the schema definition of a relationship between two object types. A **link** refers to a single instance of that relationship between two objects in the same Ontology.

For example, in the Ontology Manager, you may create a link type between the `Employee` object type and the `Company` object type that defines the relationship between `Employee` and `Employer`. A link refers to a single instance of the `Employee → Employer` link type, like the relationship between the notional employee “Melissa Chang” and her employer, “Acme, Inc.”

Similarly, in the Ontology Manager, you may create a link type between the `Flight` object type and the `Aircraft` object type that defines the relationship between `Scheduled Flight` and `Assigned Aircraft`. A link refers to a single instance of the `Scheduled Flight → Assigned Aircraft` link type, like the relationship between “JFK → SFO 24-02-2021” and its assigned aircraft “Boeing 737-123”.

Links can also exist between two objects of the same type. A link type `Direct Report ↔ Manager` can be defined between the `Employee` object type and itself.

Note that links between object types across different Ontologies is not supported. In this case, you may prefer to leverage a shared Ontology.

Directionality

A link type is bidirectional: it always has two **sides**, one for each of the two object types it relates. A link type can be traversed independently and has its own display name and a `name` attribute. For example, a single `Flight ↔ Aircraft` link type includes a `Flight` side and an `Aircraft` side. From code, calling `flight.assignedAircraft` returns the `Aircraft` side of the link type to retrieve the aircraft assigned to a flight, and calling `aircraft.flight` returns the `Flight` side of the link type to retrieve the flight assigned to an aircraft.

API Reference ↗

Send feedback

`aircraft.flights.all()` traverses the `Flight` side to retrieve the flights assigned to that aircraft.

i Note

Creating a single link type between two object types does not implicitly create a second, reverse link type. Instead, that single link type already supports traversal in both directions through its two sides, so there is no need to separately define a link type from `Aircraft` to `Flight` and another from `Flight` to `Aircraft` to represent the same relationship.

You can define multiple, distinct link types between the same two object types, but each one represents a separate real-world relationship rather than a reverse direction of an existing one. For example, in addition to a `Flight ↔ Aircraft` link type representing an assigned aircraft, you could define a second, independent `Flight ↔ Aircraft` link type representing a scheduled maintenance record. Each link type requires unique [API names](#) on its sides so applications can distinguish between the relationships.

The concepts underpinning the Ontology have analogous concepts in the structure of a dataset. The definition of a link type in the Ontology is analogous to that of a join between two datasets, while the definition of a link is analogous to that of a row joined with the fields of the same row in another dataset. For example, you can join the `Employee` dataset with the `Company` dataset to explore the relationship between `Employees` and their `Employers`. In the joined dataset, a single row that joins “Melissa Chang” with her employer “Acme, Inc.” represents a link.

Rather than being an abstract data model, the Foundry Ontology maps each ontological concept to an organization's actual data, enabling this data asset to power real-world applications. Links are created and displayed in user applications by adding backing datasources to the object types referred to in the link type in the Ontology Manager. In the case of link types where object types are related with a many-to-many cardinality, datasources back the link types themselves. To create links of type `Employee → Employer`, an organization will add backing datasources to the `Employee` and `Company` object types and connect their employee directory and other enterprise data into the Ontology.

API Reference ↗

Getting started: how to [create a new link type](#).

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Link types / Create a link type

Create a link type

We recommend creating and configuring a new link type with the guided helper outlined below. However, if you exit the helper before completing the object creation process, you can manually complete the process by specifying the link type, keys, and API names for the new link type.

Access the link type creation helper

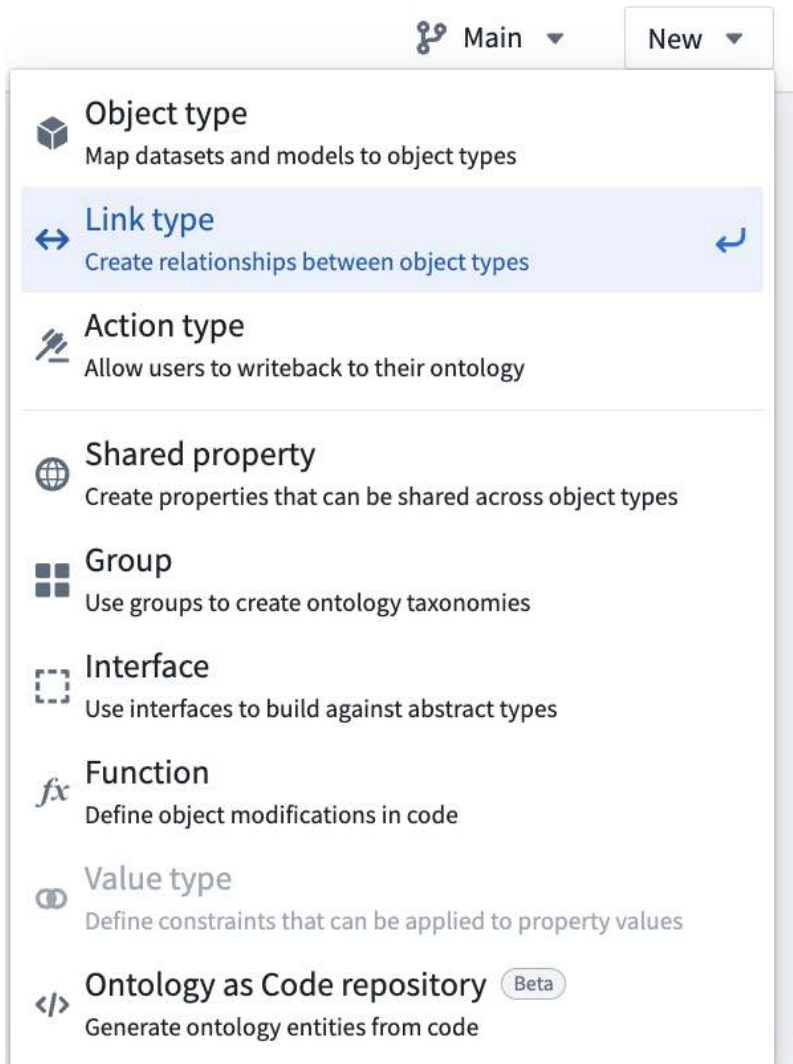
Navigate to Ontology Manager. To access the link type creation helper, choose one of the following methods:

- Select **New** from the top right corner, then select **Link type**.

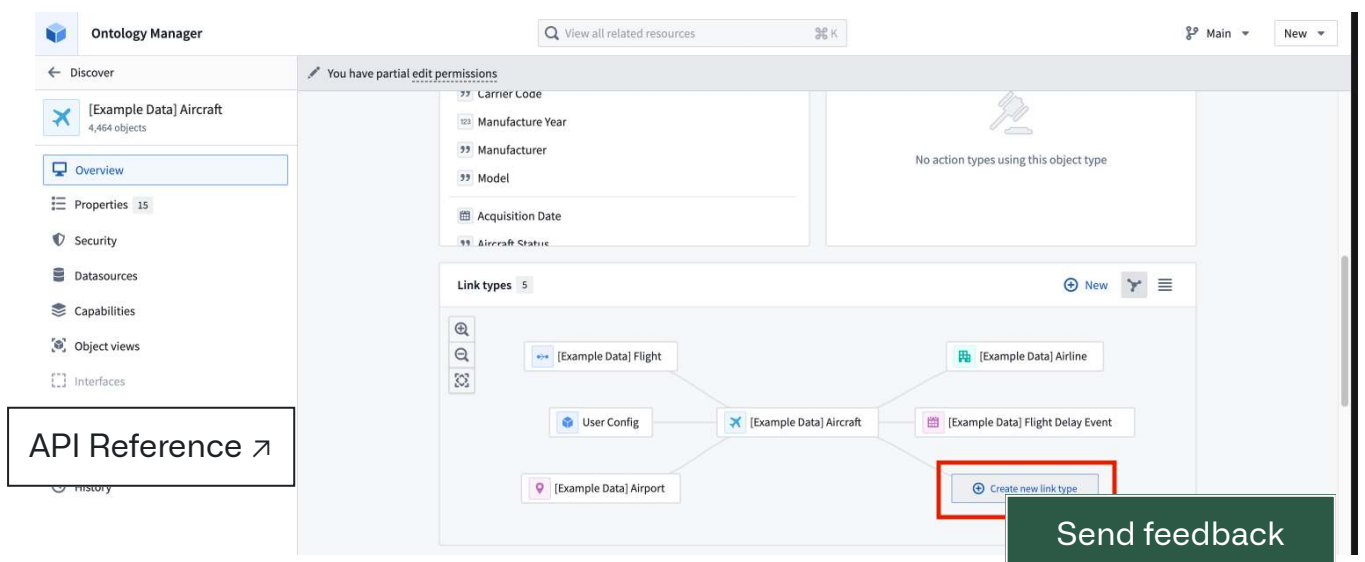
API Reference ↗



Send feedback



- In the left sidebar, select **Link types** under **Resources**. Then, select **New link type** in the top right corner of the **Link types** page.
- Navigate to an object type you want to link, then select **Create new link type** from within the link type graph on the object type's **Overview** page.



Configure a new link type

The new link type helper will guide you through the following steps:

- [Choose the relationship type](#)
- [Define link resources](#)
 - [Foreign key relationship type](#)
 - [Join table dataset relationship type](#)
 - [Backing object relationship type](#)
- [Define link type names](#)
- [Save location](#)
- [Save change to ontology](#)

Choose the relationship type

1. In the first step of the **Create a new link type** dialog, select the relationship type for the link.
2. Choose the relationship type for defining links between your two objects:
 - **Object type foreign keys:** Supports "one-to-one" and "many-to-one" cardinality link types. This option allows you to select properties that represent the foreign key and corresponding primary key for the desired objects. See [below for details](#) in defining link resources with a foreign key.
 - **Join table dataset:** For "many-to-many" cardinality link types. This option allows you to use a join table dataset to back the link. See [below for details](#) in defining link resources with a dataset.
 - **Backing object type:** Object-backed link types expand on many-to-one cardinality link types, providing first class support for object types as a link type storage solution. See [below for details](#) on defining link resources backed by an object. For additional information, refer to the [object-backed links](#) section.

In the examples below, assume that there are two object types that are related to each other through a cardinality: an `Aircraft` object type and a `Flight` object type. Cardinality types include:

API Reference ↗

- *One-to-one cardinality:* This indicates that one `Aircraft` should be associated with one `Flight`. The one-to-one cardinality serves as an indicator of the

Send feedback

but the one-to-one cardinality is not enforced.

- *One-to-many cardinality:* This indicates that one **Aircraft** can be linked to many **Flights**.
- *Many-to-one cardinality:* This indicates that many **Aircraft** can be linked to one **Flight**.
- *Many-to-many cardinality:* This indicates that one **Aircraft** can be linked to many **Flights**, and one **Flight** can be linked to many **Aircraft**.

3. Select **Next** to proceed to the next step.

Create a new link type

1 Relationship type

2 Link resources

3 Link type names

4 Save location

STEP 1

Relationship type

Choose method for defining links between instances

☒ Object type foreign keys

Link object types using a foreign key property.
Used for one-to-one and one-to-many link types.



☐ Join table dataset

Link object types using a dataset containing columns with object type primary keys.
Used for many-to-many link types.

☐ Backing object type

Link object types and define properties using a middle backing object type.
Used for many-to-many link types that include properties.

Next

Define link resources

Foreign key relationship type

API Reference ↗

A **foreign key** is a property on one object type that stores the value of another object type's primary key, representing a real-world relationship between the two concepts. This concept is analogous to a foreign key in a relational database, where a column in one table

Send feedback

references the primary key column of another table. In a one-to-one or many-to-one cardinality link type, you will define the foreign key property and primary key properties for the link. The **foreign key** property of one object type must refer to the **primary key** property of the other object type.

For example, the `Tail Number` property is the primary key on the `Aircraft` object type, uniquely identifying each aircraft. The `Flight Tail Number` property on the `Flight` object type is the foreign key, representing the aircraft that was assigned to that flight. Links will be created between `Aircraft` and `Flight` object types when the `Tail Number` of the `Aircraft` matches a `Flight Tail Number` of a `Flight`.

1. In the **Link resources** step, choose the object types for your link.
2. Select the primary key object type from the dropdown menu on the right (`Aircraft` in our example).
3. Select the foreign key object type from the dropdown menu on the left (`Flight` in our example). The creation dialog will detect and automatically select a foreign key if the following conditions are met:
 - The foreign key matches the primary key of the linked object type.
 - The property types of both objects match.
4. Choose the properties that will form the link:
 - For the foreign key object type, select the property that will be used as the foreign key for the source object type (`Flight Tail Number` for the `Flight` object types).
 - The primary key of the object type is auto-selected since there is only one primary key for each object type (`Tail Number` for the `Aircraft` object type).
5. Select **Next** to continue.

[API Reference ↗](#)

[Send feedback](#)

Create a new link type

1 Relationship type

2 Link resources

3 Link type names

4 Save location

STEP 2

Link resources

Choose the object types you wish to link and the property to use as a foreign key.

[CSS - Example Data] F...

[CSS - Example Data] A...

Flight Tail Number

Tail Number Primary key

Back

Next

Join table dataset relationship type

In a many-to-many cardinality, select a datasource that includes all combinations of links between the primary key of the first object type (`Aircraft` in our example) and the second object type (`Flight` in our example).

A many-to-many cardinality, which requires a backing datasource, is required to enable users to [edit or write back](#) to the link type.

1. In the **Link resources** step, choose the object types for your link.
2. Select the first object type from the dropdown menu on the left (`Flight`).
3. Select the second object type from the dropdown menu on the right (`Aircraft`).
4. Choose the join table dataset. Select a dataset that contains columns matching the primary keys for both selected object types. A column can only be mapped to one

API Reference ↗

- It is now possible to automatically generate a join table for new

Generate join table option will create a dataset with the correct cardinality and create

Send feedback

primary keys of the two object types you have selected. This means that you can get started faster if you have user edit-backed data, or if you want to provide production data later on.

5. Select which columns in the link type's backing datasource map to the primary keys of each of the linked object types.
6. Select **Next** to continue.

Create a new link type

1 Relationship type

2 Link resources

3 Link type names

4 Save location

STEP 2

Link resources

Choose the object types you wish to link.

[CSS - Example Data] F... [CSS - Example Data] A...

Flight ID Primary key Tail Number Primary key

Select join table dataset

Flight Aircraft Join Table /Users/ Replace

Column matching [CSS - Example Data] ... Column matching [CSS - Example Data] ...

Flight__Tail_Number Aircraft_Tail_Number

Back Next

Backing object relationship type

Before creating the object-backed link, ensure that the [prerequisite](#) object and links have been created.

1. Select the object types created in the prerequisites to represent your desired link type.

The objects on the left and right represent the two entities that will be linked together.

API Reference ↗ middle serves as the intermediary and provides additional metadata about the connection between the two entities, and backs the link.

2. If there are multiple links between the objects on the left and right object in the middle, use the dropdown menus to select the desired links between the

Send feedback

left and right objects and the intermediary object.

Create a new link type

1 Relationship type

2 Link resources

3 Link type names

4 Save location

STEP 2

Link resources

Choose the object types you wish to link. Properties will be defined by the middle backing object type.

Airplane Data

Flight Manifests

Flight Data

Link type from Flight Manifest to Airplane Data

Flight Manifests Flight Manifest to Airplane Datum Experimental

Link type from Flight Manifest to Flight Data

Flight Manifests Flight Manifest to Flight Datum Experimental

Skip

BackNext

Define link type names

1. In the **Link type names** step, provide the display and API names for your new link type.
2. Enter a display name for each side of the link type. A link type has exactly two sides, one for each of the two object types it relates, and each side represents the link to that object type. In our example, the display name for the **Aircraft** object type describes the link from **Flight** to **Aircraft**. You could choose the display name **Assigned Aircraft** since one **Flight** has one **Assigned Aircraft**. Because both sides are configured together for the same link type, the link type can be traversed in either direction; there is no separate link type to define for the reverse direction. See [directionality](#) for more details.

API Reference be automatically generated based on the display name, but you can modify it if needed.

- The API name field is used when referring to a link type programatically. The API name on a side of a link type can be used to return objects of that type. For

Send feedback

example, if the API name on the `Aircraft` side of the link type is `assignedAircraft`, then calling `Flight.assignedAircraft.get()` will return the `Aircraft` objects linked to those `Flight` objects.

- Link type API names *must* adhere to the following:
 - Begin with a lowercase character and consist of only alphanumeric characters.
 - Be unique across all link types associated with the same object type.
 - Be between 1 and 100 characters long.
 - Be NFKC normalized.
 - Not be a reserved keyword.
- [Learn more about API names.](#)

4. Select **Next** to proceed.

Create a new link type

1 Relationship type

2 Link resources

3 Link type names

4 Save location

STEP 3

Link type names

[CSS - Example Data] Flight → [CSS - Example Data] Aircraft

Each [CSS - Example Data] ... has many [CSS - Example Data] Ai X

API Name cssExampleDataFlight. cssExampleDataAircraft .all()

[CSS - Example Data] Aircraft → [CSS - Example Data] Flight

Each [CSS - Example Data] ... has many [CSS - Example Data] FI X

API Name cssExampleDataAircraft. cssExampleDataFlights .all()

Back

Next

Save location

API Reference ↗

In the final step, choose a project to save this link type to. Then, **Submit** these steps, your new link type will be created, but not yet saved.

Send feedback

Create a new link type

1 Relationship type

2 Link resources

3 Link type names

4 Save location

STEP 4

Save location

Select a project to save this link type to

Browse

Permissions

Back

Submit

Save change to ontology

Back in Ontology Manager, select **Save** in the upper right corner to [make the change to your ontology](#).

Object-backed links

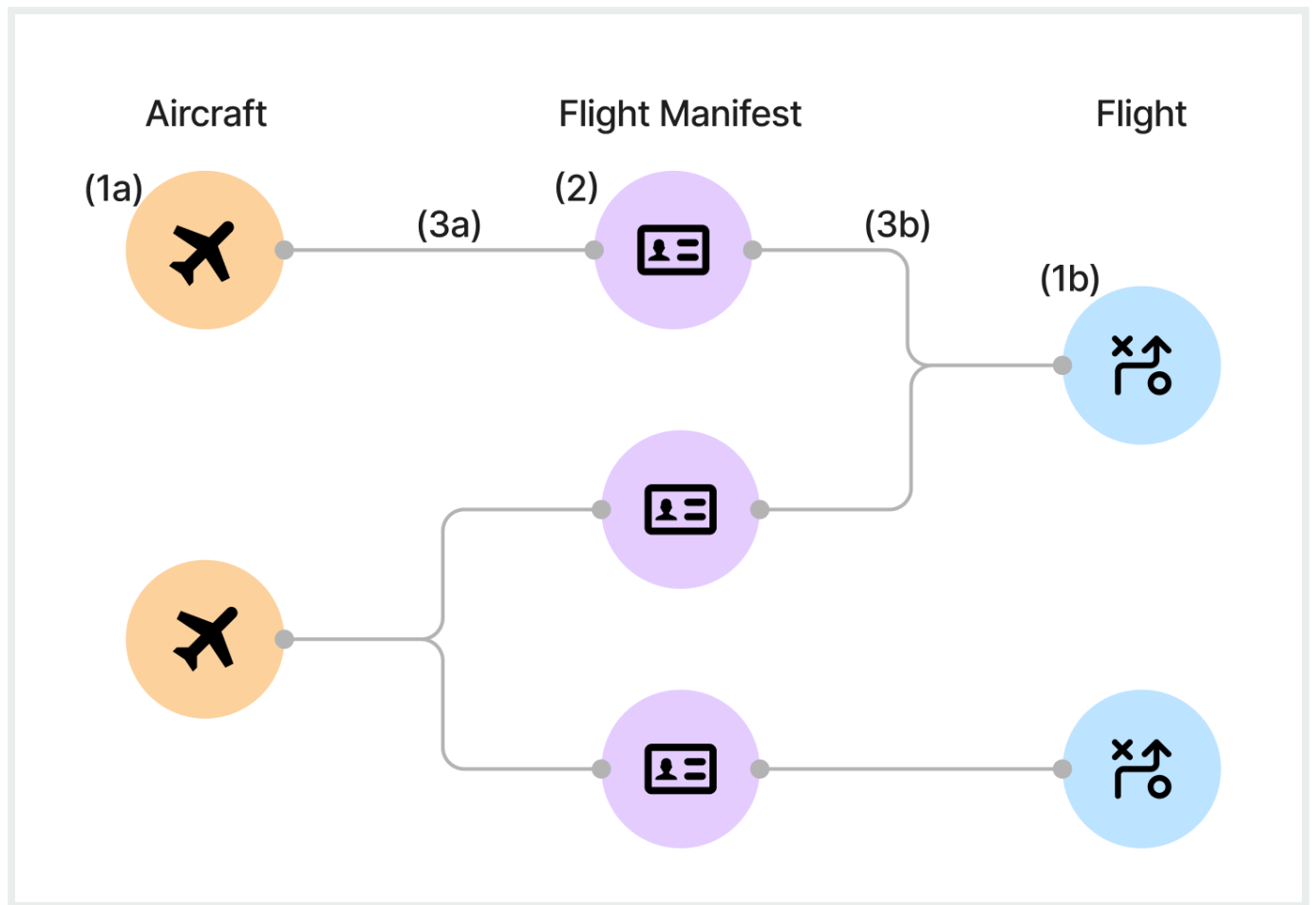
Object-backed link types expand on many-to-one cardinality link types, providing first class support for object types as a link type storage solution. Object-backed link types allow for the inclusion of additional metadata on the link and support restricted views.

For object-backed links, in addition to the `Aircraft` and `Flight` objects, assume an additional object type for the `Flight Manifest`. With an object-backed link, you can have the `Flight Manifest` object type that links the `Aircraft` and `Flight` objects. Unlike a foreign key or data-set backed link, this `Flight Manifest` object can contain additional p

API Reference ↗

ilot and First Mate to provide additional metadata on the link.

Send feedback



Prerequisites for creating an object-backed link type

Before you can create an object-backed link type, you must first do the following:

1. Create the object types on either side of the link type. See [create an object type](#) for additional details.
2. Create the backing object type that links the two object types together.
3. Create the many-to-one link types between each side of the link type to the backing object type. See [configure a new link type](#) for additional details.

For the **Aircraft**, **Flight**, and **Flight Manifest** example from above, you need to create the following resources:

1. Create the object types on either side of the link type.

API Reference ↗
Object type
Link type

2. Create the backing object type that links the two object types together.

1. **Flight Manifest** object type

Send feedback

3. Create the many-to-one link type between each side of the link type to the backing object type.
 1. Link between the `Aircraft` object type and the `Flight Manifest` object type
 2. Link between the `Flight` object type and the `Flight Manifest` object type

Once these have been created, you can create the object-backed link type.

Convert existing links to object-backed link types

Existing links can be converted to object-backed link types. Before modifying existing links, the [prerequisites](#) for object-backed link types must be fulfilled.

To modify the link type of an existing link:

1. Open the link in Ontology Manager.
2. In the **Configuration** section, update the join method and select **Object type**.
3. Select the backing object type in the **Update link type to object-backed link type** dialog.
4. Select the link type from the link edges to the backing object in the **Update link type to object-backed link type** dialog.
5. Select the **Update to object-backed**.

Use object-backed link types

Currently, object-backed link types can be viewed in Object Explorer, Vertex, and Workshop. Select a link to view the link's backing object properties. Note that in Vertex, the link title will display the link's backing object title instead.

Troubleshooting

Error: Phonograph2:DatasetAndBranchAlreadyRegistered

[API Reference ↗](#)

If you receive the error `Phonograph2:DatasetAndBranchAlreadyRegistered`

[Send feedback](#)

backing the link type you are trying to save is already backing a different link type in the

Ontology and cannot be used again.

PREVIOUS
← Overview

NEXT
Edit link types →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Edit link types

⚠ Warning

Editing a link type can have **application-breaking consequences** that can disrupt user workflows. Read the section below on [potential breaking changes](#) **before** proceeding with any link type edits.

Potential breaking changes

Link type without writeback

Changes that require Object Storage v1 (Phonograph) to unregister and reregister the backing datasource of a link type will make the links of that type **unavailable** in user applications during that reindex time; these changes are described below.

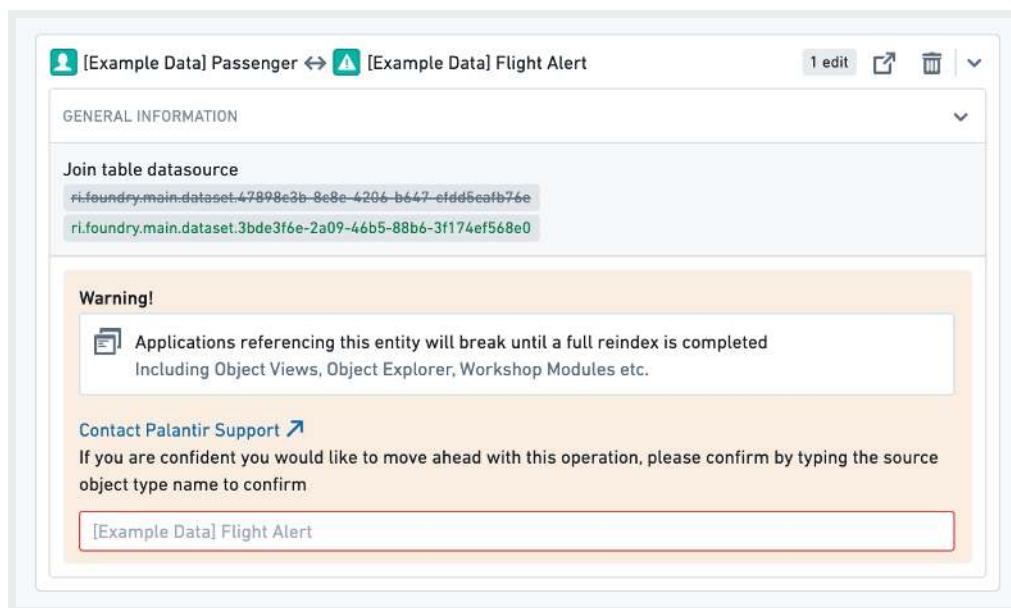
The following changes will unregister and reregister (or delete) the backing datasource of a link type when saved:

- Changing a many-to-many link type's backing datasource.
- Changing the cardinality of a link type.
- Changing the foreign key of a link type.
- Deleting a link type.

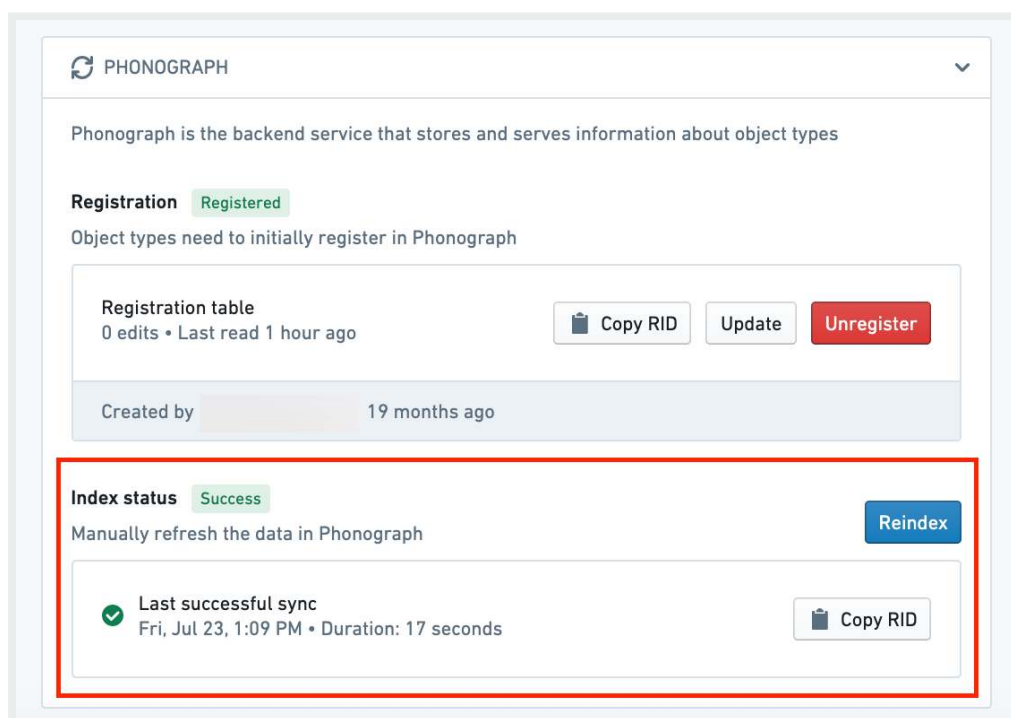
When you make any of these changes, you will be warned about the potential impact on user applications.



Send feedback



For example, if a link type is used in a search around in a Workshop application, that Workshop application will be broken until the reindex completes. You can track the progress of the reindex for a link type in the **Phonograph** pane of its **Datasources** page.



[API Reference ↗](#) [Object Storage v1 \(Phonograph\)](#)

Send feedback

Link type with writeback

If a link type has writeback enabled, extra precaution should be taken when making edits to that link type. The history of edits made to a link type are stored in Object Storage v1 (Phonograph). Every time a writeback dataset is built, the history of edits is reapplied to get the final state of edited links in the writeback dataset. When the backing datasource of a link type is unregistered from Object Storage v1 (Phonograph), the history of edits in Object Storage v1 (Phonograph) is deleted and future builds of the writeback dataset will fail.

In addition to the changes that require unregistering that were listed in the [previous section](#), unregistering is required for link types with writeback when schema changes are made to **any** column of a backing datasource to a link type that has **ever** received edits, even if does not currently receive edits. Schema changes include changes to the name and base type of the column.

Warning

Object Storage v1 (Phonograph) will **not** automatically unregister the backing datasource of a link type in response to one of these schema changes. Instead, the reindex will fail and will only succeed if the saved schema changes are undone, or if you manually unregister and reregister the backing datasource of a link type in the Phonograph pane of the link type's Datasources page.

When you try to save any changes that risk erasing the edits history, you will be warned about the potential impact on edits.

Warning!

 **388 edits will be undone**
This includes all additions, deletions and modifications to objects of this type

 Applications referencing this entity will break until a full reindex is completed
Including Object Views, Object Explorer, Workshop Modules etc.

[Contact Palantir Support](#) 

If you are confident you would like to move ahead with this operation, please confirm by typing the number

API Reference 

Send feedback

Now that you understand the considerations in editing existing link types, you can safely make your changes.

Edit an existing link type

- [Navigate to an existing link type](#)
- [Delete a link type](#)
- [Change a backing datasource](#)
- [Edit a link type's metadata](#)

Navigate to an existing link type

You can always change the link type you are working on by selecting the link type page from the home page sidebar and selecting a different link type from the list. You can also always search for a new link type in the search bar in the application header. Read more about [navigation](#).

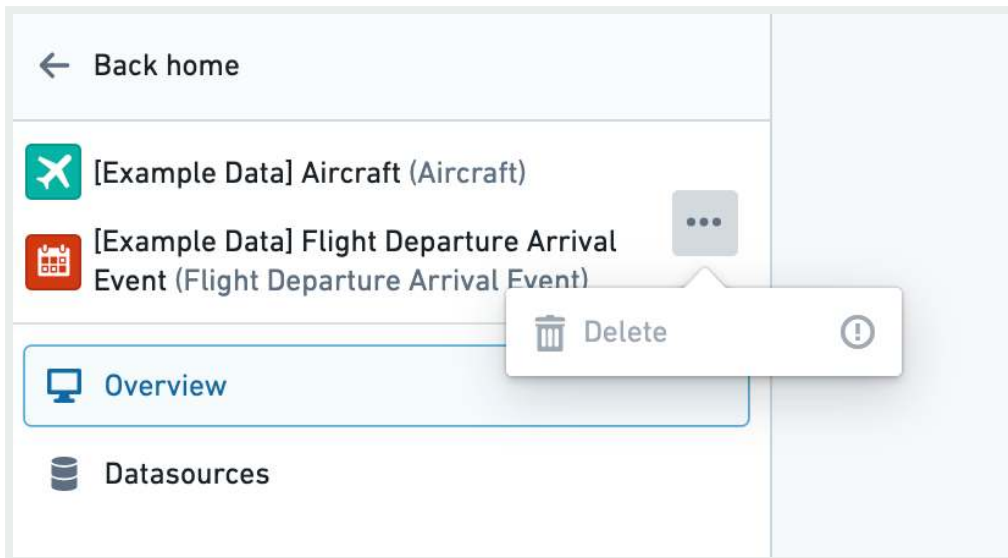
Delete a link type

You can delete an object type by selecting the  (three dots) icon at the top right of the link type view sidebar (see image below) and then selecting the **Delete** option from the dropdown. A dialog will pop up to confirm you want to stage the link type for deletion.

- Note that the deletion of the link type only takes effect after you save your changes, and will break any views or applications referencing the object type.
- Note that link types with an **active** status cannot be deleted. Read more about [statuses](#).

[API Reference ↗](#)

[Send feedback](#)



Change a backing datasource

You can change a backing datasource:

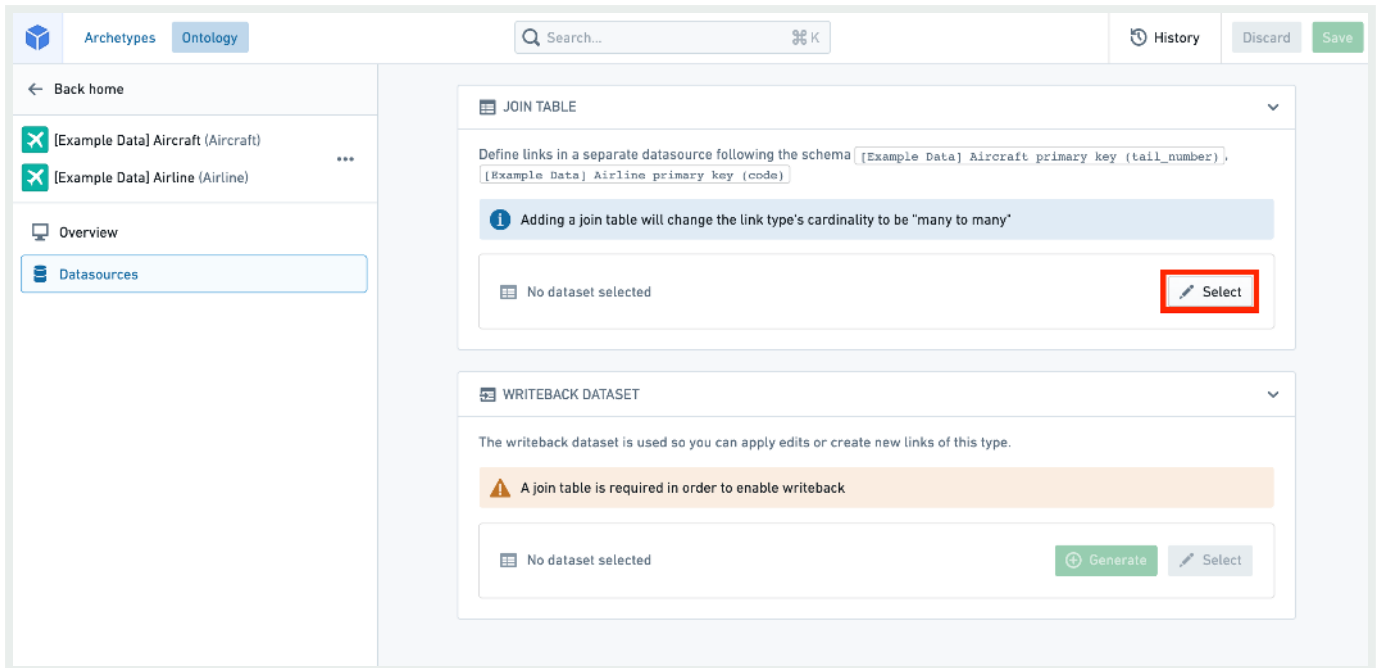
1. Navigate to the **Datasources** page of the link type view.
2. Select the  **Select** icon next to the existing datasource. This will allow you to browse and select available datasources in Foundry.

Warning

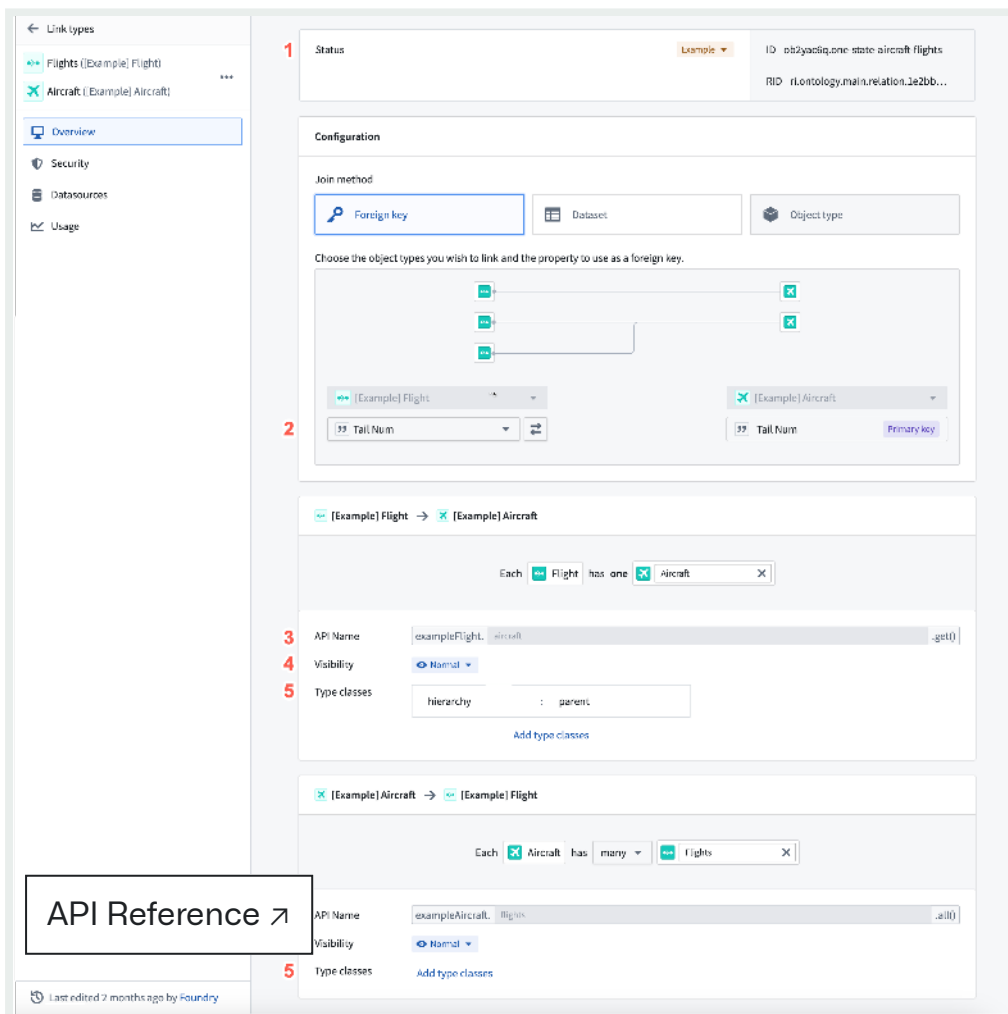
Changing the backing datasource of a link type will remove any connection between columns in the old datasource and the keys that define your link type. Keys will be automatically remapped for you **only if** you change to a new datasource with the **same schema** as the old datasource. Otherwise, you will need to remap the keys to the new datasource.

[API Reference ↗](#)

[Send feedback](#)



Edit a link type's metadata



1. **Status:** Select the existing status at the top of the link type pane to open a dropdown of available statuses. Select from the `deprecated`, `experimental`, and `active` statuses.
 - Read more about [statuses](#).
2. **Key:** Select from the dropdowns to change foreign keys, or column mappings in a many-to-many link type.
 - Note that in a link type with many-to many cardinality, the columns in the backing datasource must map to the primary keys of the object types. If the type of the primary key property of the object type is not the same as the type of the column it is being mapped to in the link type's backing datasource, an error will prevent you from saving.
 - In a link type with any other cardinality, the application requires that the key of one of the object types must map to the Primary key of that object type, ensuring that the "one" side of the Cardinality is unique.
3. **API name:** Select into the existing API name to change its value.
 - Note that you cannot change the API name for link types with an `active` status.
 - Read more about [statuses](#).
 - Read more about [valid API names](#).
4. **Visibility:** Check the visibility from the link visibility list. A `prominent` link type will prompt applications to show this link type first to users. A `hidden` link type will not appear in user applications.
5. **Type classes:** Apply type classes as additional metadata that can be interpreted by applications.
 - Consult the [list of available type classes](#) for more information.

PREVIOUS

← Create a link type

NEXT

Metadata reference →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

Send feedback

Metadata reference

A link type is represented in the Foundry Ontology by the following metadata:

- **ID:** A unique identifier of the link type, primarily used to reference links of this type when configuring an application. For example, `employee-employer` may be the ID of the link type defined between the `Employee` and `Company` object types.
- **RID:** An automatically generated unique identifier for every resource in Foundry. A link type's RID will be referenced in error messages across the platform.
- **Status:** A signal to users and other Ontology builders about where in the development process the link type stands. It can be `active`, `experimental`, or `deprecated`. By default, the `Employee → Employer` link type will have status `experimental`. Read more about [statuses](#).
- **Object types:** The object types related through the link type definition. For example, the `Employee → Employer` link type will refer to the `Employee` and `Company` object types.
- **Cardinality:** Indicates to applications if each object type in the link type has one or many objects. For example, The `Employee` object type in link type `Employee → Employer` has cardinality `many` and the `Company` object type has cardinality `one`, since many employees are linked to a single employer. The `Employee` object types in link type `Direct Report ↔ Manager` will each have cardinality `many` if a direct report can have multiple managers and if a manager can have multiple direct reports.
- **Key:** The properties or columns used to create the links.
 - In a one-to-one or in a one-to-many cardinality link type, a property of one object type (the foreign key) refers to the primary key property of the other object type. This reference between a foreign key and primary key defines the links between objects. For example, in the `Employee → Employer` link type, the `Employee` object type may have an `employer ID` property (the foreign key) that refers to the `company ID` (primary key) of the `Company` object type.

API Reference ↗

In a many-to-many cardinality link type, a table containing pairs of objects represents the links between two objects. These link types require a table specified, along with mapping these keys that tell applications which columns in the

Send feedback

join table refer to the primary keys of which object types in the link type. For example, the join table backing the `Direct Report ↔ Manager` link type might contain pairs of `employee numbers`, for which each pair represents a `Direct Report ↔ Manager` link.

- **Display name:** The name shown to anyone accessing a link of this type in user applications. Each side of a link type has a display name. A side of a link type represents the link to that object type. For example, In the `Employee → Employer` link type, the display name for the `Employee` object type is `Employee` and the display name for the `Company` object type is `Employer`.
- **Plural display name:** The name shown to anyone accessing a link of this type with many linked object types in user applications. For example, In the `Employee → Employer` link type, the plural display name for the `Employee` object type is `Employees` and there is no plural display name for the `Company` object type, as there can only be one company per employee.
- **API name:** The name used when referring to the link type programmatically in code. The API name on a side of a link type can be used to return objects of that type. For example, if the API name on the `Employee` side of the `Employee → Employer` link type is `employee`, then calling `Company.employee.get()` will return the `Employee` objects linked to those `Company` objects. Read more about [API names](#).
- **Visibility:** An indication to user applications for how prominently to display the side of the link type (referring to links to the object type on that side). A `prominent` side of a link type will lead applications to show this side of the link type first to users. A `hidden` side of a link type will not appear in user applications. By default, the `Employee` and `Company` sides of the link type will have visibilities `normal`.
- **Type classes:** Additional metadata that are interpreted by user applications. Read more about [type classes](#).

[Learn more about creating and configuring a link type in the Ontology and about validation requirements for link type metadata.](#)

[Learn more about properties \(characteristics of an object type\).](#)

← PREVIOUS
Edit link types

NEXT →
Allow users to edit objects and links

API Reference ↗

© 2026 Palantir Technologies Inc. All rights reserved.

Send feedback

[Cookies Statement](#) ↗

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Allow users to edit objects and links

Allow users to edit objects and links

Editing data from Foundry object applications

You can allow users in user applications (like Workshop and Object Views) to edit property values, add and remove links, and create and delete objects. You can also configure side effects (like notifications) based on edits made by users.

The supported way to configure this functionality is to create and configure action types in the Ontology Manager. [Learn more about how to set up action types.](#)

The remainder of this documentation covers what needs to be configured on object types and link types before users can take actions.

Editing data from external applications

The [Objects API](#) provides endpoints for external clients to write and update objects, properties, and links with full permissions enforcement.

Set up the prerequisites

In order for a user to be able to take an action defined in an action type configuration, a writeback dataset must be created. The writeback dataset will read the edits made by users with the [Objects API](#) and will reflect the most up-to-date state of any given object.

[API Reference ↗](#)



that edits are written to the writeback dataset and not the original object type or link type. This ensures that users have access to both the original

Send feedback

data and the edited data in their analyses.

To set up a writeback dataset:

1. Navigate to the **Datasources** page of the object type or link type you want to enable edits on.
2. Select **Generate** in the **Writeback dataset** portion of the page to create a new writeback dataset. A dialog will open asking you to choose a Project where you'd like to place the dataset. Select a location.
3. Make sure the users who you want to be able to edit the object type or link type have edit permissions on the writeback dataset.
4. Ensure that the users who you want to be able to view changes made to the object type or link type have view permissions on the writeback dataset.
 - The ability to view objects and links is controlled by an object type and link type's backing datasources.
 - The ability to view the edits on objects and links is controlled by the permissions on the writeback dataset.
 - If users have access to only the former, they can see only the object as it exists without edits applied. If users have access to the latter, they can see both the edits and the object as it exists at present.

i If you want a property in your object type to capture manually-entered data from an end user (via an Action or other writeback method) and this data does not yet exist in Foundry, you will need to add an empty column to the object type's backing dataset and map it to a new property in your object type. You also need to enable edits; this is done by creating a writeback dataset in Object Storage v1 or by toggling on edits in Object Storage v2.

PREVIOUS

← [Link types / Metadata reference](#)

NEXT

[Value types / Overview](#) →

[API Reference ↗](#)

© 2020 Parantur Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Send feedback](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Value types

Value types are semantic wrappers around a [field type](#) that include metadata and constraints that can enhance type safety, improve expressiveness, and provide additional context. Value types encapsulate domain-specific data types and enforce data validation in a manner reusable across the platform. Unlike object types, properties, link types, or other types that define and build the Ontology, value types are associated with a [space](#) in the platform. A space can hold a single ontology. Value types can only be used within the space in which they were defined. Value types are not available for the Default ontology.

Dataset [field types](#) and property [base types](#) reflect the primitive types found in programming languages. These types are domain-agnostic and provide no domain context. By contrast, value types capture the context and semantic meaning of data and centralize data validation. Users define and consume meaning directly from the value type, rather than relying on surrounding information such as column names or property descriptions. Value types also enforce their validation constraints on data in Builder pipelines and the ontology, so data integrators and ontology managers can ensure proper semantic typing in their data flows and models.

For example, a user can define an “email” value type that has a regular expression constraint to ensure any property that uses the value type represents a valid email address. This value type can then be reused across multiple object types and pipelines without having to duplicate the validation logic for every such property. Additionally, each property that uses this value type is explicitly understood to contain an email address.

Since value types are intended for reuse across multiple pipelines and object types, they are [permissioned](#) to ensure users can apply them where needed and [versioned](#) to handle both breaking edits.

API Reference ↗

by learning how to [create a new value type](#) or [use an existing](#) property.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Value types / Create a value type

Create a value type

Follow the steps below to create a value type to use across your platform [space](#).

1. Navigate to the **Value Types Manager** application from the platform sidebar.
2. From the top left corner, use the dropdown menu to select the space in which you would like to create a value type.
3. Select **Create New Value Type** from the upper right corner.
4. Provide a clear name, description, and unique API name for your value type.

Create new value type

1 Metadata

2 Constraints

3 Preview

Metadata

Give your value type a searchable name, description and base type.

Name

Give your value type a unique name

Description

Describe what the value type does

API name

Give your value type an api name to allow references in code

Base type Once saved cannot be modified

String

Close Next

API Reference ▸ [API Reference](#) for your value type.

6 (Optional) Define a constraint for your value type. Validators can be used to enforce constraints on value types, enums, ranges, or other validation methods depending on the type of value type.

Send feedback

type. For a full list of constraints supported by base type, review our [value type constraints](#) documentation.

Create new value type

1 Metadata

2 Constraints

3 Preview

Constraint

Constraint type

☐ RID

☐ UUID

☐ Length

☐ Regex

☒ Enum

☐ Case insensitive

⋮

Salmon

×

⋮

Cod

×

Close

Back

Next

7. (Optional but recommended) Provide an example preview value for your value type.

Create new value type

1 Metadata

2 Constraints

3 Preview

Preview

Check inputs against your constraints

Value

Salmon

×

Preview

Salmon

Passing validation ✓

Close

Back

Create value type

API Reference ↗

Send feedback

8. Save your value type.

← PREVIOUS
[Overview](#)

NEXT →
[Use value types](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Use value types

Once you have [created a value type](#), you can use it in as a data type across Foundry. Value types can be supported for the use cases listed below.

- Assigning a value type to an object type property.
- Assigning a value type to a shared property.
- Assigning a value type to a Pipeline Builder pipeline property as a logical type using the `logical type cast expression` and selecting the value type on the property when you write to the objects target.

To assign a value type to a property, select the value type from the dropdown menu during property configuration.

The screenshot shows a configuration panel with the following elements:

- Type:** A dropdown menu with a clock icon and the text "Timestamp".
- Allow multiple:** A toggle switch with a question mark icon, currently turned off.
- Value type:** A dropdown menu with a calendar icon and the text "Values in the past", accompanied by a question mark icon.

[API Reference ↗](#)

value type to an object property that contains property values that fail
ion, that object type will fail to index. You can view such i

[Send feedback](#)

object type health status in Ontology Manager, where you can correct your data or update your value type to fix the issue.

PREVIOUS

← [Create a value type](#)

NEXT

[Value type versions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

—[Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Value type versions

Value types are versioned to handle breaking and non-breaking edits. Value type versions include two parts: metadata and constraints. The metadata values for name, description, and apiName can be changed whenever necessary. The base type metadata and the constraints that define the validation rules for the type are immutable.

If you choose to update the constraints of a value type, a new version of the value type is created. If your value type has no consumers, you can freely change these constraints. However, if you make breaking changes to the constraints and your value type has consumers, we recommend deprecating the current value type and creating a new one instead. This approach avoids potential runtime errors and data inconsistencies.

API Reference ↗



Send feedback

☒ Length

☐ Regex

☐ Enum

☒ Minimum size

☐ Maximum size

3

^

v

0

^

v

Failure validation message

Constraint failed

×

You are updating the constraints of this value type. Resources that depend on this value type (see Usage tab), might break if their values are incompatible with the new constraints. You may want to consider deprecating this value type and create a new one instead

When you make non-breaking changes to a value type, a new version is also created. This new version will automatically propagate to the Ontology, ensuring that all uses of the value type across the Ontology are updated to the latest version.

Value type permissions

Permissioning for value types is managed through platform [space](#). Any value types in a space are automatically imported and made available for the associated ontology. Other consumers can import the value types into their project scopes, similar to how users can import transforms profiles or inputs to pipelines.

Users who have View (read) permissions to a space can assign value types to property types or shared property types in that space and associated ontology. An Editor or Owner of a space can create, edit, or delete value types in that space.

PREVIOUS

← Value type versions

Value type constraints →

NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)



Send feedback

Value type constraints

Each value type may optionally define a constraint to enforce data validation. You can configure these constraints when [creating a new value type](#) in the **Value Type Manager** application. The available value type constraints, along with what base types they can be applied to, are below:

- **Enum (one of):** A constraint representing a static set of allowed values.
 - **Valid base types:** String, Boolean, Decimal, Double, Float, Integer, or Short.
 - For String properties, the enum values may optionally be case-sensitive or case-insensitive.
- **Range:** A minimum value, maximum value, or range of allowed values.
 - **Valid base types:** Decimal, Double, Float, Integer, Short, Date, Timestamp, String, or Array.
 - For String properties, the length of the string is constrained.
 - For Array properties, the size of the array is constrained.

Additionally, the following property types have additional type-specific constraints available:

- **String:**
 - **Regex:** A regex pattern that the string must match. The regex validation may optionally pass when matching only a substring of the property value.
 - **RID:** The string must be a valid rid.
 - **UUID:** The string must be a valid UUID.
- **Array:**
 - **Uniqueness:** All elements of the array must be unique.

API Reference ↗

Each value type constraint can be applied to the elements of the array. For example, a regex constraint could be applied to every string in an array.

Send feedback

- **Element constraints:** A mapping between a struct field identifier and a value type reference, where the struct field identifier indicates the struct component to which the referenced value type should be applied.

← PREVIOUS
Value type permissions

NEXT →
Metadata / Type classes

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

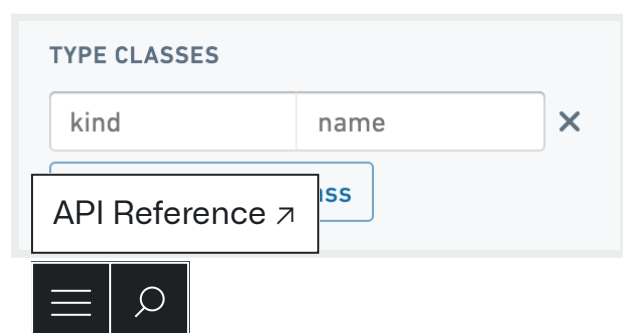
[Send feedback](#)

Type classes

Type classes can be applied to properties, link types, and action types. With the exception of the `analyzer` type class kind, which affects [indexing](#) behavior, type classes define additional metadata that can be interpreted by user applications that interact with the Ontology. For example, the specification of some `hubble` property type classes affect how the property value is rendered in [Object Views](#).

The chart below provides a list of known type classes. The columns in this chart are as follows:

- The **Deprecated** column indicates whether a type class is still supported.
 - The **Deprecated** column also indicates whether a type class should now be configured in the **Capabilities** page.
 - In the Ontology Manager, object types now have a **Capabilities** page to configure features historically defined as type classes. The configuration of all supported type classes will move to the **Capabilities** page.
- The **Type** column indicates whether the type class is applied to a property, link type (formerly known as relation), or action type.
- The **Kind** and **Name** columns contain string values from two user-defined fields that are set when adding a type class in the Ontology Manager. These values are used by Foundry products to label a type class.



Send feedback

- The **Description** column describes the intended behavior of user applications when interacting with a property value, link, or action for which the listed type class has been added.
 - [Object Explorer](#) is an application that consumes `hubble` type classes.
 - [Quiver](#) is an application that consumes `timeseries` type classes.

Deprecated	Type	Kind	Name
	Property	hubble	media_url
Deprecated	Property	hubble	editable
	Property	hubble	icon
Deprecated	Relation	hubble	creatable
Deprecated	Property	hubble	endorsement_status: endorsed
Deprecated	Property	hubble	endorsement_status: not_endorsed
Deprecated	Property	hubble	thumbnail

API Reference ↗

Send feedback

Deprecated	Type	Kind	Name
Deprecated	Property	hubble	array
Deprecated	Property	hubble	default_sort_descending
Deprecated	Property	hubble	quick_filter
	Relation	hierarchy	parent
API Reference ↗ ☰	Property	choropleth_map_config_id	<map_config_id>
			Send feedback

Deprecated	Type	Kind	Name

Deprecated	Property	oe_home_page_ object_type_group	<your_object_type_ group_name>
------------	----------	------------------------------------	-----------------------------------

API Reference ↗ 	Action type	hubble-oe	hide-action
			Send feedback

Deprecated	Type	Kind	Name
	Action type	hubble-oe-object-set-rid	<object_type_RID>
	Action type	hubble-oe-security-rid	<compass_RID>
	Action type	actions	generate_uuid
	Action type	actions	prefill_current_user
	Action type	actions	view_object_with_type
Deprecated	Property	analyzer	not_analyzed

Deprecated	Property	analyzer	simple / standard / whitespace / english / french / japanese / arabic / korean / german / combined_arabic_english
API Reference ↗	Property	analyzer	not_indexed
			Send feedback

Deprecated	Type	Kind	Name

DeprecatedPropertymultipassuser_id

Deprecated	Property	global	<your_property_id>
------------	----------	--------	--------------------

DeprecatedPropertygeogeojson

API Reference ↗

Deprecated	Property	geo	latitude	Send feedback
------------	----------	-----	----------	---------------

Deprecated	Type	Kind	Name
Deprecated	Property	geo	longitude
Configure in Capabilities page of object type	Property	geo	altitude
	Property	vertex	link_merge
	Relation	vertex	link_merge_incoming
	Relation	vertex	link_merge_outgoing
	Relation	vertex	component
	Property	vertex	component_subtype
API Reference ↗	Property	vertex	event_intent.<intent_>
			Send feedback

Deprecated	Type	Kind	Name
	Property	vertex	event_value
	Property	vertex	event_value_unit.<unit_>
	Property	vertex	event_property
	Property	vertex	min
	Property	vertex	max
	Property	vertex	threshold_measure.<measure_>
	Property	vertex	threshold_high_limit
	Property	vertex	threshold_low_limit
API Reference ↗	Property	vertex	threshold_exceed_intent. <intent_>

Send feedback

Deprecated	Type	Kind	Name
	Property	vertex	key_measure.<measure_>
Deprecated	Property	vertex	enum_values
Configure in Capabilities page of object type	Property	timeseries	timeseries_id
Configure in Capabilities page of object type	Property	timeseries	timeseries_measure
Deprecated	Property	timeseries	timeseries_sensor_type
Configure in Capabilities page	Property	timeseries	timeseries_units
API Reference ↗			
Configure in Capabilities	Property	timeseries	timeseries_i
Send feedback			

Deprecated	Type	Kind	Name
page of object type			
Configure in Capabilities page of object type	Property	timeseries	timeseries_root_object_id
Configure in Capabilities page of object type	Property	timeseries	timeseries_is_enum
Configure in Capabilities page of object type	Property	timeseries	timeseries_is_deprecated
	Relation	timeseries	parent
	Property	timeseries	timeseries_is_value_inverted
API Reference ↗	Property	timeseries	timeseries_depth_units
			Send feedback

Deprecated	Type	Kind	Name
Configure in Capabilities page of object type	Property	timeseries	event_id
Configure in Capabilities page of object type	Property	timeseries	event_start_time
Configure in Capabilities page of object type	Property	timeseries	event_end_time
Configure in Capabilities page of object type	Property	timeseries	event_description
Configure in Capabilities page of object type	Property	timeseries	event_root_object_id
Configure in Capabilities page of object type	Property	timeseries	event_linked_series_id
	Property	schedules	schedulable-start-time

Deprecated	Type	Kind	Name
	Property	schedules	schedulable-end-time

Property schedules segment-by

	Action type	schedules	schedulable-start-time
--	-------------	-----------	------------------------

Action type schedules schedulable-end-time

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

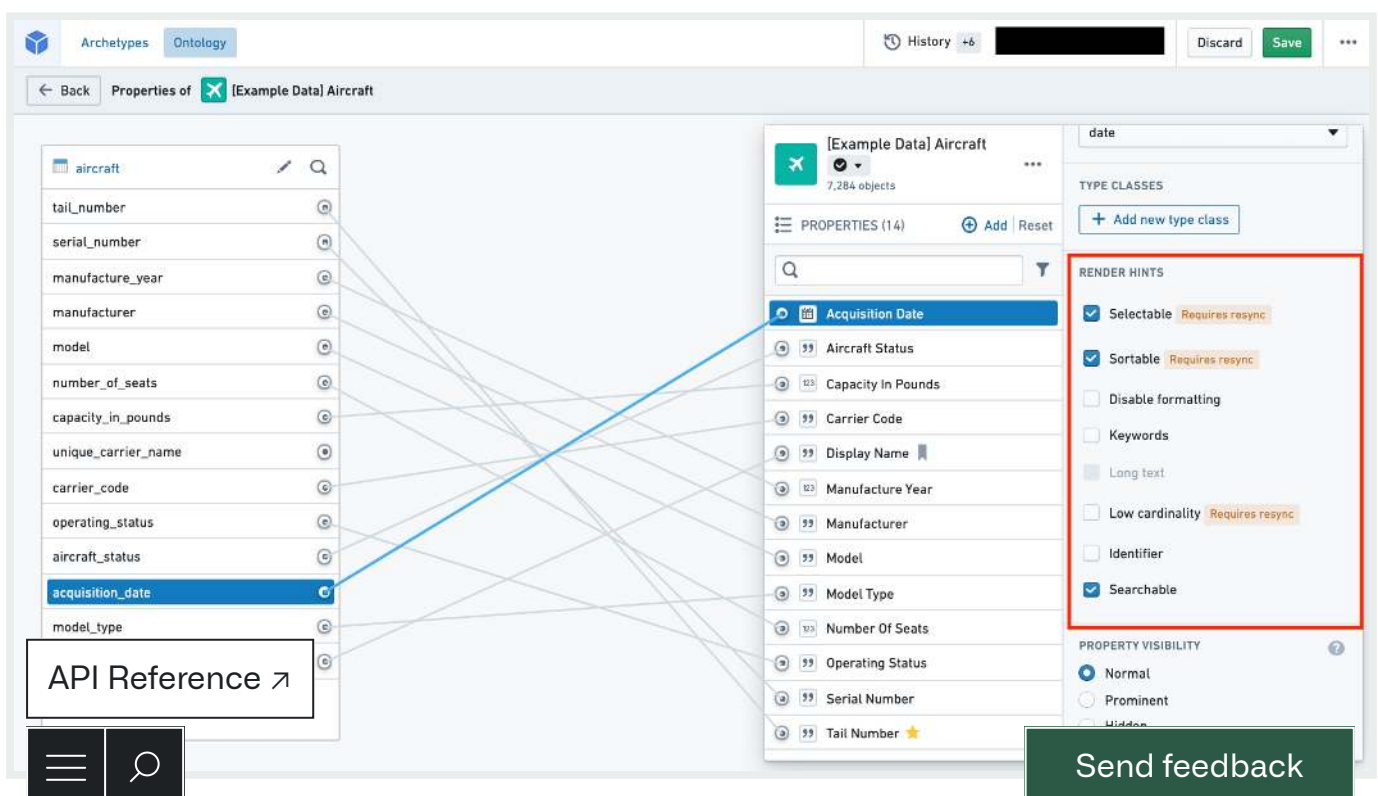
[Send feedback](#)

Render hints

Foundry uses **render hints** to communicate information about the use of Ontology [properties](#) to [Object Storage v1 \(Phonograph\)](#) and user applications in the platform. For example, the `sortable` render hint on a string property tells applications to allow users to sort on that property, as in a timeline or a chart.

Many render hints are tied to reindex performance for an object type. For instance, you can use render hints to indicate to [Object Storage v1 \(Phonograph\)](#) that a property does not need to be aggregated or sorted in applications, so that Object Storage v1 has less work to do when indexing those properties.

You can select and deselect render hints in the properties pane of the property editor (see image below).



The following table shares the **Name** and **Description** for each of the available render hints. The table also provides information on two technical aspects of render hints: "Adds raw index?" and "Requires reindex?" (described below).

- **Adds raw index?**
 - In order to apply a render hint that adds a raw index, Object Storage v1 (Phonograph) stores the render hint information by creating another index when storing the backing dataset.
 - Because of this additional index, for the property with a render hint applied, two columns will be counted toward the total number of columns indexed into Object Storage v1 (Phonograph).
 - This explains why deselecting these render hints improves performance of reindex into Object Storage v1 (Phonograph).
- **Requires reindex?**
 - Some render hints will be immediately applied in user applications as soon as their selection is saved in the Ontology Manager.
 - For other render hints that require a reindex, the object type's backing datasources must be reindexed into Objects Storage V1 (Phonograph) before the changes will be reflected in user applications.
 - You can wait for the next triggered reindex or you can manually start the reindex by navigating to the **Datasources** tab of the object type and selecting the blue **Reindex** button in the **Phonograph** pane.

Name	Description	Adds raw index?	Requires reindex?
Disable formatting	- Enable if property values should not be formatted in Object Views according to a browser location's local numerical formatting standards.		
Identifier	- Enable to improve reindex performance and specify primary keys and foreign keys that have a numerical base type and don't need to be formatted or treated as numbers.		
API Reference ↗	- For example, Object Views won't format the property values as numbers		
			Send feedback

Name	Description	Adds raw index?	Requires reindex?
	and Object Explorer won't enable filtering the keys by a range.		
Keywords	- Enable to highlight this property in its own section when displaying properties in Object Views.		
Long text	- Enable if property values contains a large amount of text. - For example, Object Views will display this property's values in a more readable format.		
Low cardinality	- Enable to indicate to applications that there are not many possible values for this property. - For example, some Object View widgets will only allow filtering on properties with not many possible values. - The Searchable render hint must also be selected along with Low cardinality.	yes	yes
Selectable	- Enable on string properties to allow users to perform aggregations on this property. - For example, this property will be aggregated in Object Explorer histograms and Object View charts. - Enable on numeric and date properties to allow users to perform aggregation on exact term values and not only distributions. - Disable to improve reindex performance if the property will not be aggregated in applications.	yes	yes
API Reference ↗ 	Enable to use the Exact Match filter capability.		
			Send feedback

Name	Description	Adds raw index?	Requires reindex?
	- The Searchable render hint must also be selected along with Selectable.		
Sortable	- Enable on string properties to allow users to sort on this property. - Numeric and date properties are always sortable. - For example, timelines and charts in Object Views will be sorted on this property. - Disable to improve reindex performance if the property will not be sorted on in applications. - Not recommended for arrays, which will sort based on the minimum value in the array. - The Searchable render hint must also be selected along with Sortable.	yes	yes
Searchable	- Disable to improve reindex performance if the property will not be searched or sorted on in applications. - The performance improvements will be especially significant if the property contains large strings. - Searchable must be selected in order for applications to apply the Selectable, Sortable, or Low cardinality render hints.	yes	yes
Enable leading wildcards	- Enable on string properties to support leading wildcard queries. - The Searchable render hint must also be selected along with Enable leading wildcards.	yes	yes
API Reference ↗ queries	- Enable on string properties to support regex queries. - The Searchable render hint must also	yes	yes

Send feedback

Name	Description	Adds raw index?	Requires reindex?
	be selected along with Enable regex queries.		

PREVIOUS
← Type classes

NEXT
Statuses →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

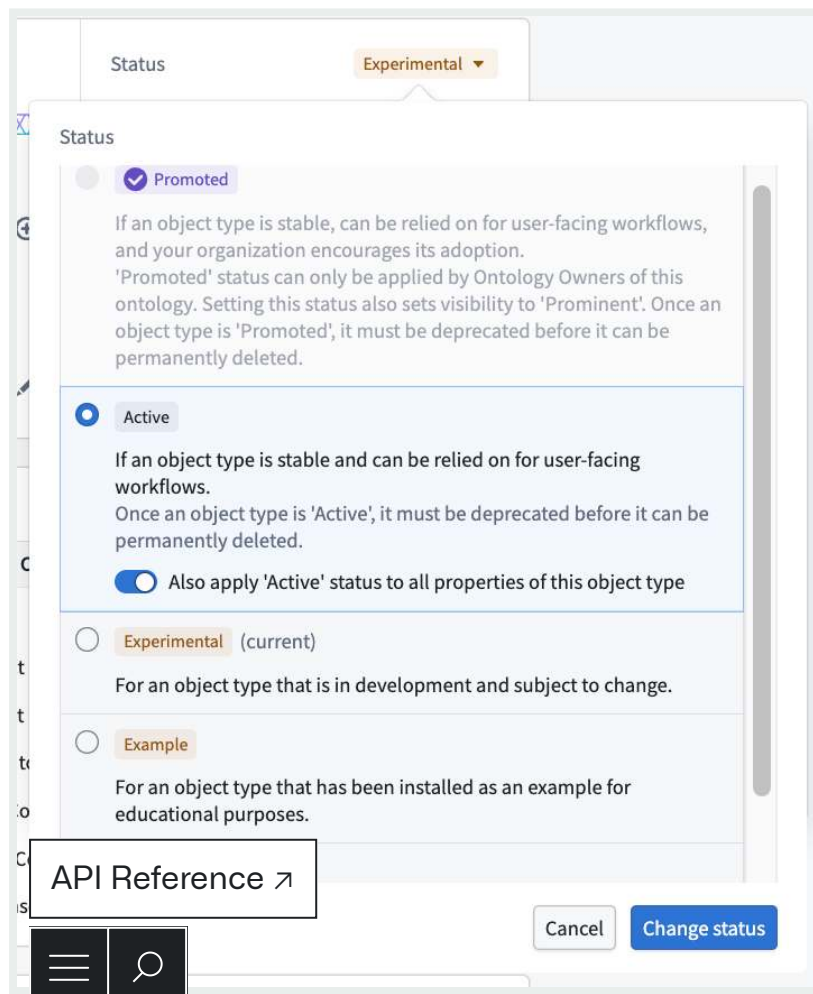
—Cookies Settings

[API Reference ↗](#)

Send feedback

Statuses

Every object type, property, link type, action, or interface in the Ontology has a **status** that indicates developmental state. An ontological resource's status can be either active, experimental, deprecated, or example; object types can also be classified as [promoted](#). Status metadata helps Ontology-editing users to know what resources are being actively relied on by user applications. These statuses are viewable in [Object Explorer](#), [Object Views](#), and [Workshop](#) to provide more information about which object types are intended for use in user applications.

[Send feedback](#)

The status can take on one of five values:

Available Status Values

- **Promoted (object types only):** Indicates that the object type is a core, trusted resource that has been vetted by an ontology owner. **Promoted** object types inherit similar protections as **active** object types for API names.
- **Active:** Indicates that the resource is actively in use in user-facing applications and major breaking changes will not be made in the Ontology Manager.
- **Experimental:** Indicates that the resource is still under development. Changes may be made that make the experimental item unavailable in user facing applications.
- **Deprecated:** Indicates that the resource will soon be deleted. The deprecated item should not be relied on in user facing applications.
 - A deprecated resource also has metadata that includes:
 - A description for why it is being deprecated;
 - A deadline for when it is expected to be deleted from the system; and
 - The resource that is meant to replace the one that is deprecated.
- **Example:** Indicates that the resource has been installed as an example. Example resources are notional and are only suitable for trainings or early-stage, exploratory use. Examples are *not* intended for use in production workflows.

Promoted status (object types only)

Object types support a special **promoted** status to signify a higher level of trust and official standing within the ontology. This status, represented by a new purple checkmark icon, helps users differentiate core, reusable object types from more use-case-specific or experimental ones.

The **promoted** status provides prominence beyond the standard **active** status. An object type with this status is meant to be considered a "core" resource, held to high standards and managed by a central team. **Promoted** object types inherit similar operational protections of the **active** status, such as restrictions on deletion.

Key features of the **promoted** status include:

[API Reference ↗](#)

[Send feedback](#)

- **Scope:** The `promoted` status applies only to object types. It is not available for properties, link types, action types or interfaces.
- **Visibility:** Setting an object type's status to `promoted` will automatically set its visibility to `prominent`, increasing its discoverability across the platform. Users can optionally move all properties of the object type to `active` status.
- **Permissions:**
 - Only users with the `Ontology Owner` role on the ontology level can directly apply the `promoted` status.
 - Other users must submit a proposal for review and approval by an `Ontology Owner` on the ontology level to apply the status.

Status

☒
Promoted

If an object type is stable, can be relied on for user-facing workflows, and your organization encourages its adoption.
'Promoted' status can only be applied by Ontology Owners of this ontology. Setting this status also sets visibility to 'Prominent'. Once an object type is 'Promoted', it must be deprecated before it can be permanently deleted.

☒
Also apply 'Active' status to all properties of this object type

☐
Active

If an object type is stable and can be relied on for user-facing workflows. Once an object type is 'Active', it must be deprecated before it can be permanently deleted.

☐
Experimental (current)

For an object type that is in development and subject to change.

☐
Example

For an object type that has been installed as an example for educational purposes.

☐
Deprecated

If an object type should no longer be used, and may be deleted in the future.

Operations that are not allowed

[API Reference ↗](#)
ns rely on ontological resources, there are several potentially destructive operations that are not allowed when a resource has the

Send feedback

- It cannot be deleted. A resource's status must be `experimental` or `deprecated` before it can be deleted.
- The API name of an active resource cannot be changed. Changing an API name is only possible for those marked as `experimental`.

Edit a status

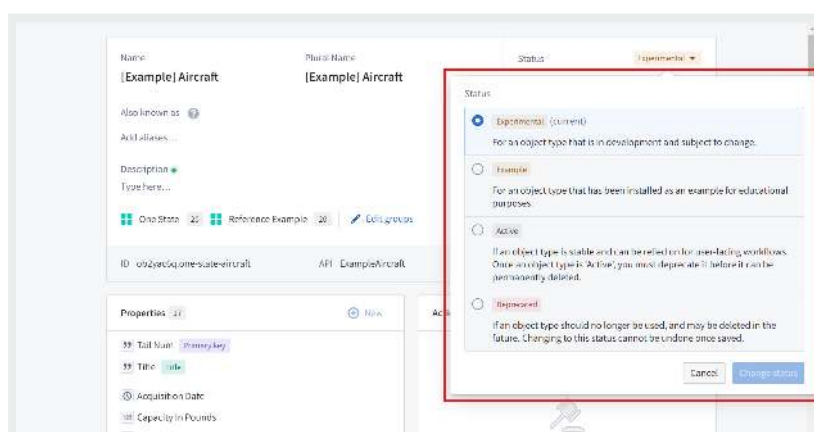
By default, any new ontological resource will be given the `experimental` status. To change the status:

1. Select the dropdown next to the current status.
2. Select the new status.

When changing a resource to the `deprecated` status, you will be prompted to:

- Fill out a description for why it is being deprecated,
- Input a deadline for when you expect it to be deleted from the system, and
- Optionally, select a resource that is meant to replace the one you are deprecating.

These statuses are viewable in Object Explorer, Object Views, and Workshop to provide more information about which object types are intended for use in user applications.



The **API Reference** ensures status consistency between an object type and its related properties. For example, if an object type is changed from `active` to `experimental`, all of its properties will be marked `experimental` as well.

[Send feedback](#)

The table below indicates available statuses for a link type between object types of different statuses. In general:

- If at least one object type in a link type is changed to `experimental`, the link type will automatically be changed to `experimental`.
- If at least one object type in a link type is changed to `example`, the link type will automatically be changed to `example`.
- If at least one object type in a link type is changed to `deprecated`, the link type will automatically be changed to `deprecated`.

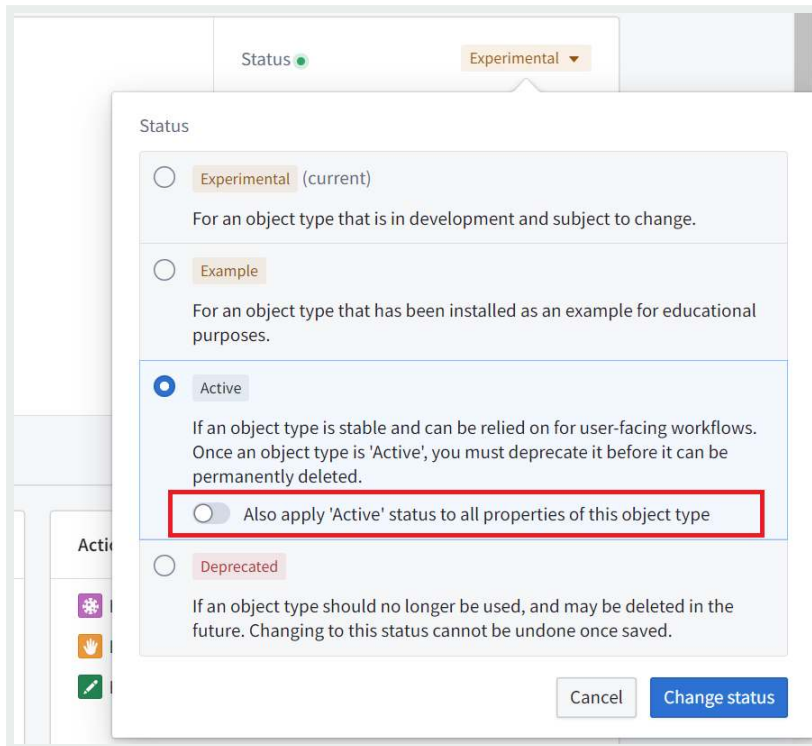
If object type A is... and object type B is...	
	EXPERIMENTAL
EXPERIMENTAL	experimental only
ACTIVE	experimental only
DEPRECATED	deprecated only

The same requirements are true of foreign keys of a link type. The application will change the status of a link type when changing a property:

- If a foreign key property is changed to `experimental`, its link type will be changed to `experimental`.
- If a foreign key property is changed to `example`, its link type will be changed to `example`.
- If a foreign key property is changed to `deprecated`, its link type will be changed to `deprecated`.

The application changes statuses in order to prevent invalid states. If a foreign key property is `experimental` and still being developed, its link type shouldn't be marked `active` and be relied on in production. In contrast, when marking a property `active`, the application won't change a link type referencing the property as its foreign key to `active`, as it is valid for a foreign key property to be in production, while the link type and its backing datasource are still in development.

When changing an object type from `experimental` to `active`, there is the option to also apply the `active` status to all properties on the object type:



When you change an object type to `example`, all of its properties will automatically become `example` also.

Statuses across properties of an object type can also be edited in bulk from the **Properties** page of the object type. [Read more about bulk editing properties.](#)

Object types

Statuses across object types can also be edited in bulk from the home page object view page by selecting the checkboxes of the object types to edit and selecting the **Edit status** button at the top right of the table.

[API Reference ↗](#)

[Send feedback](#)

3 Object Types Selected
Clear
Edit status
New object type

NAME	STATUS	VISIBILITY	ISSUES
☒ IPM Aircraft	Active	Normal	
☒ DEMO Aircraft Part	Experimental	Normal	
☒ Internal Training Aircraft	Active	Normal	

Troubleshooting

Conflicts between property status and link type status

If you receive the error

`OntologyMetadata:ConflictBetweenLinkTypeStatusAndPropertyTypeStatus` , there is a conflict between the status on a link type and the status on a property. For example, if a foreign key is `deprecated` , link types that reference that foreign key should also be `deprecated` .

Conflicts between object type status and link type status

If you receive the error

`OntologyMetadata:ConflictBetweenLinkTypeStatusAndObjectTypeStatus` , there is a conflict between the status on a link type and the status of one of its associated object types. This can happen when there is an invalid object type-link type case according to the table above. For example, an `experimental` object type cannot have an `active` link type.

Ontology building / Object and link types / Object type groups

Object type groups

Object type groups are a classification primitive that help users better search and explore their ontology. Groups are created and managed using Ontology Manager, generally by ontology [owners and editors](#).

Group configuration

Groups are created and managed via the groups menu, accessible in the Ontology Manager sidebar.

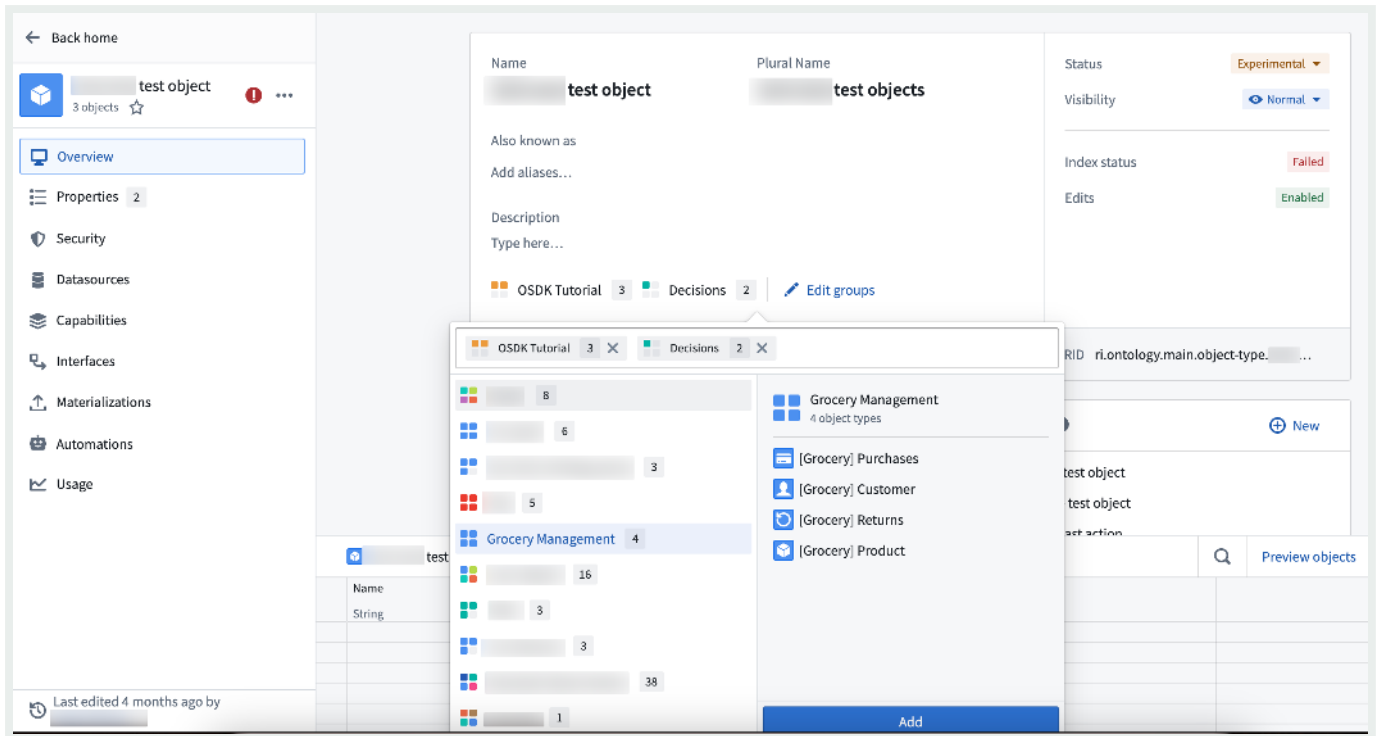
The screenshot displays the Palantir Ontology Manager interface. On the left, a sidebar lists various ontology components: Overview, Proposals, History, Semantic search, Resources, Object types (9,451), Properties, Shared properties (38), Link types (7,008), Action types (1,501), Groups (52), Interfaces (14), Functions (6,844), Health issues, and Cleanup. The 'Groups' menu item is highlighted. The main panel shows a table of groups with columns for Name and Number of objects. A 'New group' button is visible. On the right, a panel for 'Grocery Management' is open, showing 'Group content' with a list of items: Grocery Purchases, Grocery Customer, Grocery Returns, and Grocery Product. An 'Edit content' button is also present. A 'Send feedback' button is located at the bottom right of the interface.

Name	Number of objects
...	10
...	2
...	1
...	35
...	10
...	2
...	1
...	1
...	3
...	1
...	5
...	3
...	58
...	4
Grocery Management	4
...	8
...	2

[API Reference ↗](#)

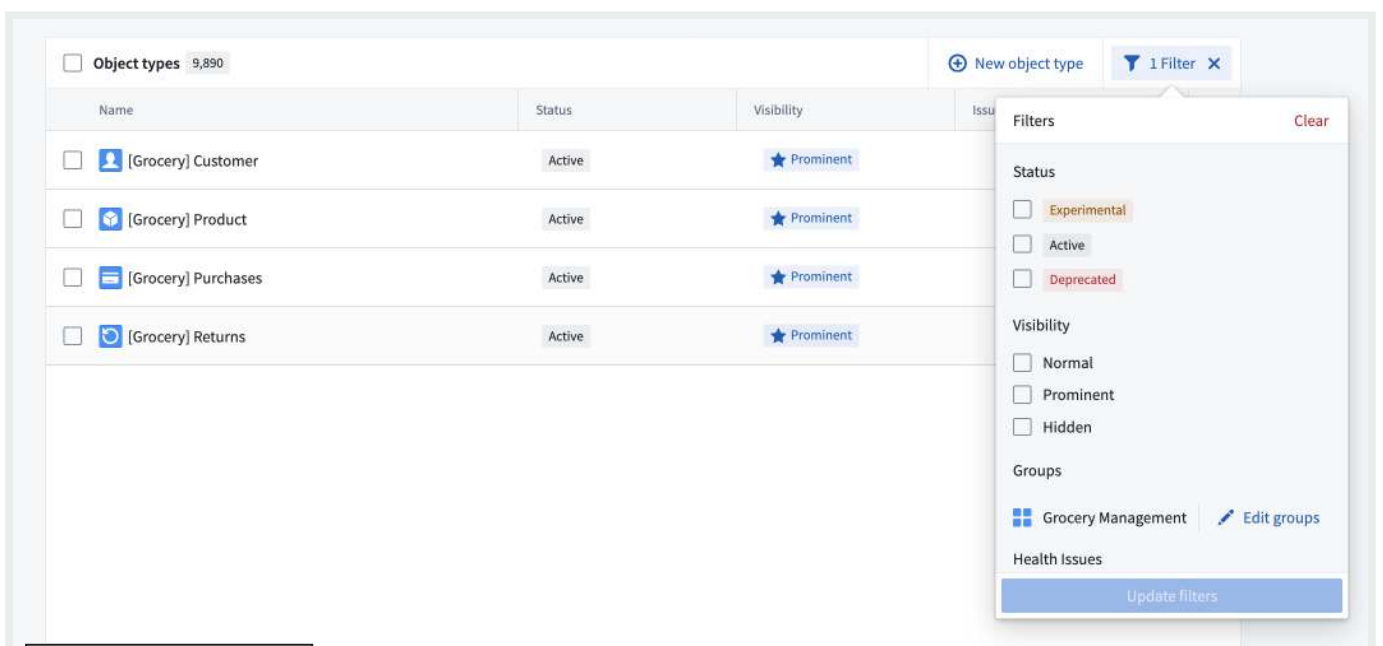
Groups can also be added directly to object types by selecting **Edit group** from the object type overview page.

[Send feedback](#)



Group search and discovery

Groups are searchable in [Ontology Manager's Search bar and Search bar dialog](#). The table of object types in Ontology Manager supports displaying and filtering by group. Groups are also displayed on the [Object Explorer home page](#).



[API Reference ↗](#)

Group permissions

[Send feedback](#)

To view object type groups, users must have **viewer** permission on the project that the object type group is in.

Legacy group migration

As of May 22, 2024, the *group* primitive described on this page has replaced the tag-based system of legacy groups.

In most cases, legacy groups were automatically migrated to object type groups at this time. Ontology owners were notified via an Upgrade Assistant intervention if manual action was necessary.

Group name visibility

Previously, if all object types inside a group were non-discoverable to a certain user (for example, due to access controls on backing datasets), the group was also non-discoverable to the user. As mentioned in the section above on [group permissions](#), all groups will now be discoverable to any user that can view the ontology. This change aligns group visibility with other [ontology primitives](#) to increase clarity and transparency in governance.

Migration of partially visible groups

Legacy groups that were not discoverable to one or more users were not eligible for automatic migration. In these cases, ontology owners were notified via an Upgrade Assistant intervention that manual action was necessary.

On May 22 2024, legacy groups that could not be safely migrated were hidden from operational users across all applications such as Workshop and Object Explorer. To provide backward compatibility, the names of legacy groups remain stored as [type class metadata](#) on object types.

Ontology owners may continue to manually migrate these hidden, legacy groups using the [API Reference](#) to do this, navigate to the **Ontology Configuration** menu in the bottom left corner and select **Approve all Groups for migration**.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[—Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Object and link types / Add Ontology types to a Marketplace product

Add object and link types to a Marketplace product

Use [Foundry DevOps](#) to include your object and link types in [Marketplace products](#) for other users to install and reuse. [Learn how to create your first product.](#)

Unsupported features

Most [object property types](#) are supported in Marketplace products, but the following are not yet available:

- [Cipher](#)
- Geo time
- Vector

Marketplace products do not yet support the following:

- Object types with streaming datasources
- Object types with no datasource

Note that objects themselves cannot be packaged with Marketplace. This means that, for example, object edits made by Actions cannot be packaged into a Marketplace product. However, datasets and object types can be packaged in order to create new objects after installing the Marketplace product.

[API Reference ↗](#)

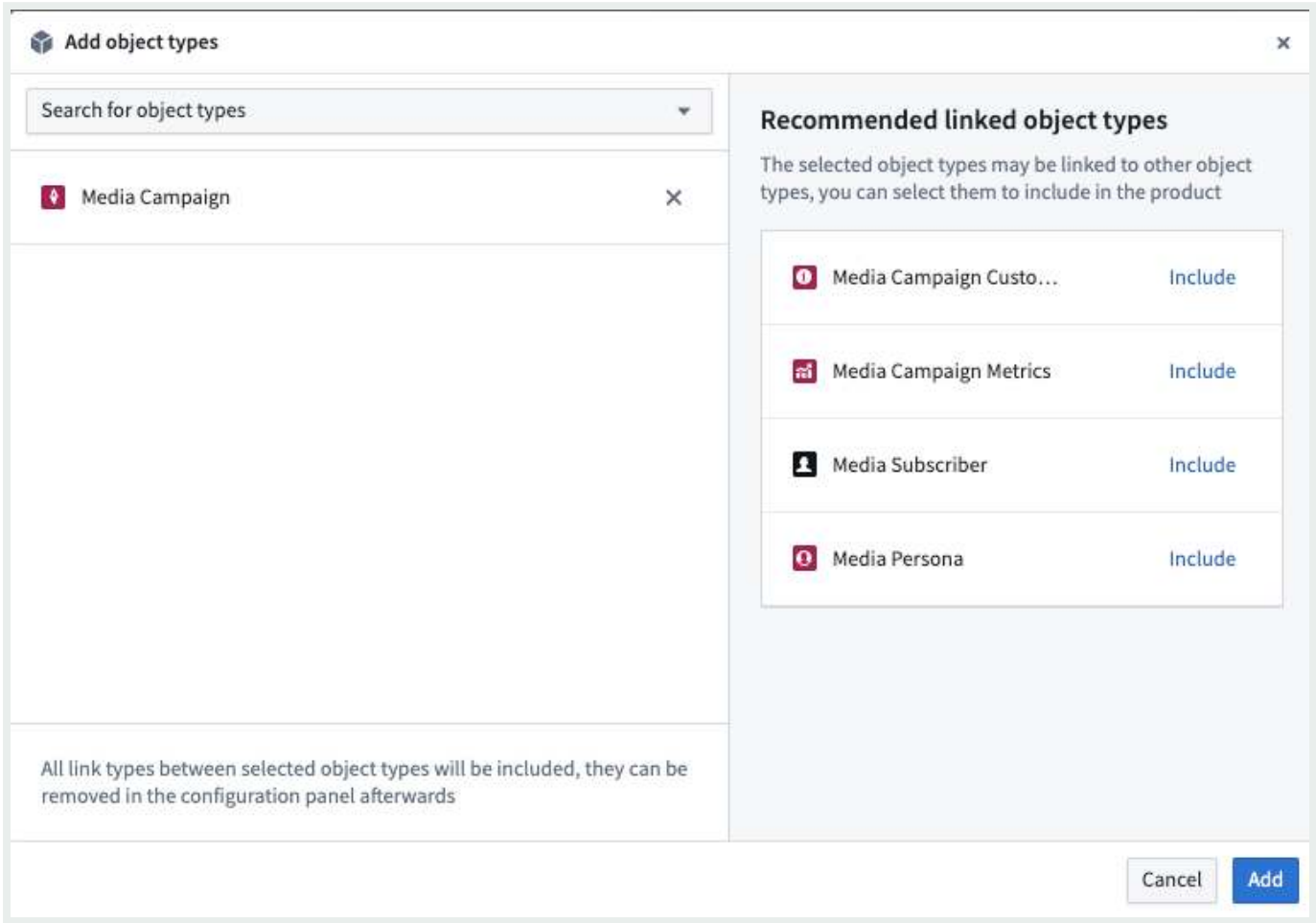
If you need support for any of the above, contact your Palantir representative.

[Send feedback](#)

Add object types to products

To add an object type to a product, first [create a product](#). [Add outputs](#) and then select the **Add ontology entities** option.

You will then be prompted to choose an object type. After selecting an object type, you will see recommendations for linked object types that you may want to add to your product.



Add link types to products

To add a link type to a product, first [create a product](#) and then select the **Link type** content type.

You will then be prompted to choose a link type as below.

API Reference ↗

Send feedback

 Add link types

Choose a link type

 Campaign to Feedback

media-customer-feedback-to-campaign

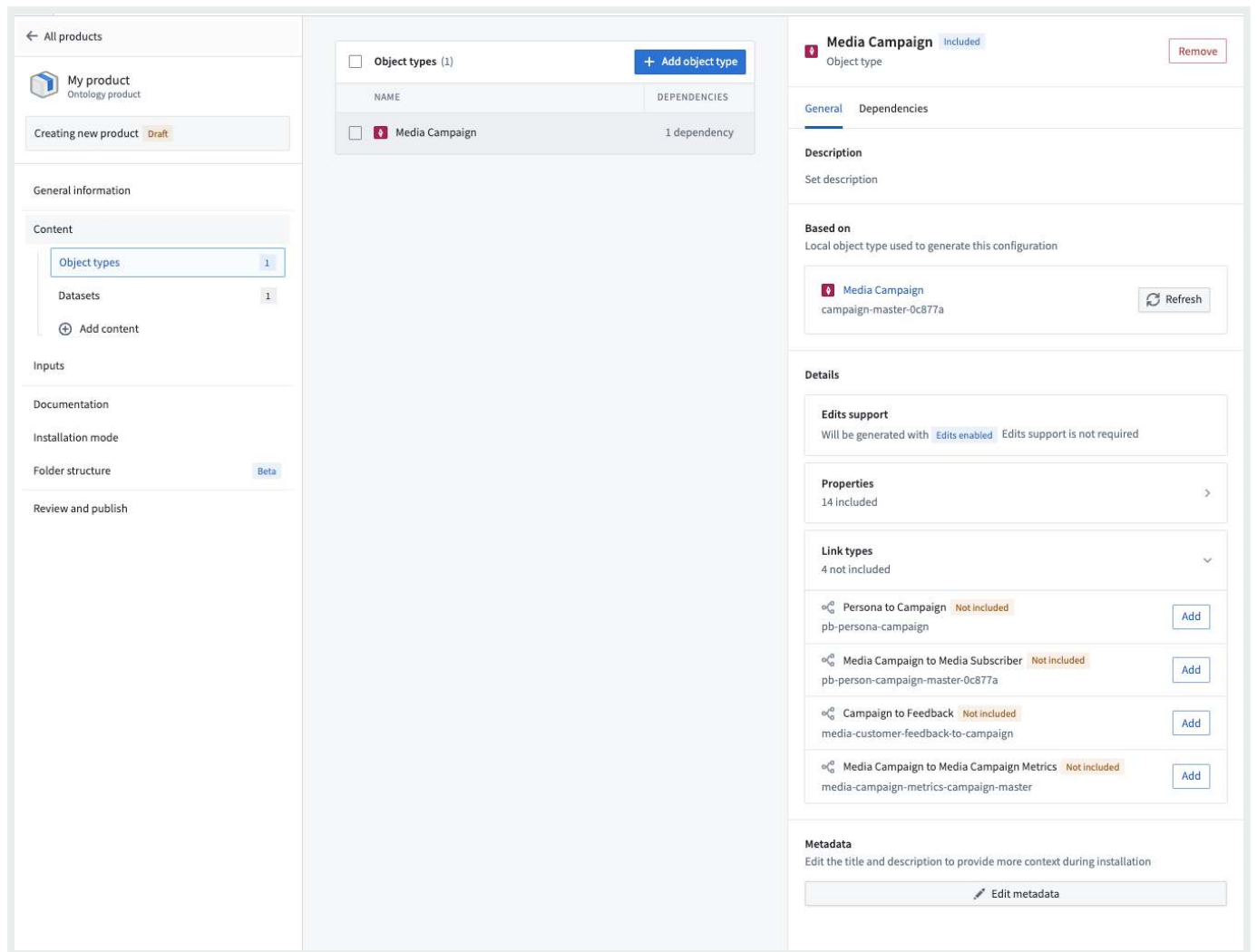
Cancel

Add

While you can select link types directly, we recommend first adding your object types and then selecting relevant links via the [information panel](#) as below.

[API Reference ↗](#)

[Send feedback](#)



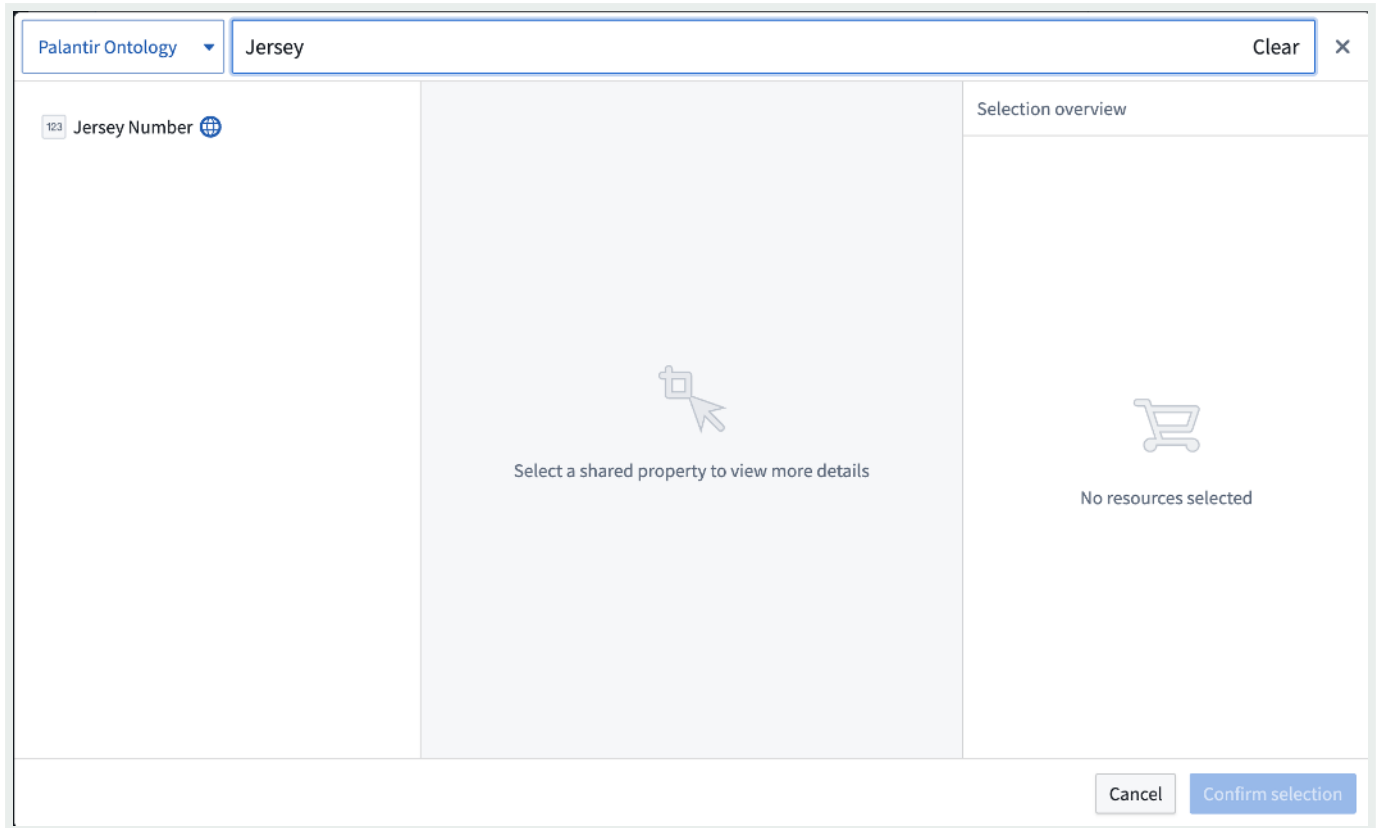
Add shared properties to products

To add a shared property type to a product, first [create a product](#). Then, select the **Shared property** content type as shown below.

You will then be prompted to choose a shared property.

[API Reference ↗](#)

[Send feedback](#)



Add interface types to products

To add an interface type to a product, first [create a product](#). Then, select the **Interface** content type as shown below.

You will then be prompted to choose an interface.

[API Reference ↗](#)

[Send feedback](#)

Palantir Ontology

Athlete

Clear

X

Athlete

Professional Athlete

Athlete

Experimental

Select

Good at sportsball!

Properties

123

Jersey Number

”

Name

Selection overview

No resources selected

Cancel

Confirm selection

Properties

A **property** of an [object type](#) is the schema definition of a characteristic of a real-world entity or event. A **property value** refers to the value of a property on an object, or a single instance of that real world entity or event.

For example, in the Ontology Manager, an `Employee` object type may have properties `employee number`, `start date`, and `role`. The notional employee “Melissa Chang” may have property values “11502” for `employee number`, “October 9, 2016” for `start date`, and “software engineer” for `role`.

Similarly, in the Ontology Manager, a `Flight` object type may have properties `departure date`, `arrival date`, and `passenger count`. The object “JFK → SFO 24-02-2021” may have property values “24-02-2021” for `departure date`, “25-02-2021” for `arrival date`, and “150” for `passenger count`.

The concepts underpinning the Ontology have analogous concepts in the structure of a dataset. The definition of a property in the Ontology is analogous to that of a column in a dataset, while the definition of a property value is analogous to that of a field in the dataset. For example, an `Employee` dataset may have columns for `departure date`, `arrival date`, and `passenger count`. In this case, a single field will have the value “11502” for the `employee number` column of the row for employee “Melissa Chang.”

Rather than being an abstract data model, the Foundry Ontology maps each ontological concept to an organization's actual data, enabling this data asset to power real-world applications. Property values are created and displayed in user applications by adding backing datasources to an object type in the Ontology Manager. To create property values for `employee number`, `start date`, and `role` on objects of type `Employee`, an organization adds backing datasources to the `Employee` object type and feed their enterprise directory and other enterprise data into the Ontology.

API Reference ↗



Send feedback

Supported property types

Property base type	Valid as title key?	Valid as primary key?	Notes
Commonly used: String, Integer, Short	Yes	Yes	
Time-based: Date, Timestamp	Yes	Discouraged	Typically, time values are inappropriate as primary keys, due to potentially unexpected collisions / uniqueness based on the storage format differing from the display format. In most cases, we recommend using String instead.
Number-like: Boolean, Byte, Long	Yes	Discouraged	Boolean limits your object type to two object instances. Byte properties can only be assigned in Actions via an Integer parameter, so in most cases we recommend using Integer properties instead. Long has representational issues in Javascript ↗ , so not all frontend libraries and code work well with Long values greater than 1e15. In most cases, we recommend using String instead.
Float-like: Float, Double, Decimal	Yes	No	
Vector	No	No	
Array	Yes	No	Array properties cannot contain null elements. If the inner type of the Array is not a valid title property, the array also cannot be used as the title property.
API Reference ↗ 			Send feedback

Property base type	Valid as title key?	Valid as primary key?	Notes
			Nested arrays are not supported in Object Storage v2.
Struct	No	No	Struct properties do not support nesting, and fields cannot be arrays. See the struct documentation for detailed information on supported field types.
Media Reference, Time Series, Geotemporal Series, Attachment	No	No	
Geopoint	Yes	No	Geopoint values are stored as a comma-separated string in the format latitude, longitude (for example, 57.64911,10.40744). See also Use geospatial data in the Ontology , Create GeoPoint , Convert to Ontology GeoPoint , and Is valid Ontology GeoPoint .
Geoshape	No	No	
Marking	No	No	
Cipher	Yes	No	

To learn more about base types, see [base types](#).

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)